MotorAnalysis-PM

Design and Analysis of Permanent Magnet Machines

Version 1.1

User Manual

Vladimir Kuptsov

2018

CONTENTS

1. INTRODUCTION

Thank you for your interest in MotorAnalysis-PM.

MotorAnalysis-PM is a free software for design and analysis of permanent magnet machines including brushless DC and permanent magnet synchronous machines with surface-mounted and interior magnets. MotorAnalysis-PM is based on automated finite element analysis (FEA) simulations and establishes a complete set of tools for design and analysis of permanent magnet machines. MotorAnalysis-PM is available as a MALTAB-based application and standalone version working without MATLAB.

We believe that our work will help to save our environment supporting green technologies like electric vehicles or wind turbine energy generation as well as enhancing the efficiency and effectiveness of electric machines.

2. BRIEF THEORETICAL BACKGROUND

The purpose of this section is to give the user a brief description of the problem examined. This section contains the formulation of the problem, short description of the nonlinear solver organization, rotation of the finite element mesh, torque and force calculation and etc. This information is critical for proper application adjustment and getting desired results.

2.1. General description.

The magnetic field problems for a permanent magnet machine can be solved using a two-dimensional approximation, which is based on the assumption that the magnetic field does not depend on z-coordinate (*z*-axis being parallel to the axis of the rotor shaft). Thus, the magnetic field is solved in the plane of the machine's cross section (*x*-*y* plane). The current density and magnetic vector potential in two-dimensional problems only have z-components and can be expressed as follows:

$$\nabla \times \left(\frac{1}{\mu_r} \cdot \nabla \times A \right) = J \tag{2.1}$$

$$J = \sigma \cdot \frac{U}{l} - \sigma \cdot \frac{\partial A}{\partial t}, \tag{2.2}$$

where $\boldsymbol{A}$ – magnetic vector potential ($\boldsymbol{B} = \nabla \times A$, $\boldsymbol{B}$ – magnetic flux density), μ_r – relative magnetic permeability, J – current density, σ – conductivity, U – voltage applied to the FE area, l – length in z direction.

There are two methods used in MotorAnalysis-PM to solve the problem defined by Exp. (2.1) and (2.2). First method is called a *magnetostatic finite element method* (FEM) which is used in the Magnetostatic Analysis. For the magnetostatic FEM the current density J is assumed to be time-independent so only Exp. (2.1) is used to define the magnetostatic problem. Note that the magnetostatic FEM can only be used for analysis of steady-state regimes.

Another method used in MotorAnalysis-PM is called a *transient finite element method* which is used in the Dynamic FE Analysis. According to Exp. (2.2) the current density consists of two components, the first one is caused by external voltage and the second one is induced by the varying magnetic field. The idea behind the transient FEM is a representation of the time derivative of the magnetic vector potential as follows:

$$\frac{\partial A}{\partial t} = \frac{A_n - A_{n-1}}{\Delta t},$$

where Δt – time step, A_n and A_{n-1} - magnetic vector potentials on n-th and (n-1)-th time steps.

Determining the initial magnetic vector potentials with magnetostatic FEM and assuming zero initial stator currents the iterative procedure can be used to find the magnetic vector potentials for each time step. This method allows one to take into account induced eddy currents, nonlinearity of iron and the rotation of the rotor and can be used for analysis of both transient and steady-state regimes.

In both methods the discretization of the problem over the plane of the machine's cross section using FEM results in the system of linear equations written in the matrix form as follows:

$$K \cdot X = F, \tag{2.3}$$

where K – stiffness matrix, X – vector of unknowns, F – right side vector of the problem. For the magnetostatic FEM the unknowns are the magnetic vector potential values in mesh nodes. For the transient FEM the vector of unknowns also includes values of stator currents and voltages.

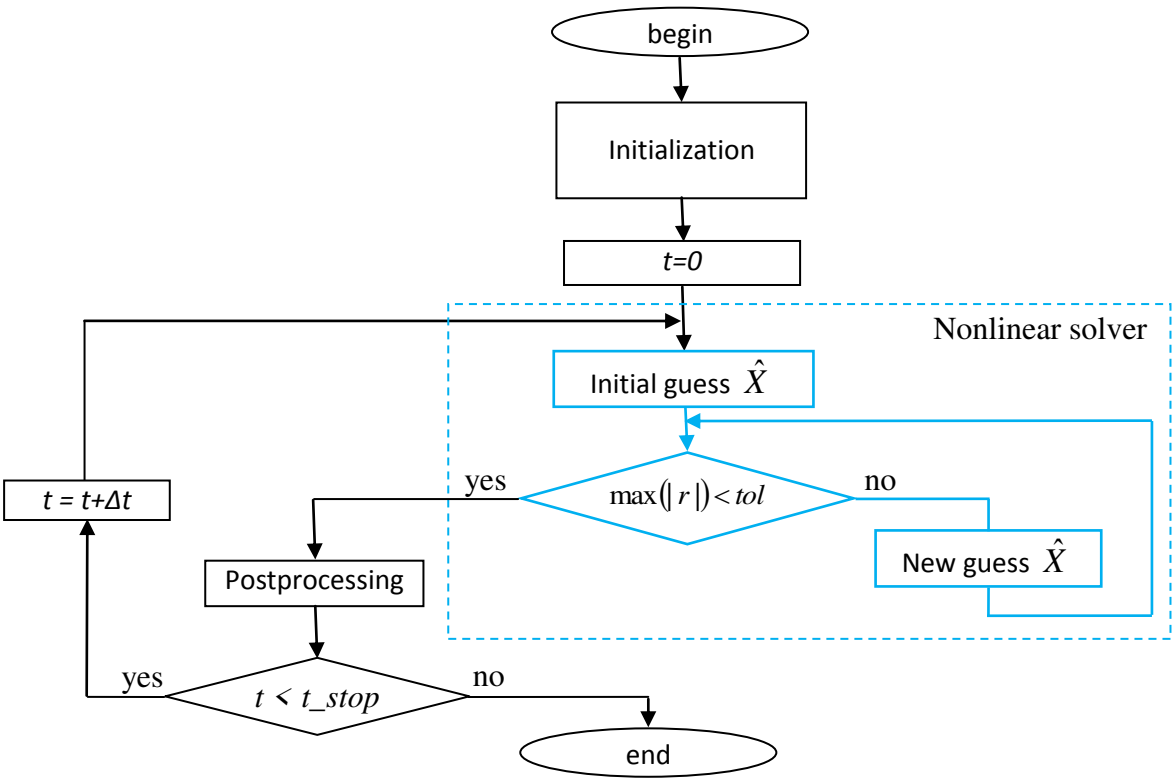


Figure 2.1. Generalized algorithm of solving a nonlinear problem with time-stepping FEM.

To solve the problem defined by Exp. (2.1) and (2.2) with a nonlinear B-H relationship the Gauss-Newton method is used. Assuming that we have a guess $\hat{X}$ of the solution and stiffness matrix $\hat{K}$ corresponding to the guess, then the residual vector of the guess $\hat{X}$ will be defined as:

$$r = \hat{K} \cdot \hat{X} - F \tag{2.4}$$

Solving system (2.3) using Gauss-Newton iteration tends to be the minimization of the residual. The problem is said to be solved if the certain condition is satisfied. In MotorAnalysis-PM this condition is defined by the maximum absolute value of the residual vector:

$$\max(|r|) < tol, \tag{2.5}$$

where *tol* – convergence tolerance defining the desired accuracy of the solution.

Both methods described above use an iterative time-stepping procedure to solve a time-varying magnetic field problem. Generalized algorithm used in MotorAnalysis-PM for a nonlinear case of the time stepping method is shown in Figure 2.1. An outer loop of the algorithm represents integration over time with step Δt. The magnetic field problem is solved on the every integration step in the nonlinear solver loop (marked with blue color). Gauss-Newton iteration continues until the inequality (2.5) is satisfied.

2.2. Finite element mesh.

In MotorAnalysis-PM the first-order triangular finite element mesh is used.

Every time while the time-stepping FE simulation is in progress when the rotor position is changed the mesh should be also changed. It is implemented by rotation of the rotor mesh connected to the fixed stator mesh via *sliding layer* located in the air gap. In MotorAnalysis-PM the air gap always has odd number of mesh layers (3, 5, 7 or 9) and the sliding layer is always located at the centre of the air gap (see Figure 2.2).

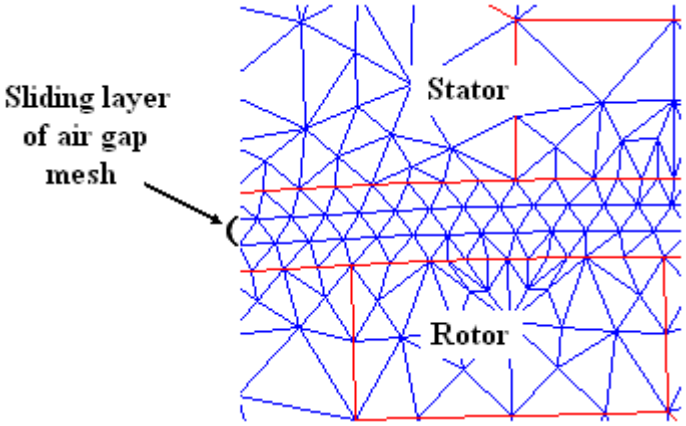


Figure 2.2. Three mesh layers in the air gap and the sliding layer at the center of the air gap.

To reduce number of finite elements and as a consequence reduce the computation time, periodic/antiperiodic boundary conditions can be applied. Using this type of boundary conditions is based on periodicity of the magnetic field of the machine.

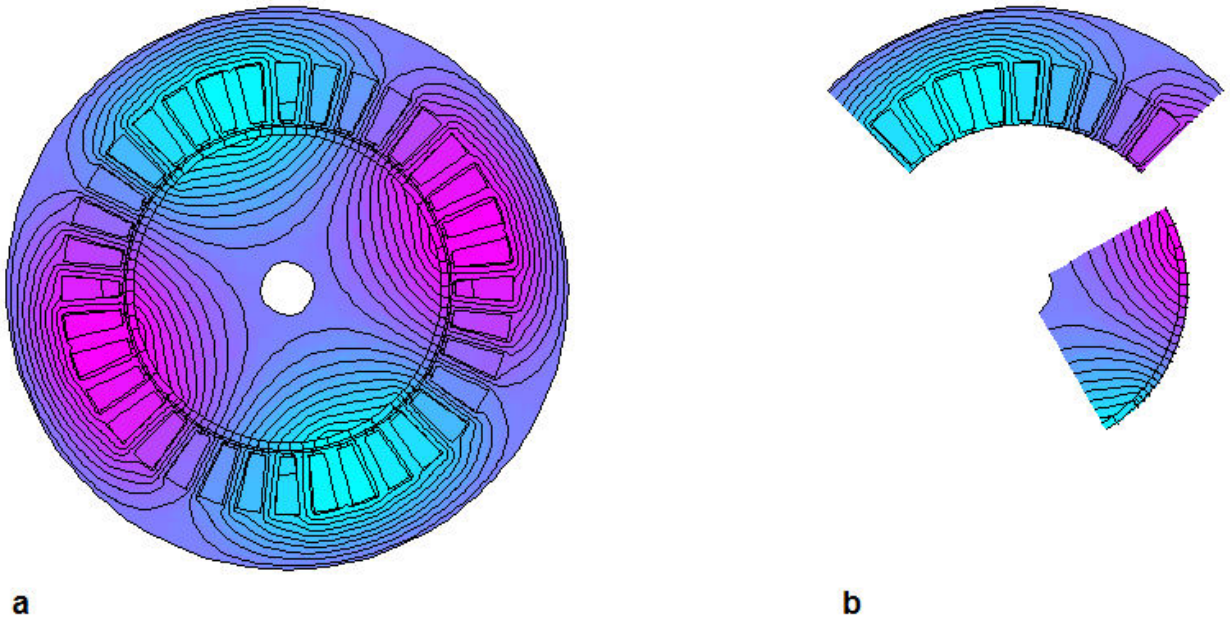


Figure 2.3. Using boundary conditions.

As it is seen from the example of four pole surface-mounted permanent magnet motor shown in Figure 2.3(a), the magnetic vector potentials repeat two times along the circle. Moreover, absolute values of the magnetic potential values repeat four times. Using this property of the field we can compute magnetic potentials for one quarter and then find the magnetic field for the whole cross-section as shown in Figure 2.3(b). In this case we use *antiperiodic* boundary conditions. If the magnetic potentials would be computed for the half, when *periodic* boundary conditions were applied. Number of periodicities of the magnetic field in MotorAnalysis-PM is called *periodicity factor*. Periodicity factor basically equals to a number of pole pairs; in the presented example the periodicity factor is 2. Note that using periodic or antiperiodic boundary conditions are only possible if geometry of the cross section repeats together with the field. In practice it means that to apply periodic boundary conditions the number of stator slots should be divided by number of pole pairs, while for antiperiodic boundary conditions the number of stator slots should be divided by number of poles.

2.3. Calculation of electromagnetic torque and force.

There are two methods used in MotorAnalysis-PM for calculation of the torque. These are the Maxwell stress tensor method and virtual work method. For calculation of the radial force acting between the stator and rotor the virtual work method is used.

According to the Maxwell stress tensor method the electromagnetic torque for the inner rotor can be expressed as the following:

$$T = -\frac{l \cdot \left(D_r + l_\delta\right)^2}{4\mu_0} \cdot \int_0^{2\pi} B_n B_t d\phi \quad (2.6)$$

,

where T – electromagnetic torque, l – lamination length in z direction, D_r – rotor diameter, l_δ – air gap length, μ_0 – permeability of the free space, B_n и B_t – normal and tangential components of the magnetic flux density in the air gap, as shown in Figure 2.4. This expression is used in MotorAnalysis-PM for calculation of the torque with the Maxwell stress tensor method.

The integration contour used in Exp. (2.6) always lies inside the sliding layer of the air gap mesh, as shown in Figure 2.4.

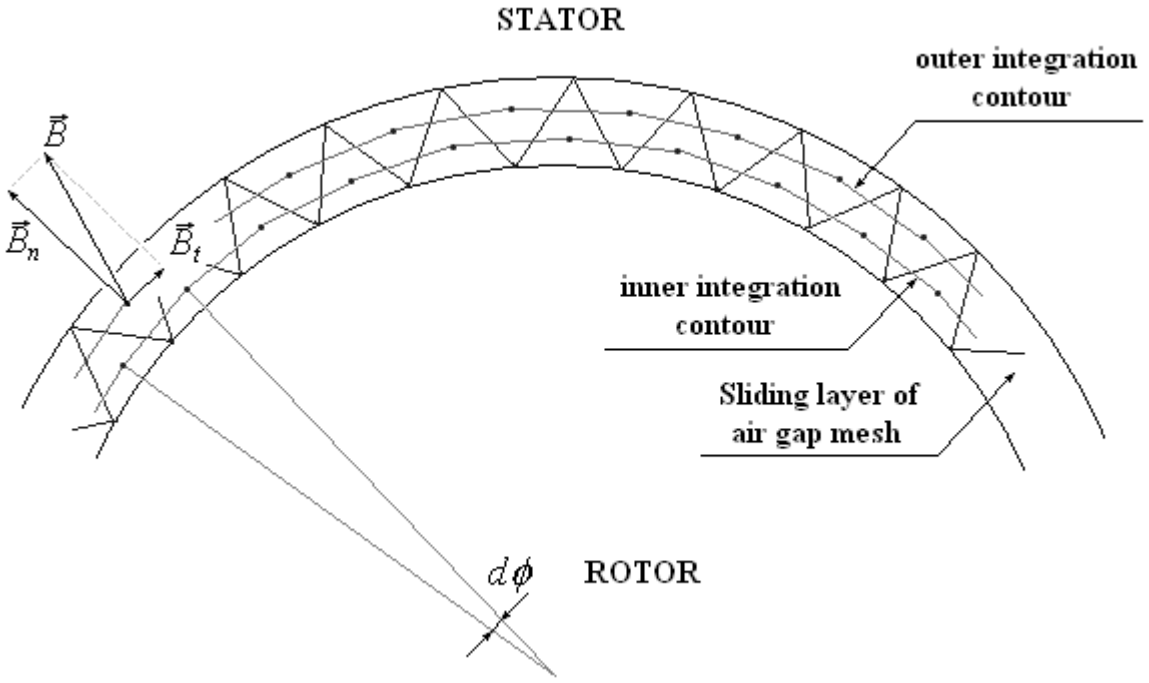


Figure 2.4. Calculation of the electromagnetic torque with the Maxwell stress tensor method.

Only the sliding layer of the air gap mesh is shown. Two integration contours are used - the inner contour passing through midpoints of the rotor-oriented triangles and the outer contour passing through midpoints of the stator-oriented triangles. The actual value of the torque is resulted as the average of two values calculated over the inner and outer integrated contours.

Source code for calculation of the electromagnetic torque with the Maxwell stress tensor method is presented in file MaxwellTorque.m.

According to the virtual work principle, the electromagnetic force is given by the derivation of the magnetic energy with respect to the virtual displacement with constant flux linkage:

$$F_u = -\left.\frac{\partial W}{\partial e}\right|_{\psi=const},$$

where ∂e - virtual displacement, ∂W - change of the magnetic energy between initial and final positions of the rotor.

Similarly, the expression for the torque calculation is defined as:

$$M = -\left.\frac{\partial W}{\partial \phi}\right|_{\psi=const},$$

where $\partial \phi$ - virtual angular displacement.

When the virtual work principal is applied for the torque and force calculation, the accuracy significantly depends on the choice of the virtual displacement value. On the one hand the displacement should be small enough not to distort the finite element mesh. On the other hand, the round-off error arises when the displacement value is too small. The optimal value of the virtual displacement is not provided automatically and should be defined by the user in menu **File -> Settings**.

It is considered that the virtual work method is more accurate comparing with the Maxwell stress tensor one since the first one implies the integration over the whole machine's cross section while using the Maxwell stress tensor method involves only finite elements of the air gap. Though in most cases the difference between two methods is not significant and the Maxwell stress tensor method is usually used.

When the time-stepping FEM is applied the calculated torque value is used to determine the current rotor position. The rotor rotation is defined as follows:

$$T = T_{load} + J\frac{d\omega}{dt}$$

$$\omega = \frac{d\varphi}{dt},$$

where T_{load} – load torque on the motor shaft, J – combined rotor and load moment of inertia, ω - rotor angular speed, φ – rotor angular position.

2.4. Multi-slice FEM.

As it was previously assumed the magnetic field of a permanent magnet machine does not depend on z-coordinate. In reality this assumption is not quite correct because of the end-winding leakage flux and rotor or stator skewing. In 2D FEM the end-winding leakage flux can be taken into consideration with satisfactory accuracy using end-winding inductance. But the two-dimensional approximation is of no help when it comes to skewed geometries. Rotor and stator skewing can be accurately simulated with 3D FEM. But these simulations are usually long. As an alternative, multi-slice FEM is used in MotorAnalysis-PM. Figure 2.5 illustrates the multi-slice approach.

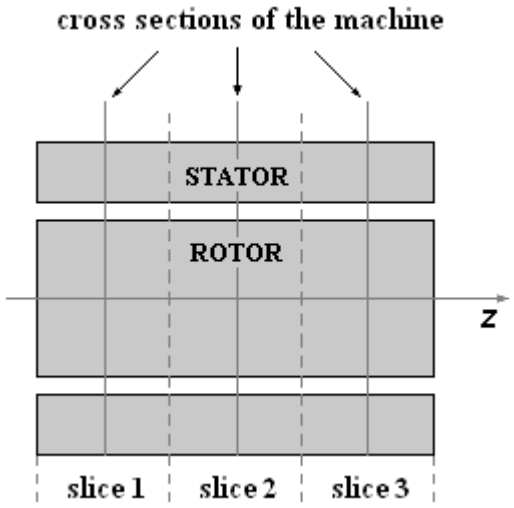


Figure 2.5. The multi-slice FEM principle.

The multi-slice FEM principle consists in dividing the machine along the z-coordinate into several slices. The cross section geometry changes from one slice to another according to the applied skew angle. Then the magnetic field problem is solved in every slice all at the same time with 2D FEM. Electromagnetic torque is calculated for every slice and the actual torque is obtained as summation of all slice values.

In MotorAnalysis-PM the multi-slice approach is used for both magnetostatic FEA and transient FEA (Dynamic FE Analysis).

Note that the calculation time is proportional to the number of slices. Set the number of slices to one to get the 2D FEM simulation. If there is no rotor or stator skewing, when using several slices is not reasonable, since the result will be the same as with one slice used.

2.5. D-Q model of a permanent magnet machine.

The dq-axes reference frame is a convenient way to represent sinusoidal quantities as constants. The dq-axes reference frame is fixed to the rotor and turning with the same speed as the rotor. As shown in Figure 2.6 on the example of a two pole surface-mounted permanent magnet rotor, the d-axis is always lying in the direction of the magnetic field of the permanent magnets. The q-axis is turned by 90 electrical degrees, i.e. it always lies between two magnetic poles of the machine.

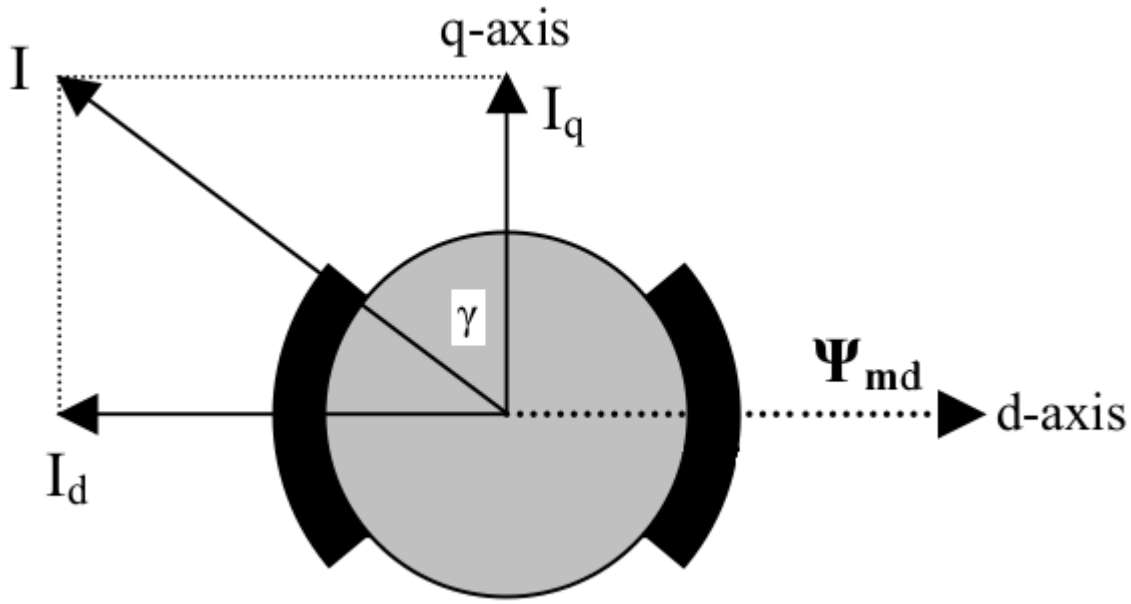


Figure 2.6. Definition of dq-axes reference frame in permanent magnet machines. [1]

As shown in Figure 2.6, the q-axis current I_q creates a magnetic field in the q-axis that interacts with the magnetic field of the permanent magnets Ψ_{md} lying in the d-axis to generate torque. The d-axis current I_d creates a magnetic field in the d-axis, which can be used to weaken the magnetic field of the permanent magnets when chosen negative (field weakening operation above base speed).

As shown in Figure 2.6, the current vector is defined by its peak value I and the advance angle γ towards the q-axis. The relationship between the amplitude and advance angle of the current vector and the dq-axes components is given by the following equations:

$$\begin{aligned} I_q &= I\cos\gamma \\ I_d &= -I\sin\gamma \\ I &= \sqrt{I_d^2 + I_q^2} \\ \gamma &= \arctan\left(-\frac{I_d}{I_q}\right) \end{aligned} \tag{2.7}$$

[1] https://www.emetor.com/blog/post/blac-machine-simulations/

To take into account the effect of cross-saturation the cross-saturation inductance L_{dq} and cross saturation magnet flux linkage Ψ_{mqd} lying in the q-axis are included. With the cross saturation terms the d-axis and q-axis flux linkages are given as follows:

$$\begin{aligned}\Psi_d &= \Psi_{md} + L_d I_d + L_{dq} I_q \\ \Psi_q &= \Psi_{mqd} + L_q I_q + L_{dq} I_d\end{aligned} \tag{2.8}$$

Where L_d, L_q, L_{dq}, Ψ_{md} and Ψ_{mqd} values depend on I_d and I_q to take into account saturation.

The steady state equations after resolving voltages into d and q components can be written in the general form:

$$\begin{aligned}V_d &= R_s I_d - \omega\, \Psi_q - \omega\, L_{sew} I_q \\ V_q &= R_s I_q + \omega\, \Psi_d + \omega\, L_{sew} I_d \\ V &= \sqrt{V_d^2 + V_q^2}\end{aligned} \tag{2.9}$$

The electromagnetic torque is calculated using the well known equation:

$$T = \frac{3}{2} p\left(\Psi_d I_q - \Psi_q I_d\right) \tag{2.10}$$

Where R_s – stator winding phase resistance, L_{sew} – stator end winding inductance, ω – electrical operating speed, p – number of pole pairs.

The calculation of D-Q model parameters L_d, L_q, L_{dq}, Ψ_{md} and Ψ_{mqd} in MotorAnalysis-PM is based on finite element method with «permeance freezing» which allows one to use superposition to extract individual parameters of the machine under load and preserve information about saturation while taking into account the effect of cross-saturation as well.[2]

And finally, the dynamic equations written for the voltage components are given by the following expressions:

$$\begin{aligned}V_d &= \frac{d\Psi_d}{dt} + L_{sew}\frac{dI_d}{dt} + R_s I_d - \omega\, \Psi_q - \omega\, L_{sew} I_q \\ V_q &= \frac{d\Psi_q}{dt} + L_{sew}\frac{dI_q}{dt} + R_s I_q + \omega\, \Psi_d + \omega\, L_{sew} I_d\end{aligned} \tag{2.11}$$

---

[2] Žarko, Damir, Ban, Drago, Klarić, Ratko. (2006). Finite Element Approach to Calculation of Parameters of an Interior Permanent Magnet Motor. AUTOMATIKA: Journal for Control, Measurement, Electronics, Computing and Communications; Vol.46, No.3-4.

2.6. Iron loss calculation

Iron core losses in MotorAnalysis-PM are calculated during the post processing so it assumes that the iron losses do not influence the magnetic field distribution. The iron loss estimation is based on the Steinmetz equation together with the eddy current term:

$$P = K_h f^{\alpha} B_m^{\beta} + K_e \left(f \cdot B_m\right)^2 \tag{2.12}$$

First term in Exp. 2.12 represents hysteresis losses, the second one – eddy current loss term. Coefficients K_h, α, β and K_e are determined by fitting equation 2.12 to the measured iron loss data from the manufacturer at different frequencies.

For the magnetostatic FEA the iron losses are estimated at a single frequency, which is a supply frequency for the stator, and zero frequency for the rotor (no iron losses in the rotor assumed). For the transient FEA (Dynamic FE Analysis) iron losses are calculated with the multiple harmonics approach i.e. the average value of iron loss is estimated over the specified time interval and FFT is used to take into consideration the non-sinusoidal shape of the flux density waveform. It allows one to include into analysis the iron loss contribution from higher harmonics of the flux density such as slot harmonics and PWM produced harmonics. Be aware that D-Q analysis in MotorAnalysis-PM does not take into account iron losses.

2.7. Demagnetization of permanent magnets.

Figure 2.7 shows an example of demagnetization curves for material N52. Irreversible demagnetization occurs if the working point in any part of the permanent magnet exceeds the linear range, i.e. falls below the "knee" of the demagnetization curve, as shown in Figure 2.7. It means that the magnet irreversibly changes its properties (remanence flux density is irreversibly reduced) and the new demagnetization curve shown by dotter line emerges.

Important parameter which characterizes the ability of permanent magnet to resist the irreversible demagnetization is intrinsic coercivity H_{cj}. Intrinsic coercivity is the applied demagnetizing field required to fully demagnetize the permanent magnet material so when the demagnetizing field is removed the material does not act like a magnet anymore. The higher intrinsic coercivity the higher demagnetizing field can be sustained by the permanent magnet without risk of demagnetization.

It is a good practice to always check your simulations for the possible risk of demagnetization at worst operating conditions (maximum current and advance angle) including possible fault conditions. In MotorAnalysis-PM the risk of demagnetization can be estimated from the maximum demagnetizing field intensity experienced by the permanent magnet during simulation. The maximum demagnetizing field intensity is available measured in A/m or in percentage of intrinsic coercivity H_{cj}. If the demagnetization curve for the given permanent magnet material is available, make sure that the maximum demagnetizing field value is to the right of the "knee" of the demagnetization curve. For

neodymium magnets there is usually no risk of demagnetization if the maximum demagnetizing field is less than 70% of intrinsic coercivity.

As seen from Figure 2.7 the demagnetization curve changes significantly with the temperature and the risk of demagnetization increases at higher temperatures of the permanent magnet. MotorAnalysis-PM varies properties of the permanent magnet depending on the temperature so to better estimate the risk of demagnetization it is recommended to check simulation results with the maximum expected temperature of the permanent magnet. You should also keep in mind that the material data and demagnetization curves represent typical properties that may vary due to magnet shape and size with tolerances of up to ±10%.

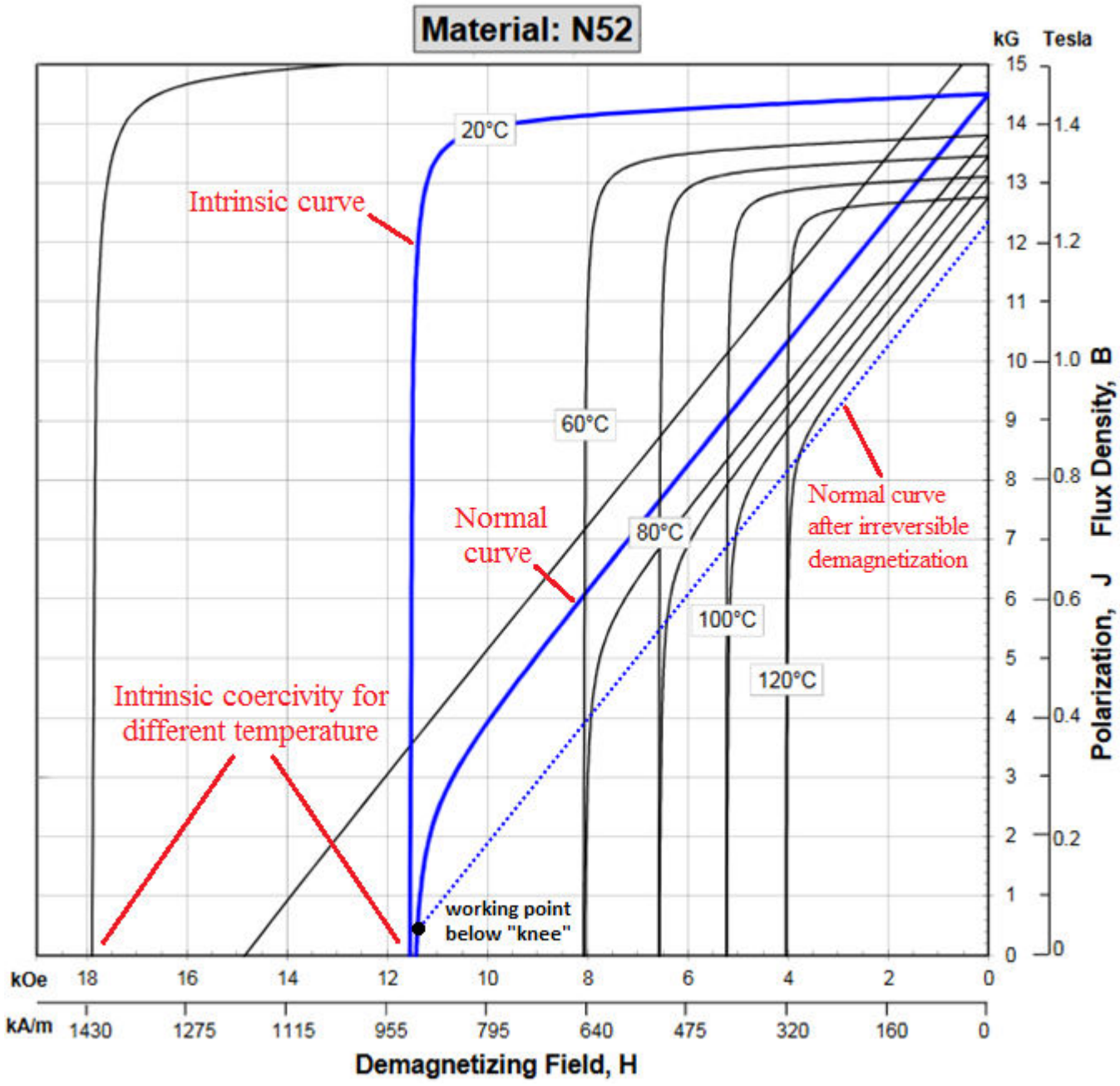


Figure 2.7. Demagnetization curves for material N52.

3. GETTING STARTED

3.1. Using MotorAnalysis-PM MATLAB version.

If you do not have MATLAB® refer to the next section to learn how to get started with standalone version of MotorAnalysis-PM.

MotorAnalysis-PM is a MATLAB-application and to use it you should have MATLAB® installed on your computer. MotorAnalysis-PM was currently tested with MATLAB R2017a and MATLAB R2017b.

To start MotorAnalysis-PM enter `motoranalysis` in MATLAB Command Window. Note that MATLAB Current Folder should be changed to the folder with the application files. MotorAnalysis-PM main window shown in Figure 3.1 will appear.

3.2. Using MotorAnalysis-PM standalone version.

MotorAnalysis-PM standalone version works like any Windows program and does not require MATLAB®. Use installation file MotorAnalysisPM1.1Installer.exe to install MotorAnalysis-PM standalone version. Internet connection is required during installation since the installation package does not include MATLAB Compiler Runtime components.

If you receive a message stating "Error finding installer class" when trying to install MotorAnalysis-PM standalone version, most likely, illegal characters are present in the path to the installation folder, or in the path to the Windows TEMP folder, or in the path to the folder containing the MotorAnalysis-PM installer. Visit https://www.mathworks.com/matlabcentral/answers/92499-why-do-i-receive-a-message-stating-error-finding-installer-class-when-trying-to-install-matlab-on for more details.

Due to some MATLAB bug, the MotorAnalysis-PM shortcut  , automatically created on the desktop after installation, does not work. You should delete it and create the shortcut manually.

Important notice: standalone version of MotorAnalysis-PM has several limitations comparing with the MATLAB version. These limitations are related to that what the MCR application is not able to execute m-files which were not included while the application was compiled. Note that several m-files are provided together with the program. These m-files were wrapped into MotorAnalysisPM.exe while the application was compiled. If you change any of these m-files or add your own m-file it will not have any effect.

Limitations of MotorAnalysis-PM standalone version:

1. Dynamic FE Analysis simulation script functions (see chapter 9) cannot be used except those provided together with the program (simscript_hystpwm.m, simscript_spacevecpwm.m, simscript_sixstep.m);

2. Electrical circuit functions (see chapter 10) cannot be used except those provided together with the program (StarConnection.m, DeltaConnection.m, InverterCircuit.m);

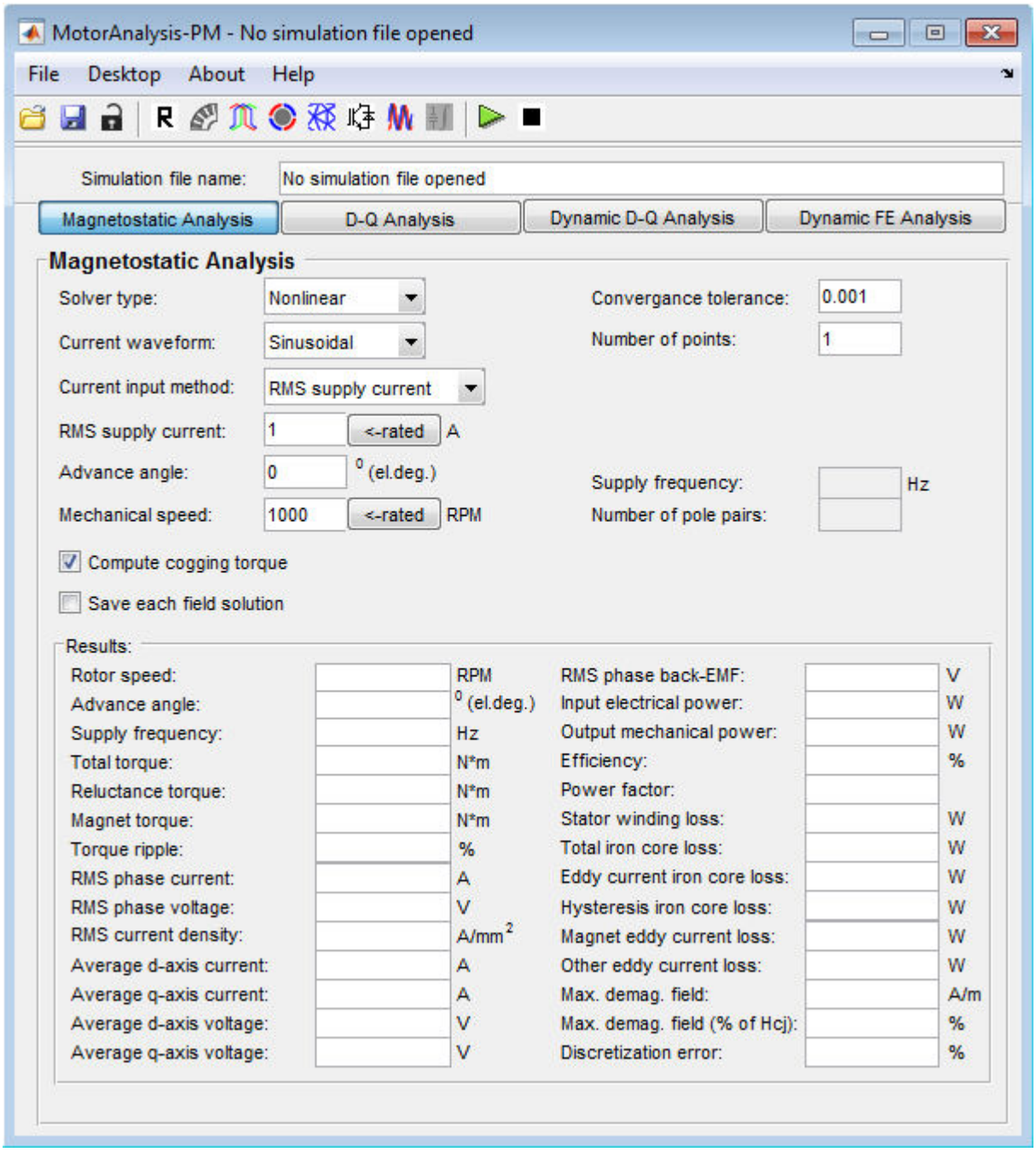


Figure 3.1. MotorAnalysis-PM main window.

If you use standalone version, keep in mind these limitations while reading this manual since some options described in the manual are available only for MATLAB version of MotorAnalysis-PM.

Using toolbar buttons you can get access to all MotorAnalysis-PM tools to create and analyze the design prototypes such as **Geometry Editor**, **Winding Editor**, **Materials**, **Drive Settings**, **Rated Data**, **Mesh Editor** and **Plot Wizard**. These are also available from the menu **Desktop**. Next chapters provide the detailed information on using these tools.

There are four analysis types available in MotorAnalysis-PM: **Magnetostatic Finite Element Analysis**, **D-Q Analysis**, **Dynamic D-Q Analysis** and **Dynamic Finite Element Analysis**. To choose the analysis type, use the corresponding button under the toolbar.

3.3. Working with simulation files.

All machine parameters, simulation settings, finite element mesh and simulation results are stored in a *simulation file*. The simulation file is a MATLAB data-file with .mat extension.

Standard file commands of opening and saving simulation files are available from the **File** menu or using toolbar buttons.

Machine parameters, materials and finite element mesh represented in the **Geometry Editor**, **Winding Editor**, **Materials** and **Mesh Editor** windows should be defined before any analysis is started and cannot be changed afterwards in order not to confuse simulation results for different design prototypes. Once you started any analysis the simulation file gets locked (**Geometry Editor**, **Winding Editor**, **Materials** and **Mesh Editor** windows become not available for editing). If you want to change any parameter in **Geometry Editor**, **Winding Editor**, **Materials** or **Mesh Editor** window after the simulation file got locked, use the **Unlock** toolbar button  to unlock the simulation file. Be aware that all simulation data of all analysis types will be deleted.

Open As New (Empty) Simulation File item of the **File** menu allows you to create a new simulation file based on another simulation file. All machine parameters, materials and analysis setting will be copied in the new simulation file while the finite element mesh and analysis results will not be included. This is a convenient way to create several design prototypes (each design prototype corresponding to the separate simulation file) changing one or several parameters to find an optimal design.

4. CREATING DESIGN PROTOTYPES

4.1. Geometry Editor.

Geometry Editor window allows you to set up cross section dimensions of the machine as well as lamination length and rotor and stator skew. **Geometry Editor** window is shown in Figure 4.1:

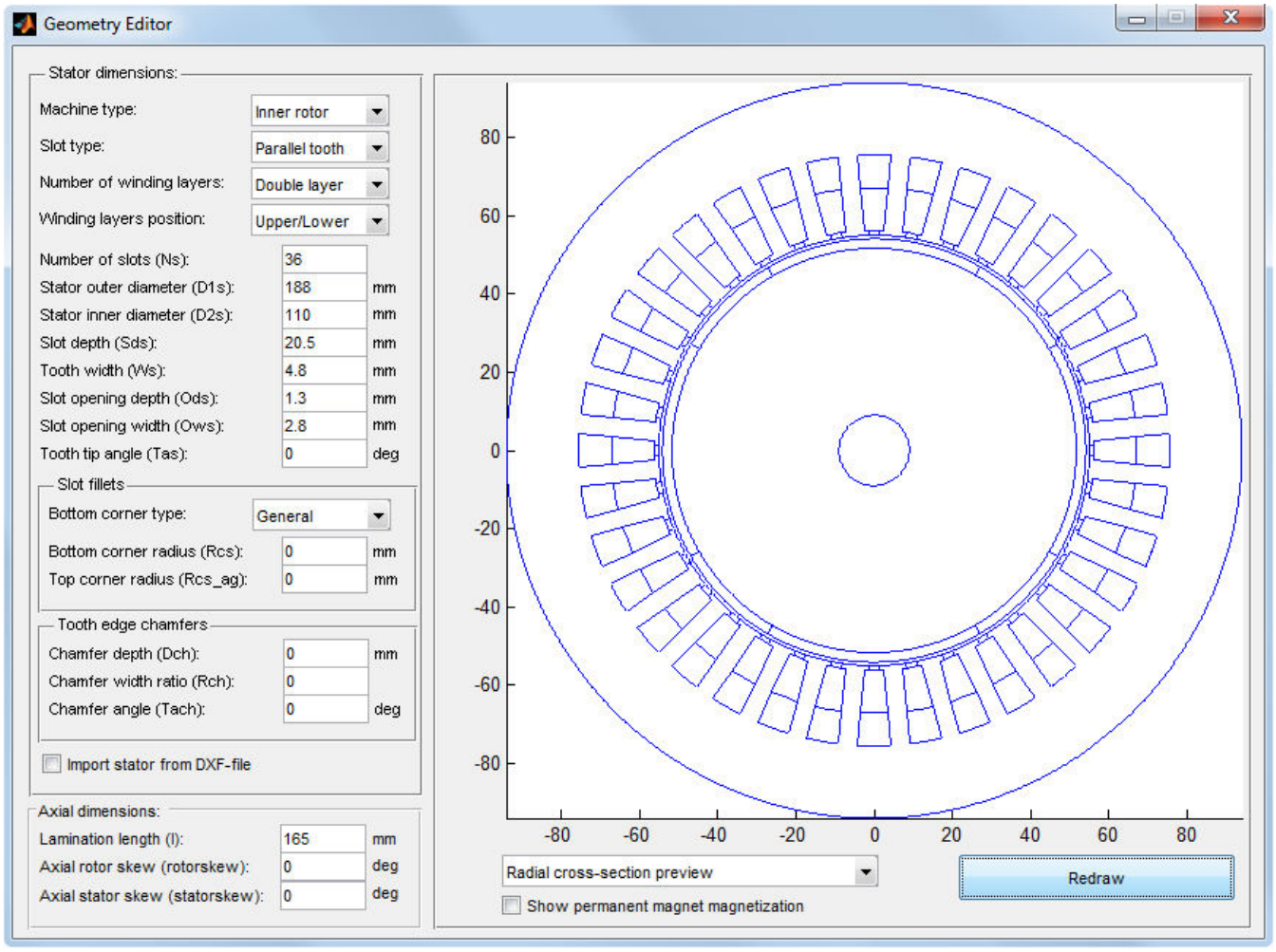


Figure 4.1. Geometry Editor window.

Machine type specifies either the machine with **inner rotor** or with **outer rotor** is used. **Slot type** pop-up menu allows you to choose the stator slot type. There are two stator slot types available: with ***Parallel tooth*** and with ***Parallel slot***. **Number of winding layers** pop-up menu allows you to choose either the ***Single layer*** or the ***Double layer*** stator winding is used. If the double layer stator winding is chosen the **Winding layer position** field (***Upper/Lo*wer** or ***Left/Right***) specifies either the stator slot will be divided horizontally or vertically into two parts with equal areas.

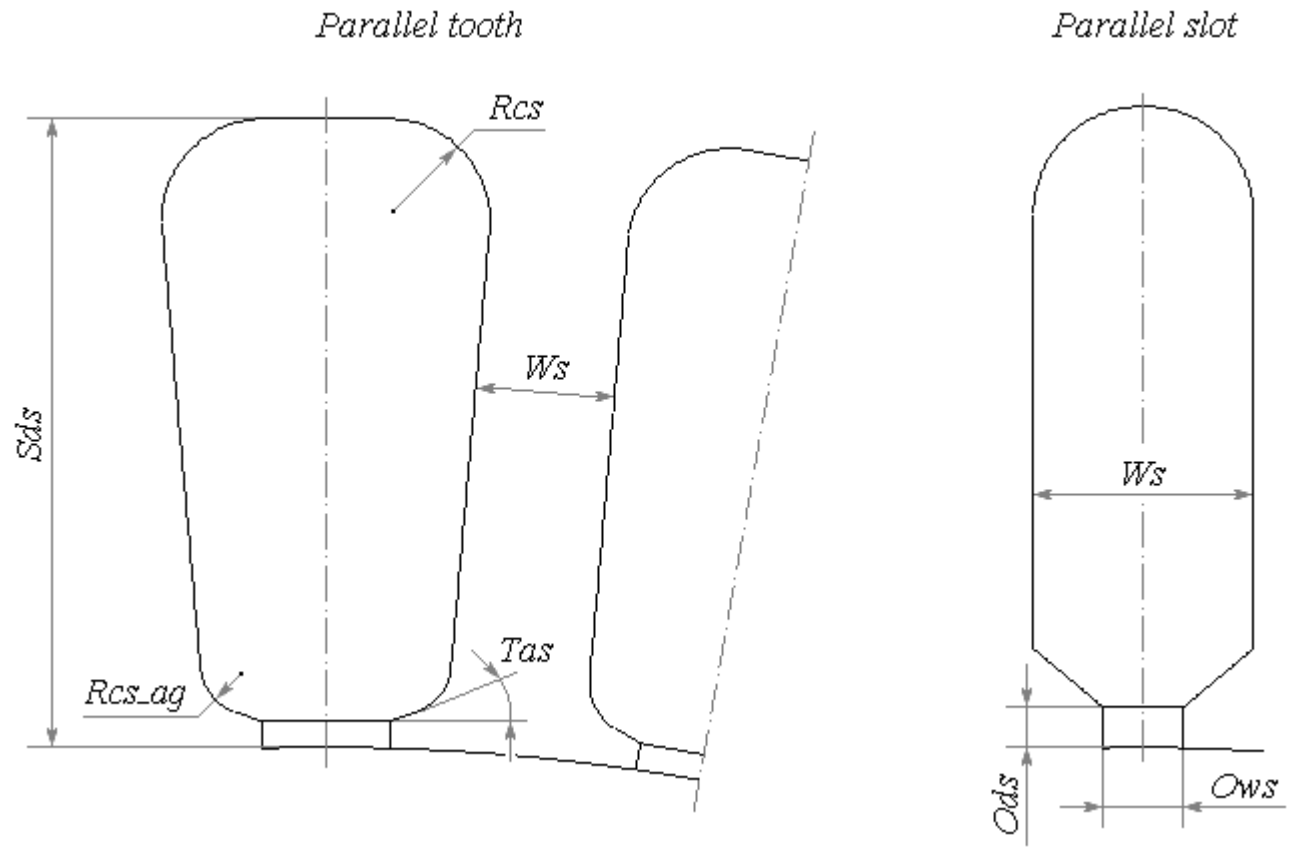
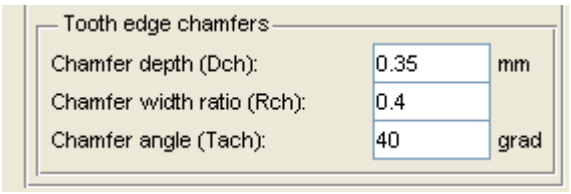


Figure 4.2. Dimensions of stator slot.



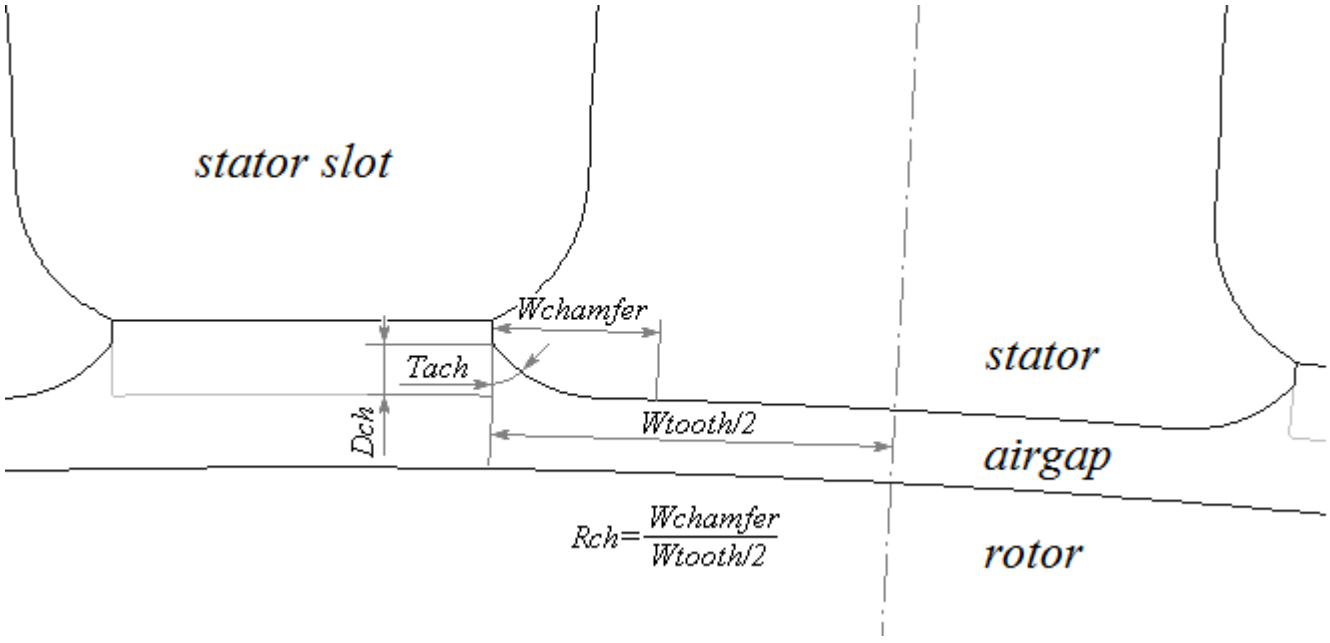


Figure 4.3. Stator tooth edge chamfer dimensions.

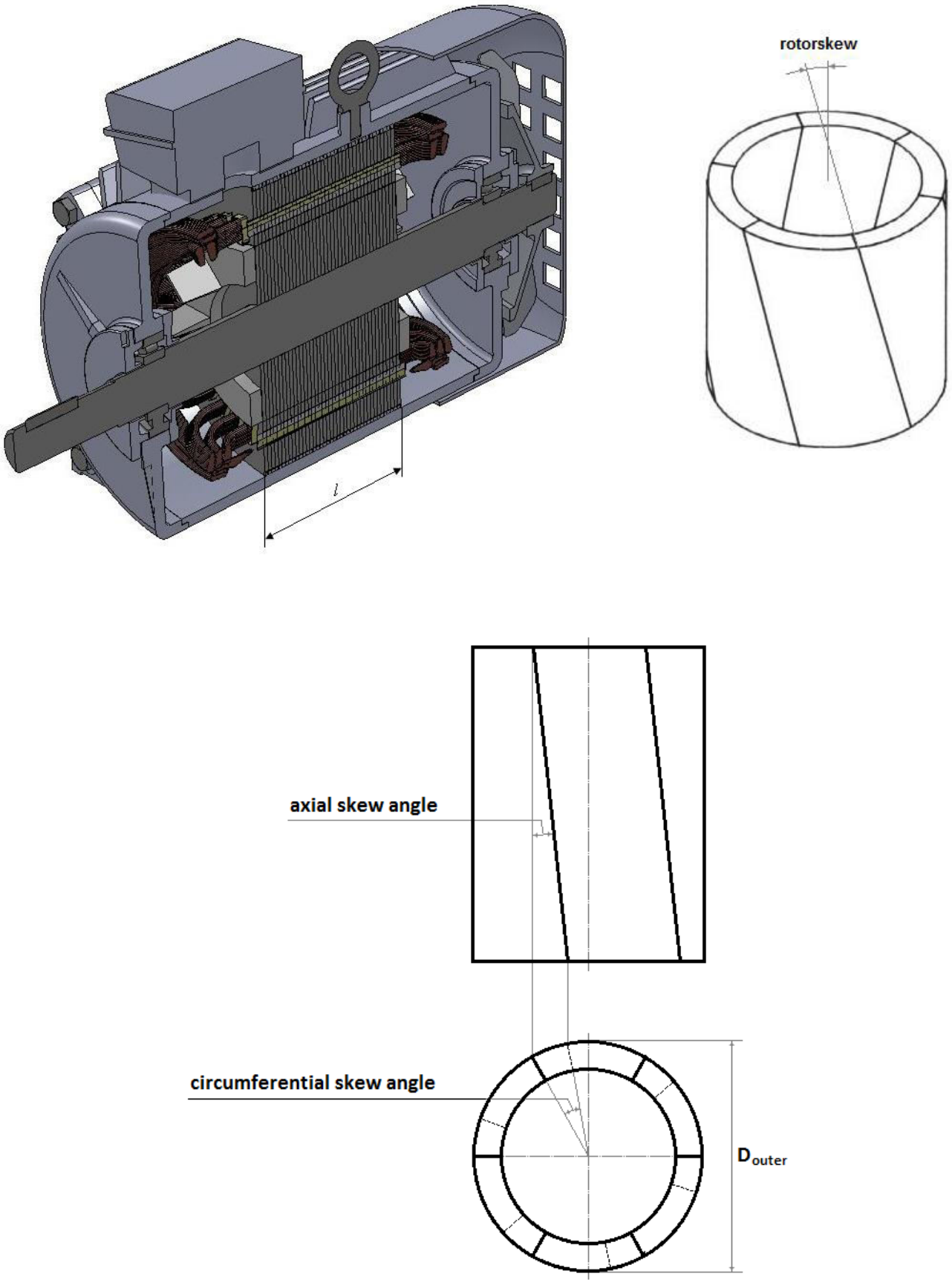


Figure 4.4. Axial dimensions: lamination length and skew angles.

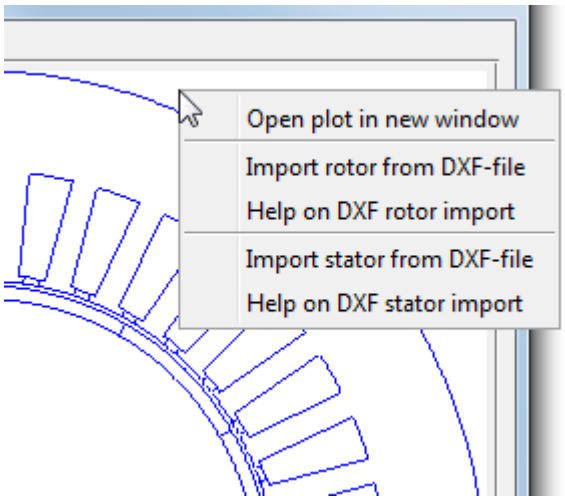


Figure 4.5. Right-click context menu of Geometry Editor.

Figure 4.2 shows stator slot dimensions used in MotorAnalysis-PM. The **Slot fillets** panel allows you to choose the top and bottom corner radius. You can also specify a ***Round*** bottom for the slot selecting corresponding item from the **Bottom corner** type pop-up menu.

There is also an option of stator tooth shape optimization in MotorAnalysis-PM. The stator tooth shape optimization can lead to reduction of the torque-ripple and the noise radiation providing the same average torque. Changing stator tooth shape can be achieved by setting up tooth edge chamfers in **Geometry Editor** as shown in Figure 4.3.

Axial dimensions of the machine are shown in Figure 4.4. **Lamination length** is the length of the stator and rotor iron cores. Note that axial skew angles are used to define stator and rotor skewing. As shown in Figure 4.4 the axial and circumferential skew angles are related to each other by the following expressions:

$$skew_{axial} = \arctan\left(\frac{D_{airgap}\sin\left(skew_{\text{circum}}/2\right)}{l}\right) \qquad skew_{\text{circum}} = 2\cdot\arcsin\left(\frac{l\cdot\tan\left(skew_{\text{axial}}\right)}{D_{airgap}}\right)$$

where l - lamination length and D_{airgap} – diameter of the center of air gap.

Example: we need to define rotor skew as one slot pitch for dimensions shown in Figure 4.1, when circumferential skew angle will be rotorskew(circum) = 360^0/Ns = 360^0/36 = 10^0, and axial rotor skew will be rotorskew(axial) = arctan(109.1*sin(10^0/2)/165) = 3.298^0, where 109.1 – air gap diameter.

The cross section of the machine is displayed in the right part of the **Geometry Editor** window. To examine the stator slot on a large scale choose the ***Stator slot preview*** item from the pop-up menu below. When the user right-clicks the picture and selects ***Open plot in new window*** (as shown in Figure 4.5), the machine geometry will be opened in a new window with MATLAB figure tools.

4.1.1. Importing rotor geometry from DXF-file.

Current version of MotorAnalysis-PM does not include rotor geometry templates – the rotor geometry should be imported from DXF-file. You can use any DXF-editor to draw the rotor geometry.

The DXF rotor geometry requirements are shown in Figure 4.6. The rotor geometry must include one half of a pole pitch with a center line of the pole pitch being aligned with the y-axis. Inner and outer boundaries should be continuous arcs with the centre on the origin (0, 0).

To import the rotor geometry, click ***Import rotor from DXF-file*** in the context menu shown in Figure 4.5 and open DXF-file with rotor geometry. Window for importing DXF-file rotor geometry is shown in Figure 4.7. Choose the units of DXF-file. Then assign subdomain type for each part of the geometry. Only **General** (for air, isolation and etc.), **Iron** (for iron core) and **Magnet** (for permanent magnets) subdomain types are allowed for the rotor. Use **Not assigned** for subdomains you want to exclude. For example if you have the drawing of the whole machine, the **Not assigned** subdomain type should be used for the stator subdomains so only the rotor geometry will be imported. For the permanent magnet the magnetization properties should be specified. The magnet can have either ***Radial*** or ***Parallel*** magnetization. For the radial magnetization the direction (from the center, towards the center, clockwise or counter-clockwise with respect to the center) and center of magnetization should be defined. The parallel magnetization is defined by the angle of magnetization.

The **Division factor** should be applied if the full cross section drawing of the machine is used to meet the DXF rotor geometry requirements shown in Figure 4.6. The division factor for the rotor should be twice of the number of poles as shown in Figure 4.8 on the example of a four pole surface-mounted permanent magnet rotor drawing with division factor 8.

The rotor geometry will be imported after clicking the **Apply** button.

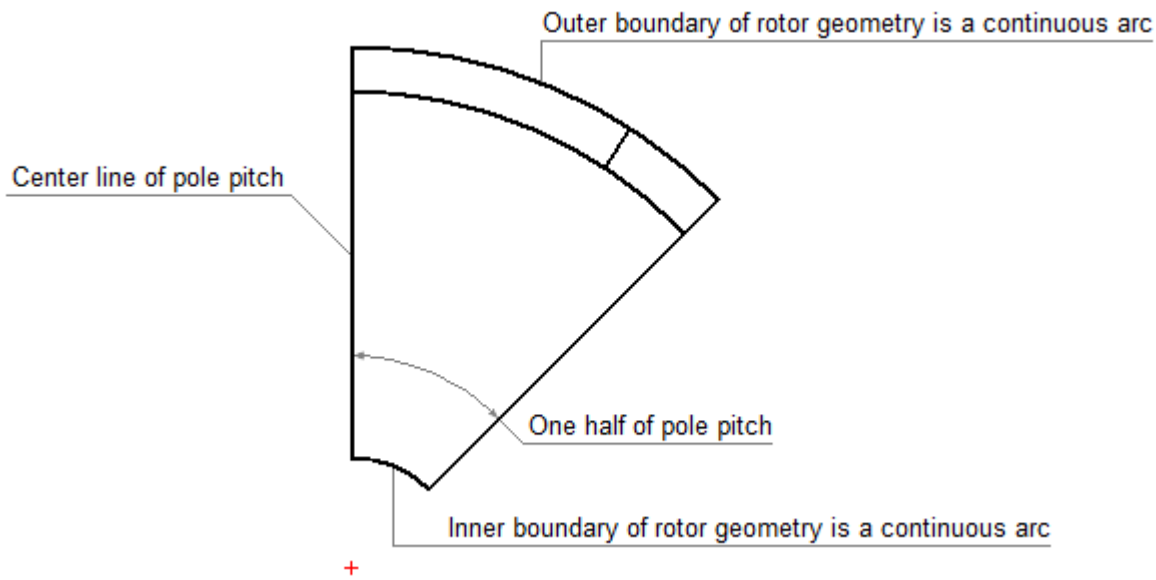


Figure 4.6. DXF rotor geometry requirements.

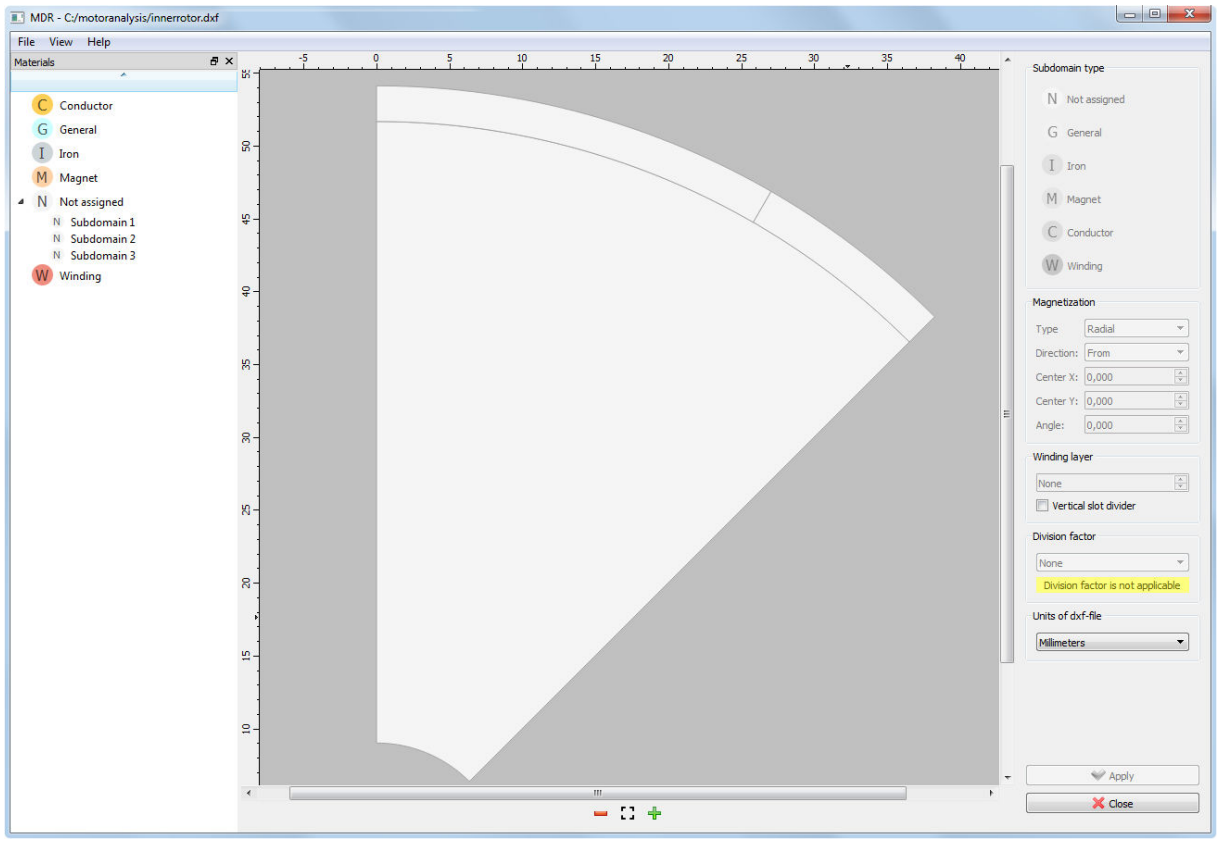


Figure 4.7. DXF rotor geometry importing window.

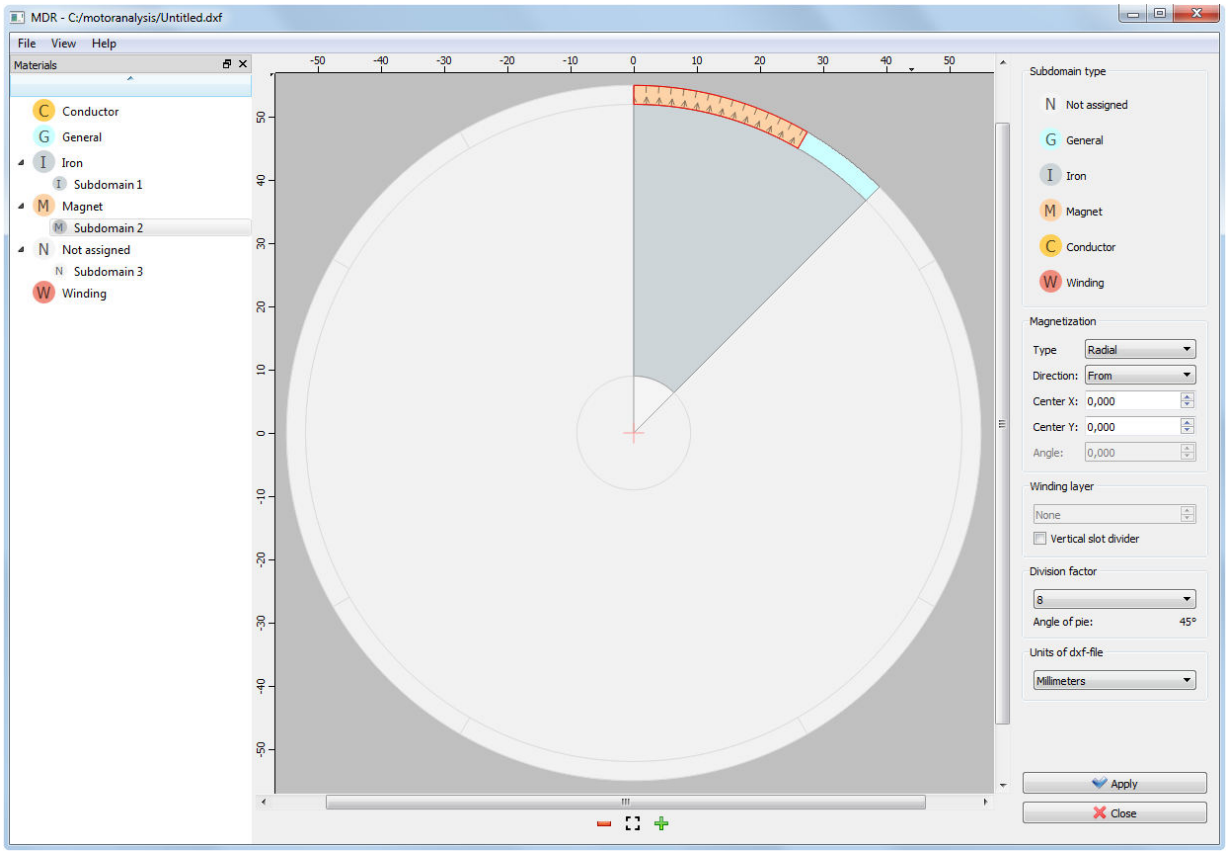


Figure 4.8. DXF rotor geometry with division factor applied.

4.1.2. Importing stator geometry from DXF-file.

To import the stator geometry from DXF-file the **Import stator from DXF-file** checkbox of the **Geometry Editor** window should be checked.

The DXF stator geometry requirements are shown in Figure 4.9. The stator geometry must include one half of a slot pitch with a center line of the slot pitch being aligned with the y-axis. Inner and outer boundaries should be continuous arcs with the centre on the origin (0, 0). This requires that the slot opening is being closed by the arc.

To import the stator geometry, click ***Import stator from DXF-file*** in the context menu shown in Figure 4.5 and open DXF-file with stator geometry. Window for importing DXF-file stator geometry is shown in Figure 4.10. Choose the units of DXF-file. Then assign subdomain type for each part of the geometry. Only **General** (for air, isolation and etc.), **Iron** (for iron core) and **Winding** (for stator winding) subdomain types are allowed for the stator. Use **Not assigned** for subdomains you want to exclude. For example if you have the drawing of the whole machine, the **Not assigned** subdomain type should be used for the rotor subdomains so only the stator geometry will be imported.

Check the **Vertical slot divider** checkbox if the left/right winding layers position is used. Assign the **Winding layer number** for each visible layer (subdomain of 'Winding' type) starting from '1' as shown in Figure 4.11 on the example of the double layer slot with the upper/lower winding layers position. The stator geometry will be imported after clicking the **Apply** button.

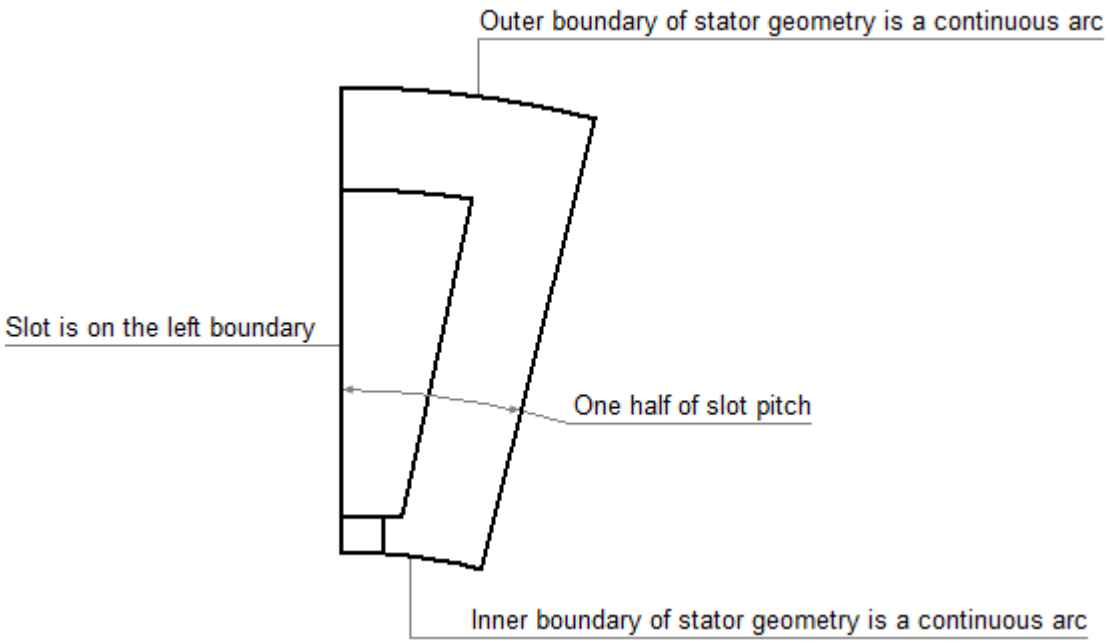


Figure 4.9. DXF stator geometry requirements.

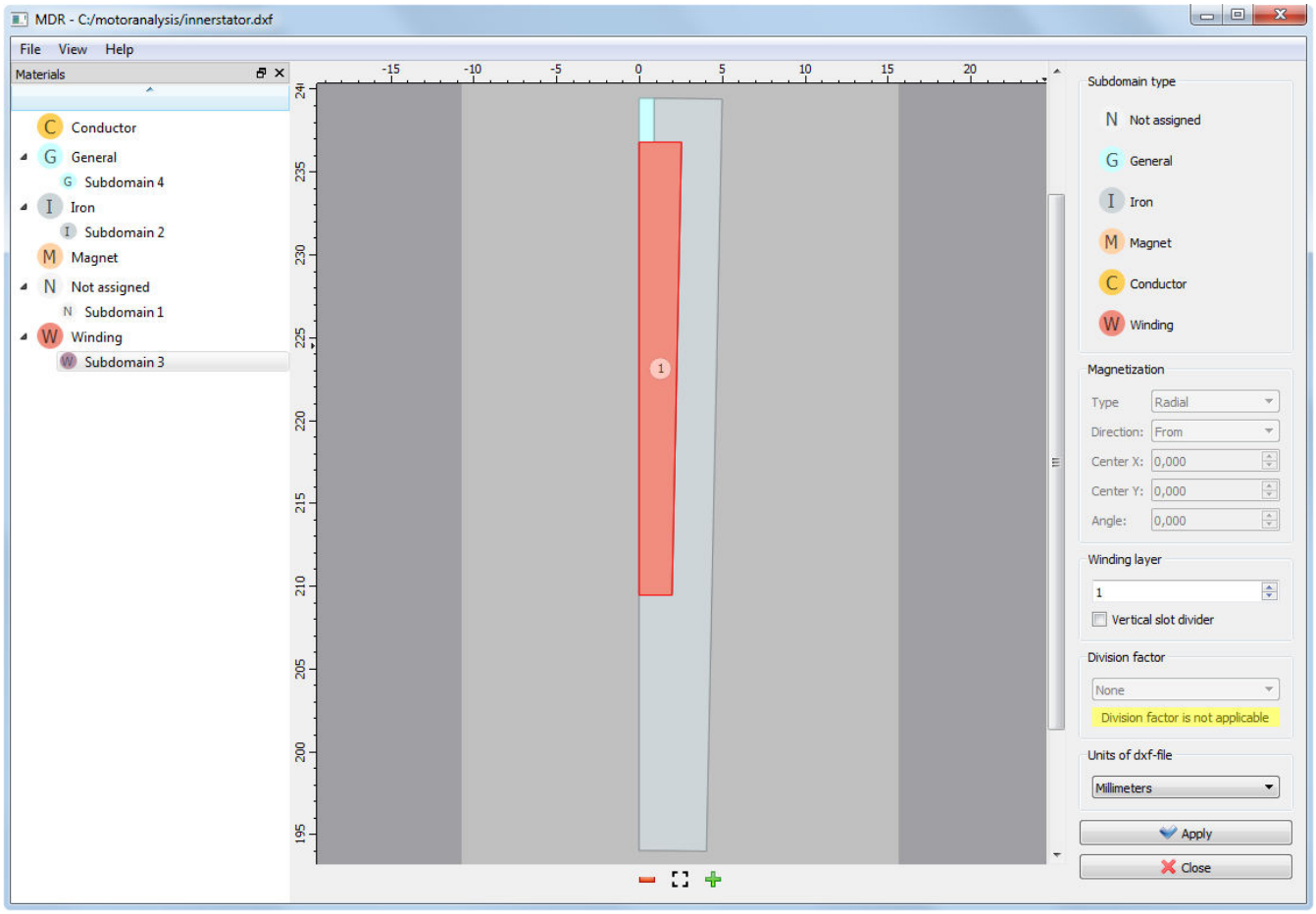


Figure 4.10. DXF stator geometry importing window (example with outer rotor machine is shown).

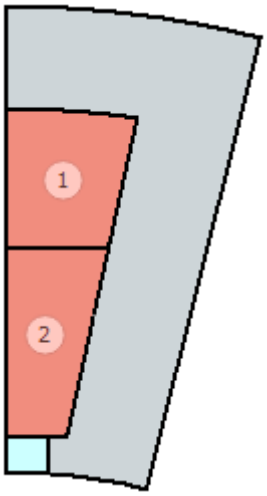


Figure 4.11. Example of DXF stator geometry with assigned subdomain types and winding layer numbers.

4.2. Winding Editor.

Winding Editor window is shown in Figure 4.12:

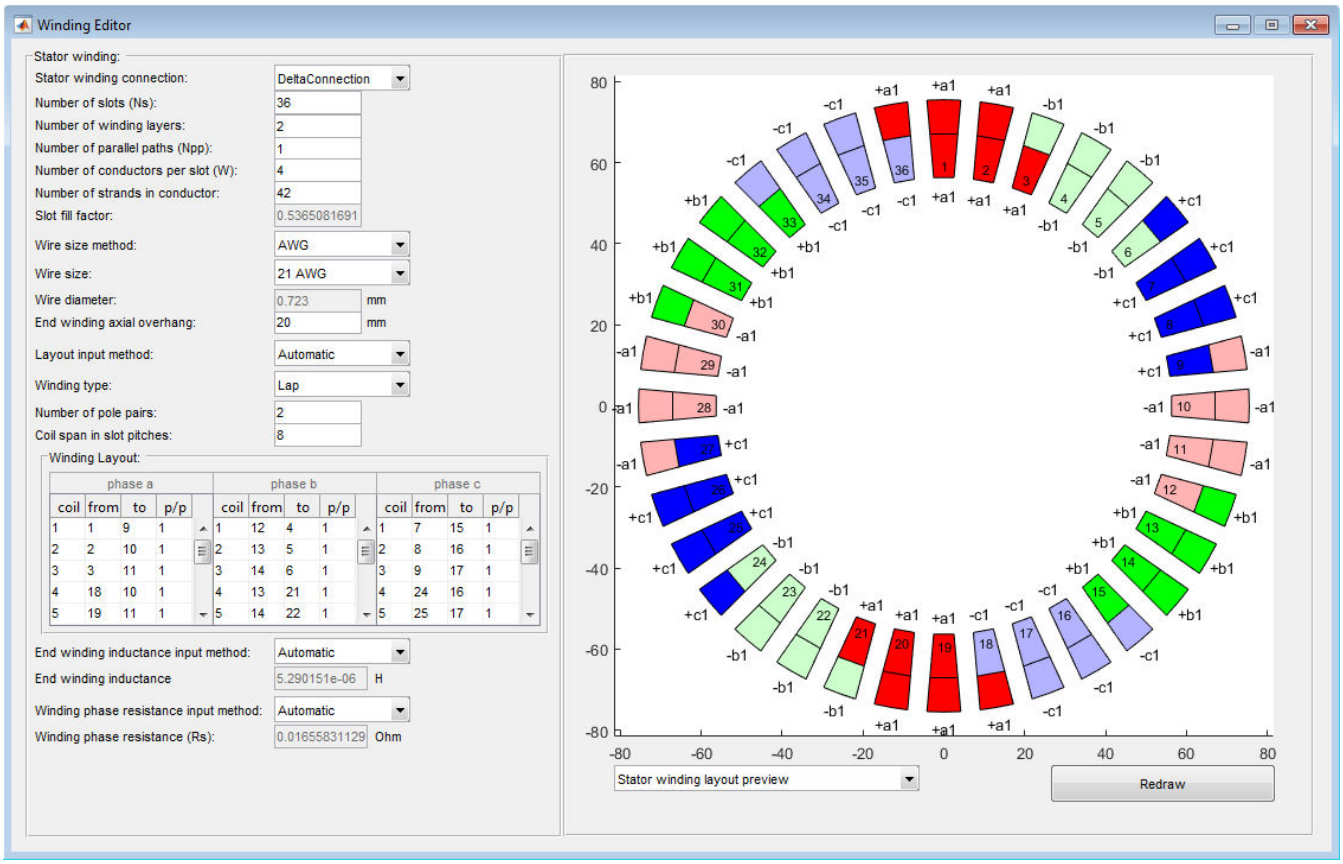


Figure 4.12. Winding Editor window.

Stator winding connection pop-up menu specifies either the winding is star-connected or delta-connected. **Number of slots** and **Number of winding layers** fields display values of the corresponding fields of the **Geometry Editor** and can be edited only from the Geometry Editor window. **Number of parallel paths** field specifies the number of groups of coils per phase connected in parallel. **Number of conductors per slot** field specifies the total number of conductors placed in one stator slot. Note that this value should be a multiple of two for a double-layer winding. **Number of strands in conductor** field specifies a number of smaller wires bundled or wrapped together to form a larger conductor to reduce the skin-effect. If the conductor consists of one piece of metal wire the number of strands is one. **Slot fill factor** field specifies the ratio of the area of all conductors (i.e. pure copper or aluminum) of a slot to the total slot area (including slot isolation). If the stator geometry is imported from DXF-file the **Coil fill factor** is used instead of slot fill factor. **Coil fill factor** is the ratio of the area of all conductors (i.e. pure copper or aluminum) of a slot to the slot area occupied by the conductors NOT including slot isolation. **Wire diameter** specifies the diameter of one strand of the conductor. The **Wire size method** pop-up menu specifies how the **Wire diameter** field value is determined. There are four wire size

methods used in MotorAnalysis-PM: ***Wire diameter*** (the wire diameter should be specified by the user), ***AWG*** (the wire diameter is specified by the AWG number according to the American wire gauge standard), ***SWG*** (the wire diameter is specified by the SWG number according to the Standard wire gauge), ***Slot fill factor*** (the slot fill factor should be specified by the user).

There are three methods to specify the stator winding layout: ***Automatic***, ***Manual*** and ***From file***. When you choose the **Layout input method** as ***Automatic*** you should also specify **Winding type** (***Lap*** or ***Concentric***) as well as **Number of pole pairs** and **Coil span in slot pitches**. When the layout input method is ***Manual*** or ***From file***, values in these three fields do not matter. Winding layout is displayed as three tables corresponding to each phase. First column of each table is a coil number; second and third columns indicate the slot numbers of the coil and the fourth column is the number of the parallel path (number of the group of coils) corresponding to the coil. The same number of parallel path corresponds to coils of the same phase connected in series. Colored representation of the winding layout is displayed in the right part of the **Winding Editor** window, where the sign "+/-" specifies the forward and return direction of the conductors within a slot, the letter following the sign specifies the phase and the phase is followed by of the parallel path number.

The difference between lap and concentric winding types is shown in Figure 4.13 on the example of 2 poles 18 slots single layer winding.

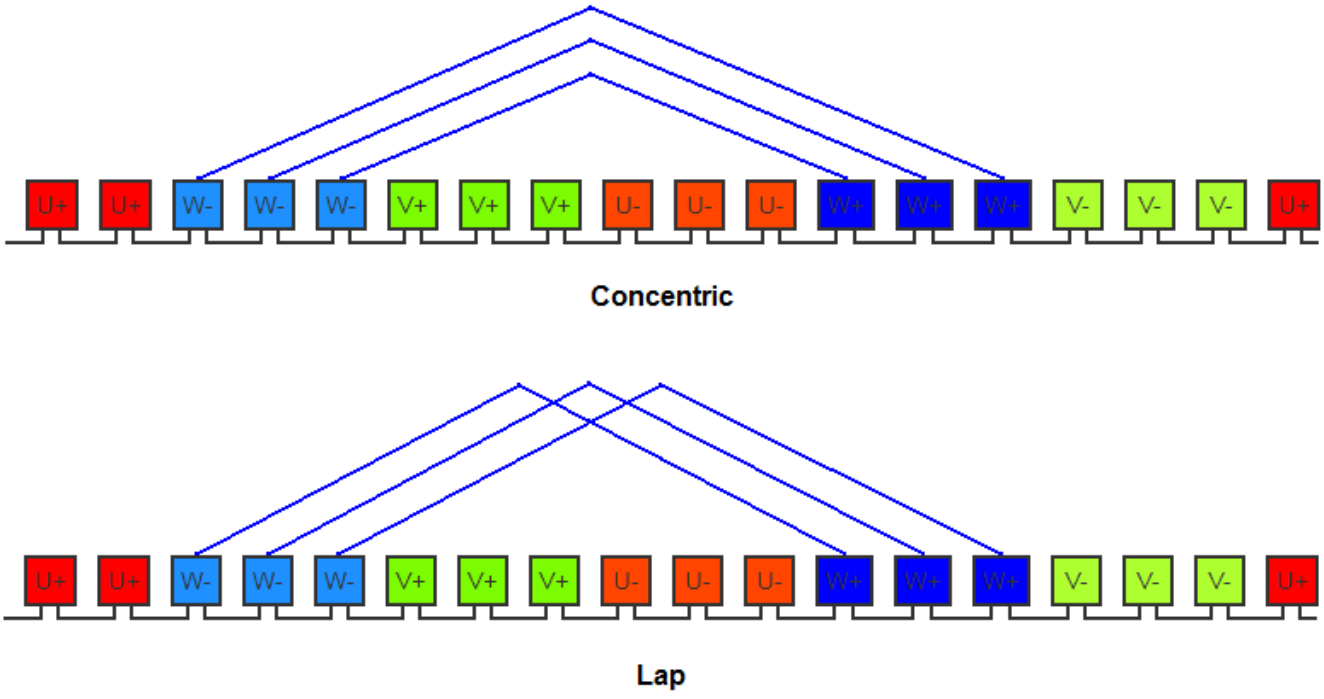


Figure 4.13. Lap and concentric winding types.

1	1	2	13	1	1
2	2	2	14	1	1
3	3	2	15	1	1
4	4	2	16	1	1
5	5	2	17	1	1
6	28	1	16	2	2
7	29	1	17	2	2
8	30	1	18	2	2
9	31	1	19	2	2
10	32	1	20	2	2
11	31	2	43	1	1
12	32	2	44	1	1
13	33	2	45	1	1
14	34	2	46	1	1
15	35	2	47	1	1
16	58	1	46	2	2
17	59	1	47	2	2
18	60	1	48	2	2
19	1	1	49	2	2
20	2	1	50	2	2

1	18	1	6	2	1
2	19	1	7	2	1
3	20	1	8	2	1
4	21	1	9	2	1
5	22	1	10	2	1
6	21	2	33	1	2
7	22	2	34	1	2
8	23	2	35	1	2
9	24	2	36	1	2
10	25	2	37	1	2
11	48	1	36	2	1
12	49	1	37	2	1
13	50	1	38	2	1
14	51	1	39	2	1
15	52	1	40	2	1
16	51	2	3	1	2
17	52	2	4	1	2
18	53	2	5	1	2
19	54	2	6	1	2
20	55	2	7	1	2

1	11	2	23	1	1
2	12	2	24	1	1
3	13	2	25	1	1
4	14	2	26	1	1
5	15	2	27	1	1
6	38	1	26	2	2
7	39	1	27	2	2
8	40	1	28	2	2
9	41	1	29	2	2
10	42	1	30	2	2
11	41	2	53	1	1
12	42	2	54	1	1
13	43	2	55	1	1
14	44	2	56	1	1
15	45	2	57	1	1
16	8	1	56	2	2
17	9	1	57	2	2
18	10	1	58	2	2
19	11	1	59	2	2
20	12	1	60	2	2

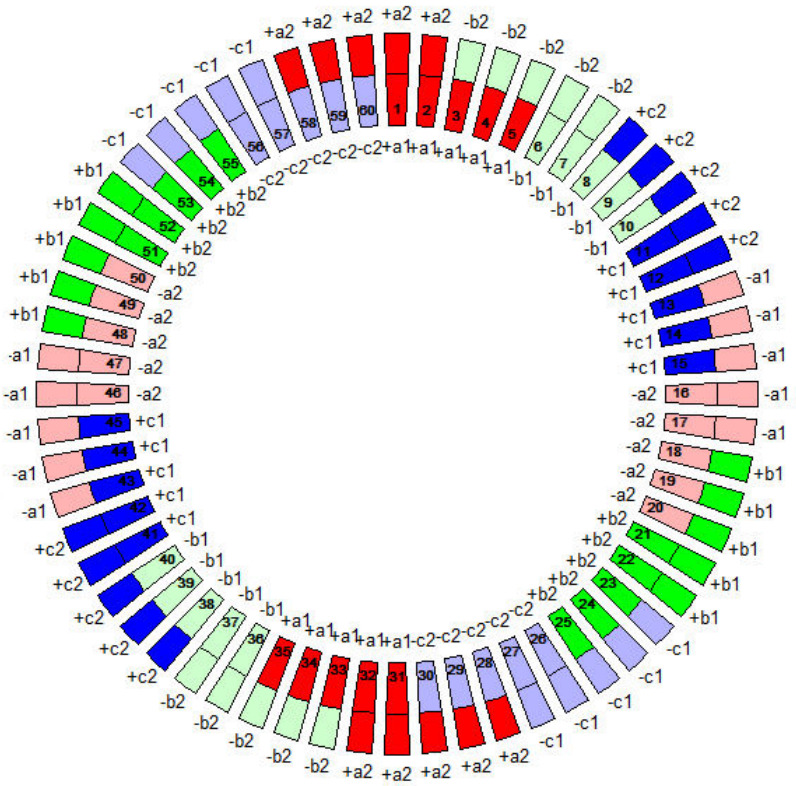


Figure 4.14. Example of the winding layout file and corresponding layout picture.

The example of the winding layout loaded from file layoutfile_example.txt is shown in Figure 4.14. Layout file should include layout for each phase. First column of the layout file is a coil number as shown in Figure 4.14, second column is a ‘from’ slot number, third column – winding layer number of the slot ‘from’, forth column - ‘to’ slot number, fifth column – winding layer number of the slot ‘to’, and the sixth column is the number of the parallel path. By default, the first winding layer is placed at the bottom of the slot as shown in Figure 4.11. When the stator geometry is imported from the DXF-file the winding layer numbers are specified by the user i.e. layers can have any position within a slot. Windings having more than two layers are also allowed. To use the winding with more than two layers the stator geometry should be imported from DXF-file and the winding layout should be loaded from file.

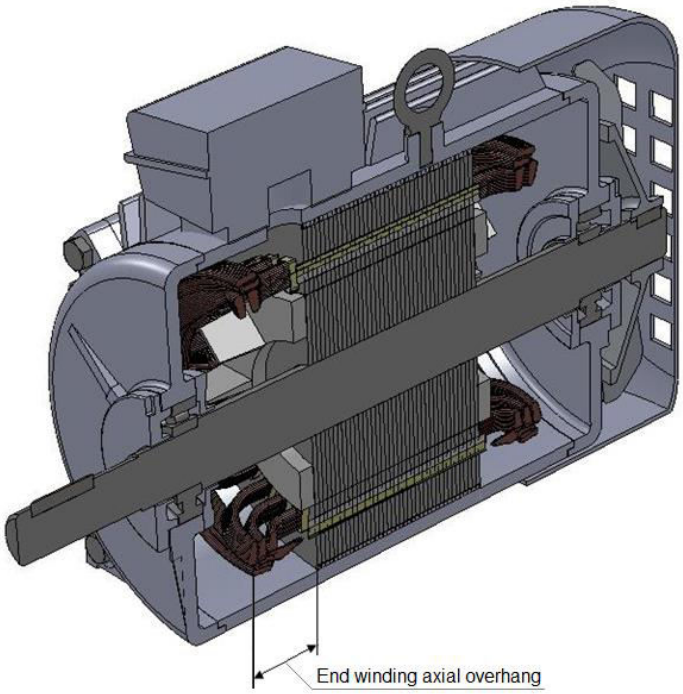


Figure 4.15. End winding axial overhang.

End winding axial overhang specifies the distance between the lamination stack and the outside of the windings as shown in Figure 4.15.

End winding inductance field specifies the leakage inductance of the stator winding end-turns per phase. Depending on the chosen **End winding inductance input method** (***Manual*** or ***Automatic***), the end winding inductance can be entered manually or calculated automatically based on the winding configuration and machine dimensions. Note that when you choose the end winding inductance automatic input method you should also specify the **End winding axial overhang** field value**.**

Winding phase resistance field specifies the active resistance of the stator winding per phase for the temperature specified in the **Materials** window. Winding phase resistance is automatically adjusted for the given temperature according to the conductor material temperature coefficient of resistance (see section 4.3). Depending on the chosen **Winding phase resistance input method** (***Manual*** or ***Automatic***), the winding phase resistance can be entered manually or calculated automatically based on the winding configuration and machine dimensions. Note that when you choose the winding phase resistance automatic input method you should also specify the **End winding axial overhang** and **Slot fill factor** field values**.**

4.3. Assigning materials.

The window for assigning materials in MotorAnalysis-PM is shown in Figure 4.16.

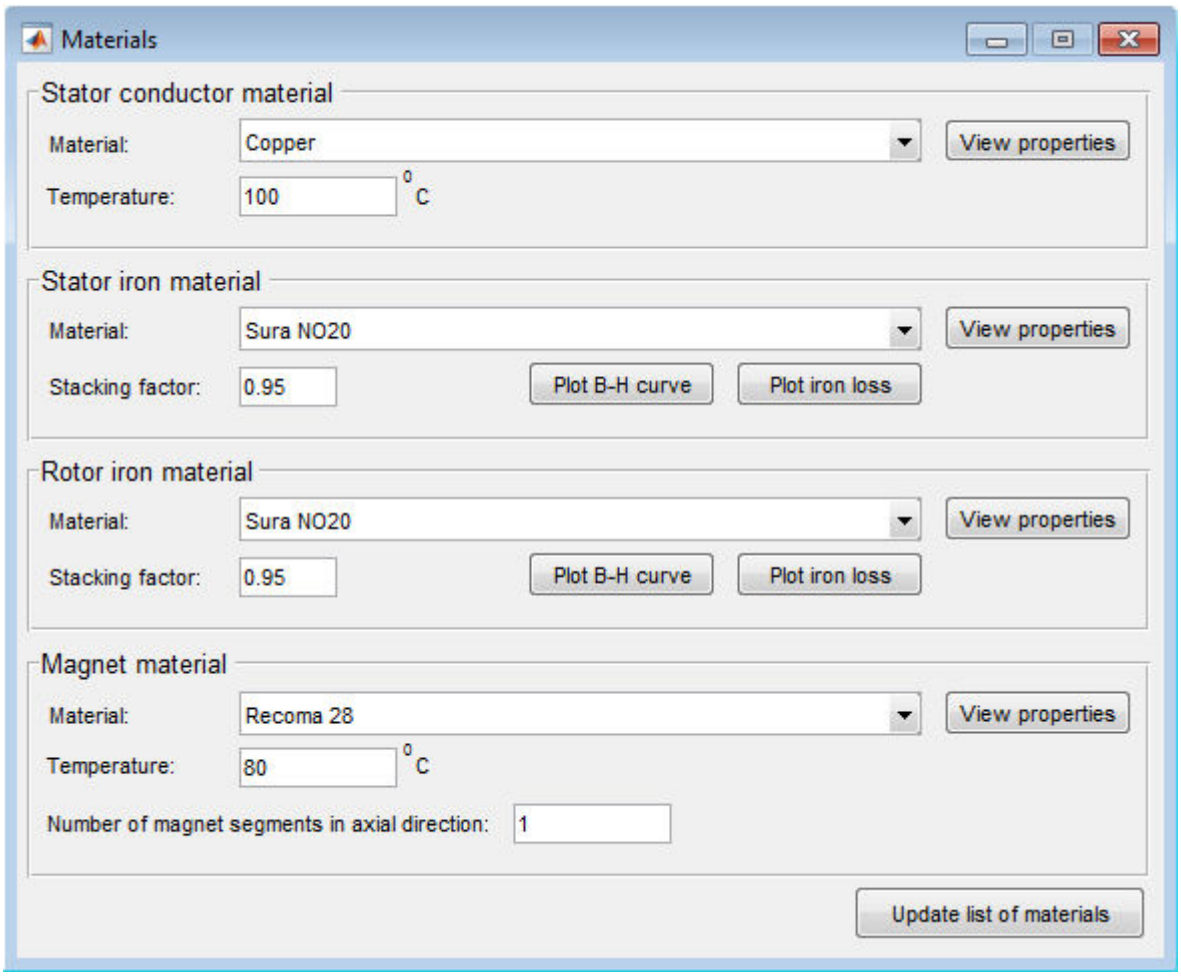


Figure 4.16. Window for assigning materials in MotorAnalysis-PM.

There are three groups of materials (Conductor, Iron and Magnet) which can be assigned to stator winding conductors, stator iron core, rotor iron core and permanent magnets. Clicking the **View properties** button opens the file with material properties.

Stator conductor material temperature specifies the temperature of stator winding conductors. Winding phase resistance is automatically adjusted for the given temperature according to the conductor material temperature coefficient of resistance. The expression for temperature effect on resistance is as follows:

$$R=R_{20^0C}*[1+\alpha*(T-20^0C)],$$

where R – resistance at temperature T; R_{20^0C} – resistance at 20^0C; α – temperature coefficient of resistance; T – winding temperature.

Stacking factor field specifies the ratio of the volume filled by electrical steel to the total volume of the iron core. The total volume of the iron core consists of the volume of lamination steel sheets and the volume of the coating between the sheets. Stacking factor reduces the flux carried by the iron core which is taken into account by MotorAnalysis-PM through the stacking factor value.

Magnet material temperature has its effect on the permanent magnet properties such as remanence flux density and intrinsic coercivity. The remanence flux density used by MotorAnalysis-PM is defined as follows:

$$Br=Br_{20℃}*[1+\alpha_{Br}*(T_{pm}-20^0C)/100],$$

where Br – remanence flux density at temperature T_{pm}; $Br_{20℃}$ – remanence flux density at 20^0C; α_{Br} – temperature coefficient of remanence flux density in $\%/^0$C; T_{pm} – permanent magnet temperature.

Similarly, the intrinsic coercivity is defined as follows:

$$Hcj=Hcj_{20℃}*[1+\alpha_{Hcj}*(T_{pm}-20^0C)/100],$$

where Hcj – intrinsic coercivity at temperature T_{pm}; $Hcj_{20℃}$ – intrinsic coercivity at 20^0C; α_{Hcj} – temperature coefficient of intrinsic coercivity in $\%/^0$C; T_{pm} – permanent magnet temperature.

Number of magnet segments in axial direction field defines the segmentation of the permanent magnet along the shaft. If number of magnet segments is more than one, it is assumed that each magnet is divided in a given number of pieces alone z-axis electrically isolated from each other while magnet losses are computed.

Use the **Update list of materials** button if the new material was added to see it on the list.

4.3.1. Adding new materials.

You can add your own materials to the Materials Library or modify properties of the existing materials. All material files are stored in the *Materials* folder. Each group of materials is stored in the separate folder (*Conductor*, *Iron* and *Magnet*) inside the *Materials* folder. Each material has the following properties:

Conductor:

- Electric resistivity at 20^0C
- Temperature coefficient of resistance
- Mass Density

Iron:

- B-H curve
- Iron loss
- Mass Density

Magnet:

- Electric resistivity

- Relative permeability
- Coercivity
- Remanence flux density
- Intrinsic coercivity
- Temperature coefficient of remanence flux density
- Temperature coefficient of intrinsic coercivity
- Mass Density

File with material properties is a txt-file of a special format. The example of the file with material properties is shown below (only part of the file is shown, full text can be found in Materials\Iron\Sura NO20.txt):

```
<Description>
Sura NO20
Non-oriented silicon steel


<B-H curve> at 20*C
% H(A/m)      B(T)
0             0
10            0.0190
20            0.0530
30            0.1100
37            0.2000
...
...
...
30000         1.9730
50000         2.0590
100000        2.1830
130000        2.2310


<Iron loss>
% frequency(Hz)  flux density(T)  iron loss(W/kg)
50               0.1000           0.0200
50               0.2000           0.0700
50               0.3000           0.1400
50               0.4000           0.2300
50               0.5000           0.3200
50               0.6000           0.4200
50               0.7000           0.5400
50               0.8000           0.6600
50               0.9000           0.8000
50               1.0000           0.9500
50               1.1000           1.1400
50               1.2000           1.3600
50               1.3000           1.6500
50               1.4000           2.0000
50               1.5000           2.4000
50               1.6000           2.7500
50               1.7000           3.0600
50               1.8000           3.3200
400              0.1000           0.1700
400              0.2000           0.7200
...
...
...
```

```
10000           0.1000          27.0000
10000           0.2000          95.6000
10000           0.3000          191.0000
10000           0.4000          315.0000


<Mass Density> (kg/m^3)
7650
```

Each property is delineated by *property tag*, written using angle brackets: `<B-H curve>`, `<Iron loss>` and etc. The tag is followed by the *property content* until another property tag is found or the end of the file is reached. If the property is not specified it means that the corresponding property tag is followed by empty line(s) until the next property tag or the end of the file. The content of a line following after the percent sign "*%*" and after the property tag is ignored and treated as a comment.

Refer to the corresponding files in the Materials Library to check the structure and units of each property content while adding new material files. There are properties defined by a single value such as `<Mass Density>`, `<Electric resistivity>`, `<Coercivity>` and others are defined by arrays of values (`<Iron loss>`, `<B-H curve>`).

The magnetization properties of permanent magnets are defined by the following property tags: `<Relative Permeability>`, `<Coercivity>` and `<Remanence flux density>`. It is allowed to specify either all of them or any two of them and leave one empty.

The iron loss of the iron can be defined either by iron loss data or by iron loss Steinmetz equation coefficients (see section 2.6). The iron loss defined by the iron loss data are shown in the example above. The first and second columns are frequency and peak flux density values respectively in increasing order exactly as shown above and the third column is the corresponding iron loss values. Example of the iron loss defined by the Steinmetz equation coefficients can be found in M-15 29 Ga.txt:

```
<Iron loss>
%Steinmetz coefficients
0.0227288148583325      % kh
1                       % alfa
1.77364043109264        % beta
2.11492029537421e-05    % Ke
```

If the iron loss property is not specified (property tag `<Iron loss>` is followed by empty line(s) until the next property tag or the end of the file) it is assumed that the iron losses are zero.

4.4. Drive Settings.

There are three drive types available in MotorAnalysis-PM (***Current hysteresis PWM***, ***Space vector PWM*** and ***Six-step***) which can be chosen in the **Drive Settings** window as shown in Figures 4.18 – 4.22. It is assumed that the machine is supplied from the three-phase inverter shown in Figure 4.17 (for the Dynamic FE Analysis the inverter circuit also includes diodes as shown in Figure 10.1):

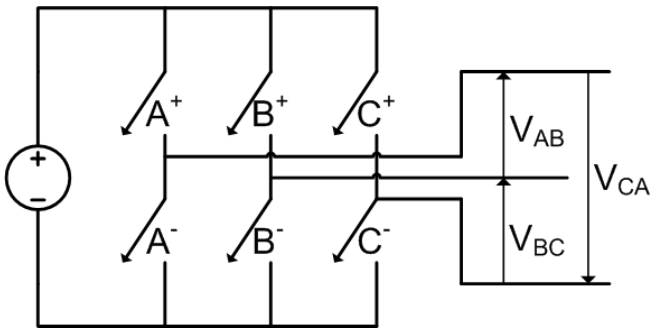
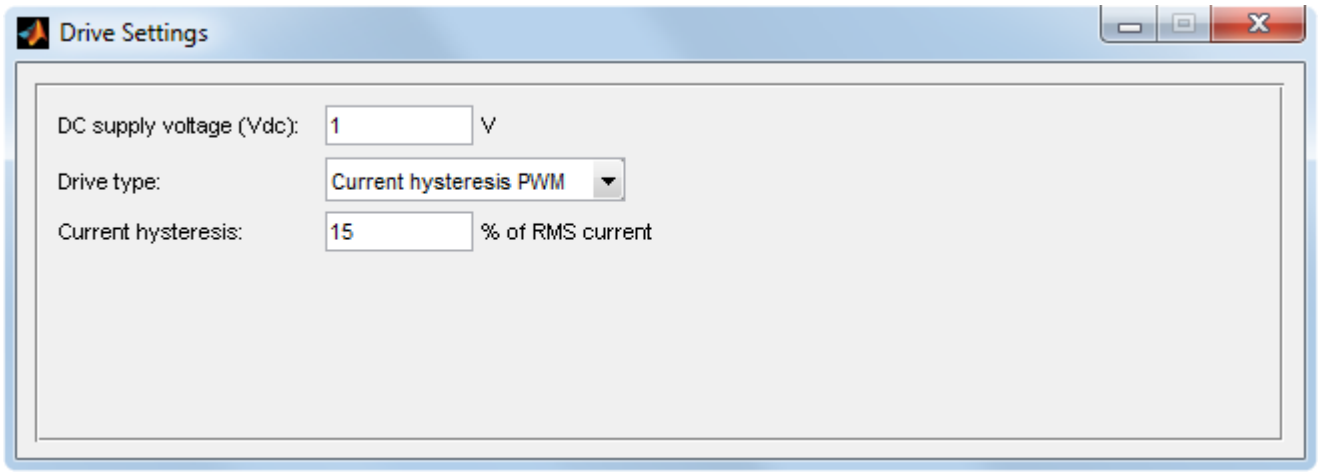


Figure 4.17. Three-phase inverter.

The *Current hysteresis PWM* drive type is shown in Figure 4.18. The phase current is kept within the upper and lower bounds defined by the **Current hysteresis** field value.



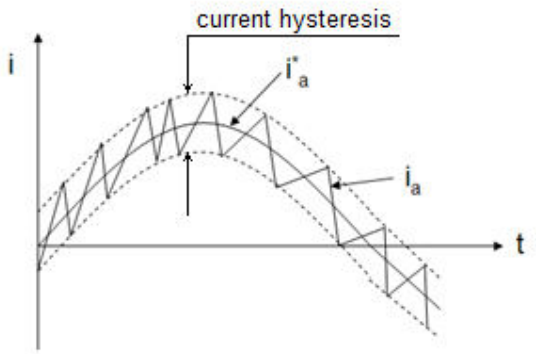


Figure 4.18. Current hysteresis PWM.

The **Drive Settings** window when the ***Space vector PWM*** drive type is chosen is shown in Figure 4.19.

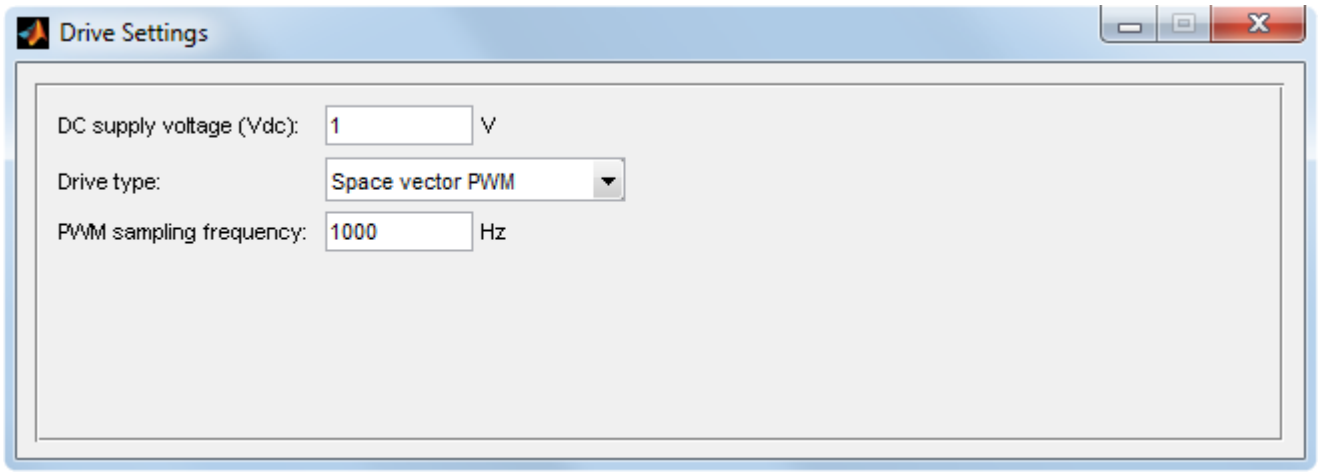


Figure 4.19. Space vector PWM.

The **Drive Settings** window when the ***Six step*** drive type is chosen is shown in Figures 4.20 – 4.22.

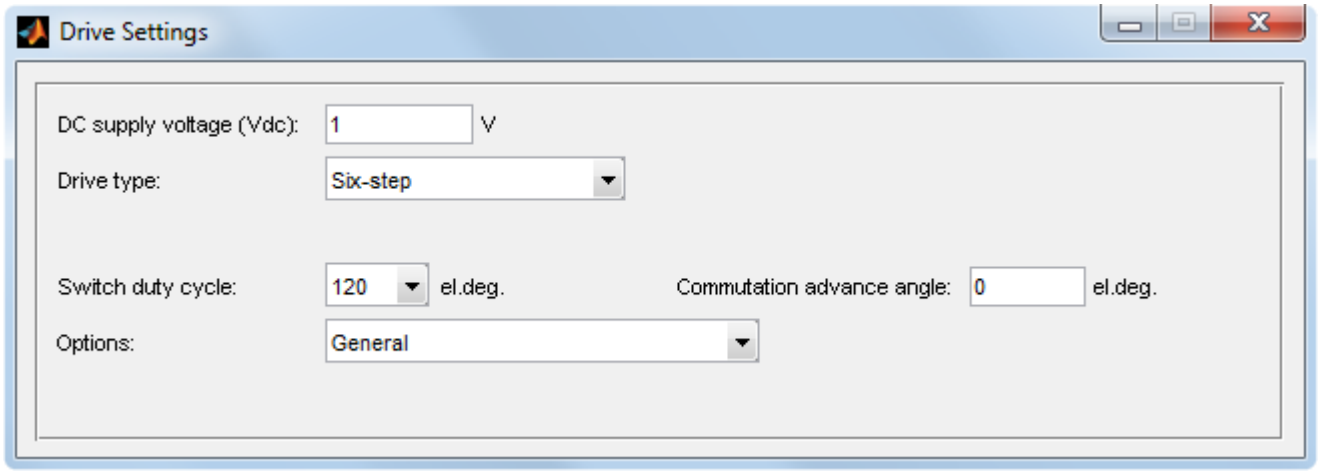


Figure 4.20. Drive Settings window with six-step drive type chosen.

Switch duty cycle specifies the time in electrical degrees when the switch is turned on. ***120*** degrees switch duty cycle corresponds to the inverter operation with two switches turned on at the same time; with ***180*** degrees switch duty cycle there will be three switches turned on at the same time.

In practice, the phase inductances and the non-ideal current commutations will cause a phase lag of the current with respect to the voltage. The lag of the current can be compensated by imposing the **Commutation advance angle** shifting forward the commutation timing.

There are two additional options for the six-step drive: ***Six-step with limited maximum current*** and ***Six-step with variable DC voltage***.

When the six-step with limited maximum current is used (Figure 4.21), all switches are turned off once the current of any phase reaches the **Motor phase current limit**. The corresponding switches are turned on again when the maximum phase current becomes lower than the current limit on the value specified by the **Phase current hysteresis** field.

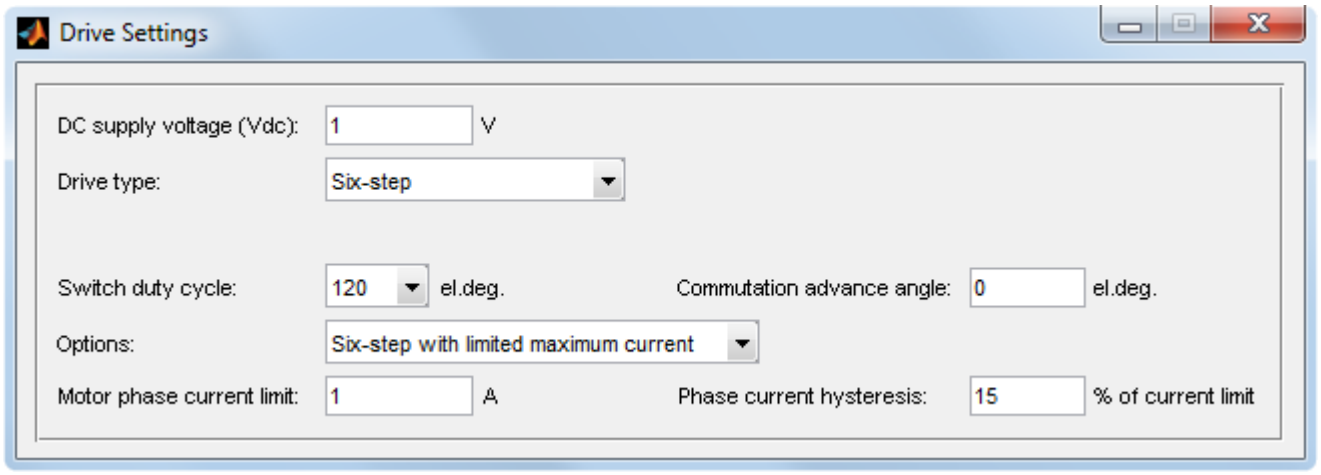


Figure 4.21. Six-step drive with limited maximum current.

When the six-step with variable DC voltage is used (Figure 4.22), the input DC voltage of the inverter is modulated so only the part of the battery voltage is used specified by the **Vdc usage percentage** field value. Make sure that the **PWM sampling frequency** is high enough not to interfere with the six-step commutation frequency.

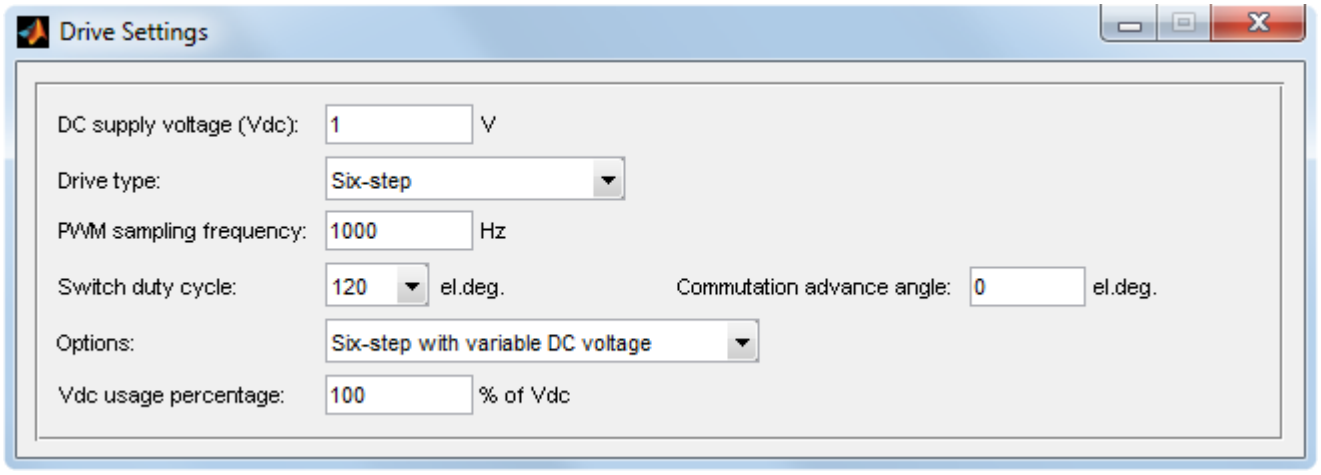


Figure 4.22. Six-step drive with variable DC voltage.

4.5. Mesh Editor.

Mesh Editor window is shown in Figure 4.23.

Number of layers in air gap field specifies the total number of finite element layers placed in the air gap. You can specify only odd number of finite element layers from 1 to 9 (refer to section 2.2 of this manual for more details).

Mesh growth rate field determines the mesh growth rate away from a small part of the geometry. The mesh growth rate should be strictly between 1 and 2 and cannot be equal to either of its bounds, 1 and 2. The default value is 1.4, i.e., a growth rate of 40%. **Air gap mesh quality** field specifies the quality of the mesh in the air gap region (***Low***, ***Medium*** (by default) or ***High***). The mesh of the highest quality would consist of equilateral triangles which is practically not possible for complex geometry. The lower air gap mesh quality usually leads to the less number of mesh triangles and faster computational speed sacrificing the accuracy. Use the **Mesh growth rate** and the **Air gap mesh quality** fields to control the number of mesh triangles.

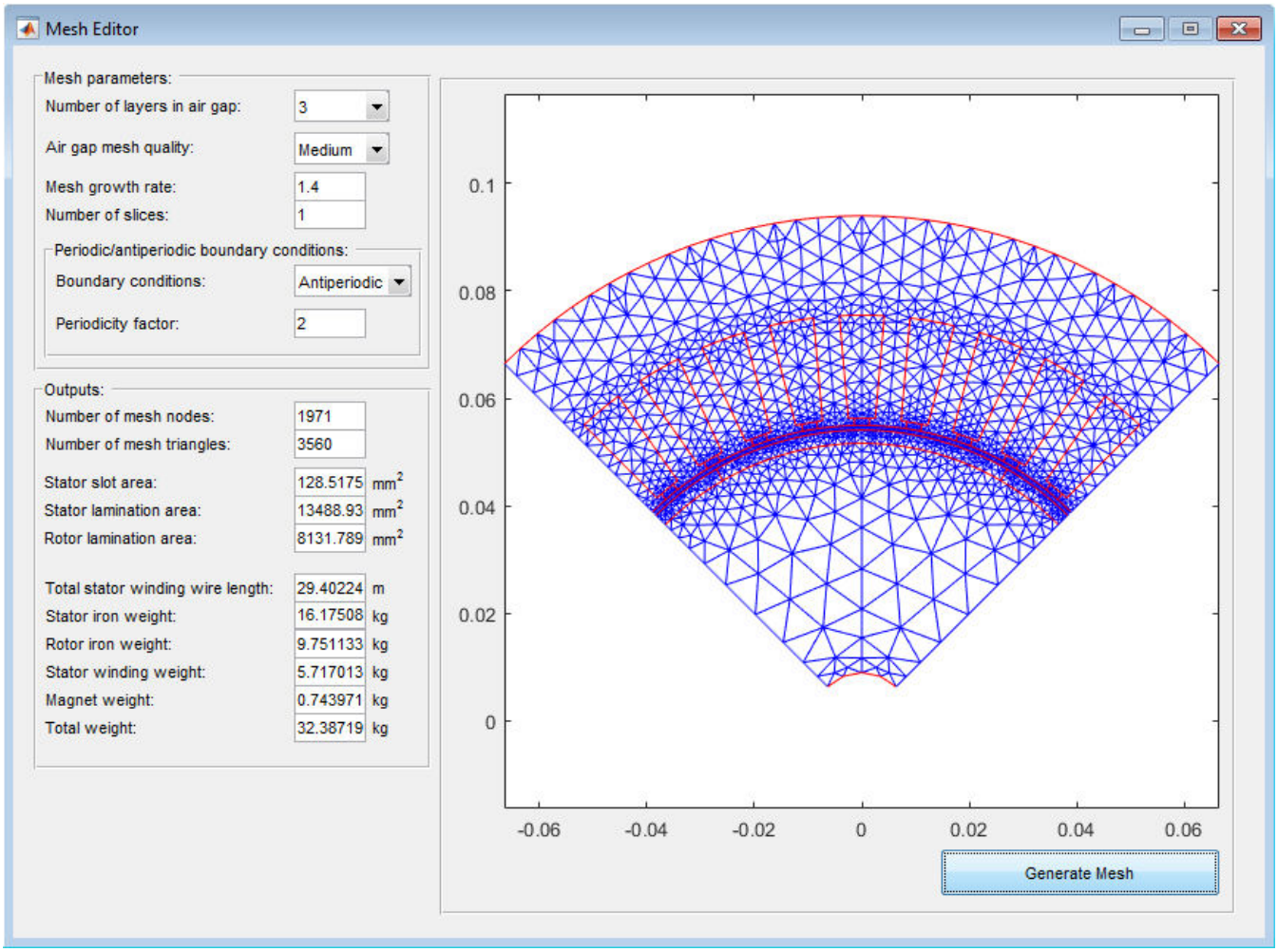


Figure 4.23. Mesh Editor window.

Number of slices field specifies the number of slices used by the multi-slice FEM, i.e. the number of the machine's cross sections in which the magnetic field is simultaneously calculated. For more details on the multi-slice FEM refer to section 2.4 of this manual.

You can apply periodic or antiperiodic boundary conditions choosing corresponding item from the **Boundary conditions** pop-up menu. If ***None*** is chosen from the **Boundary conditions** pop-up menu, the boundary conditions are not used. **Periodicity factor** is equal to the number of pole pairs. The best possible boundary conditions are automatically chosen when the rotor geometry is imported from DXF-file. For more details on boundary conditions refer to section 2.2 of this manual.

Click the **Generate mesh** button to generate the mesh. It is recommended to examine the mesh before the simulation is started. Right-click the mesh and select ***Open plot in new window***, the mesh will be opened in a new window with MATLAB figure tools for detailed examination. The accuracy of FEA significantly depends on the shape of finite elements especially within and close to the air gap region. The most accurate results are obtained provided that the shape of finite elements is as close as possible to the equilateral triangle shape. If the shape of finite elements is significantly distorted, try to change the **Mesh growth rate** and the **Air gap mesh quality** properties.

If **Error while trying to generate mesh** message occurs, make sure that all mesh parameters are correct. Incorrectly defined motor geometry can also cause mesh generation errors – make sure that all motor dimensions are consistent. In some cases, incorrect dimensions can only be seen on the enlarged scale as shown in Figure 4.24. In this case the slot corner radius is inconsistent with the slot width.

Number of mesh nodes and **Number of mesh triangles** fields are calculated for one slice. To calculate the total number of mesh nodes and total number of mesh triangles one should multiply values specified in the **Number of mesh nodes** and **Number of mesh triangles** fields by the number of slices.

Finite element mesh is also used to compute weights of active materials and total weight of the machine (see Figure 4.23).

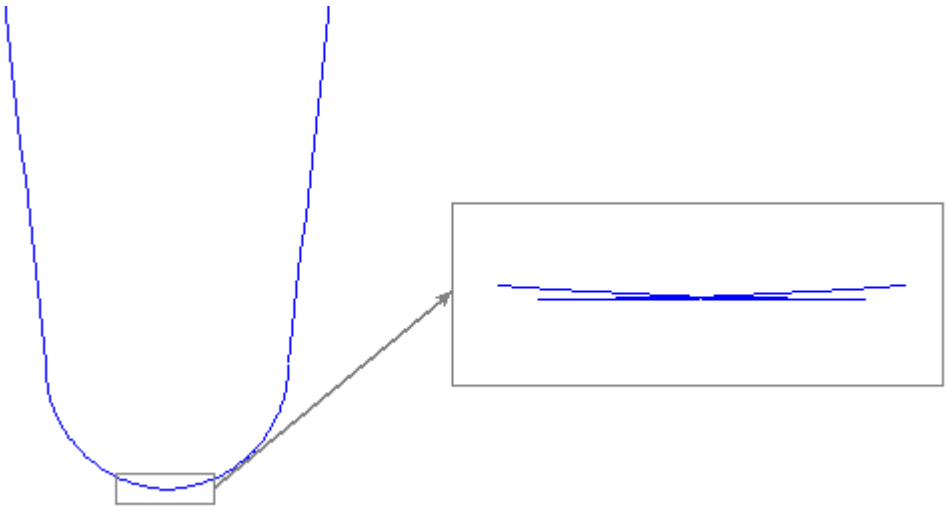


Figure 4.24. Example of incorrect motor dimensions.

5. MAGNETOSTATIC ANALYSIS

Magnetostatic Analysis allows the user to estimate motor parameters like voltage, current, back-EMF, power, torque, power factor, efficiency, losses and their waveforms as well as air gap and cross-section distribution plots of various quantities such as airgap flux and mmf, flux density, permeability, current density and etc. assuming an ideal sinusoidal or trapezoidal current waveform.

5.1. Running Magnetostatic Analysis.

The view of the main window when the Magnetostatic Analysis is chosen is shown in Figure 5.1. Magnetostatic Analysis consists of a series of finite element simulations (series of time steps) over one electrical period with predefined currents and rotor positions defined by the rotor speed. The current waveform is defined by the **Current waveform** field and can be either sinusoidal or trapezoidal. There are several current input methods defined by the **Current input method** field which also depend on the current waveform. For the sinusoidal current waveform the following current input methods are available: ***RMS supply current***, ***Peak supply current*** and ***RMS current density***. When the ***RMS supply current*** input method is chosen the phase current will depend on the **RMS supply current** field value and on the stator winding connection. Since the supply current is a line current, for star connection the phase current is equal to the supply current, for delta connection the phase current is equal to the supply current divided by $\sqrt{3}$. ***Peak supply current*** input method is similar to the previous one but peak value of supply current is used instead. When the ***RMS current density*** input method is chosen the RMS current density in stator slots can be directly defined.

For the trapezoidal current waveform the following current input methods are available: ***RMS phase current***, ***DC supply current*** and ***RMS current density***. When the ***RMS phase current*** input method is chosen the phase current can be directly defined. When the ***DC supply current*** input method is chosen the phase current will depend on the **DC supply current** field value, stator winding connection and switch duty cycle. Switch duty cycle (120 or 180 electrical degrees) is defined in the **Drive Settings** window when six-step drive type is chosen (see section 4.4). Figure 5.2 shows an example of trapezoidal current waveforms for star and delta connection and 120 degrees switch duty cycle. Be aware that the real current waveform for the six-step drive can be significantly different from those used in the Magnetostatic Analysis. When the ***RMS current density*** input method is chosen the RMS current density in stator slots can be directly defined.

The **Advance angle** field value defines the timing of the current commutation in relation to the rotor position assuming ideal current commutation and is allowed to vary between 0 and 360 electrical degrees. **Number of points** defines the number of time steps over one electrical period for which the FEA simulations are performed. For sinusoidal current waveform it is allowed to use one point simulation but in this case time plots will not be available.

Button '**<-rated**' to the right of the field allows you to set up the corresponding parameter to its rated value specified in the **Rated Data** window.

Solver type specifies either the linear or nonlinear FEA is used. Using linear solver type is recommended only for testing purposes. **Convergence tolerance** specifies the accuracy of FEA solution as defined by Exp. 2.5.

The cogging torque is computed with zero stator current and if **RMS supply current** (**DC supply current**) field value is not zero when it will require one additional FEA simulation for each time step to compute cogging torque. Be aware of this fact when checking **Compute cogging torque**.

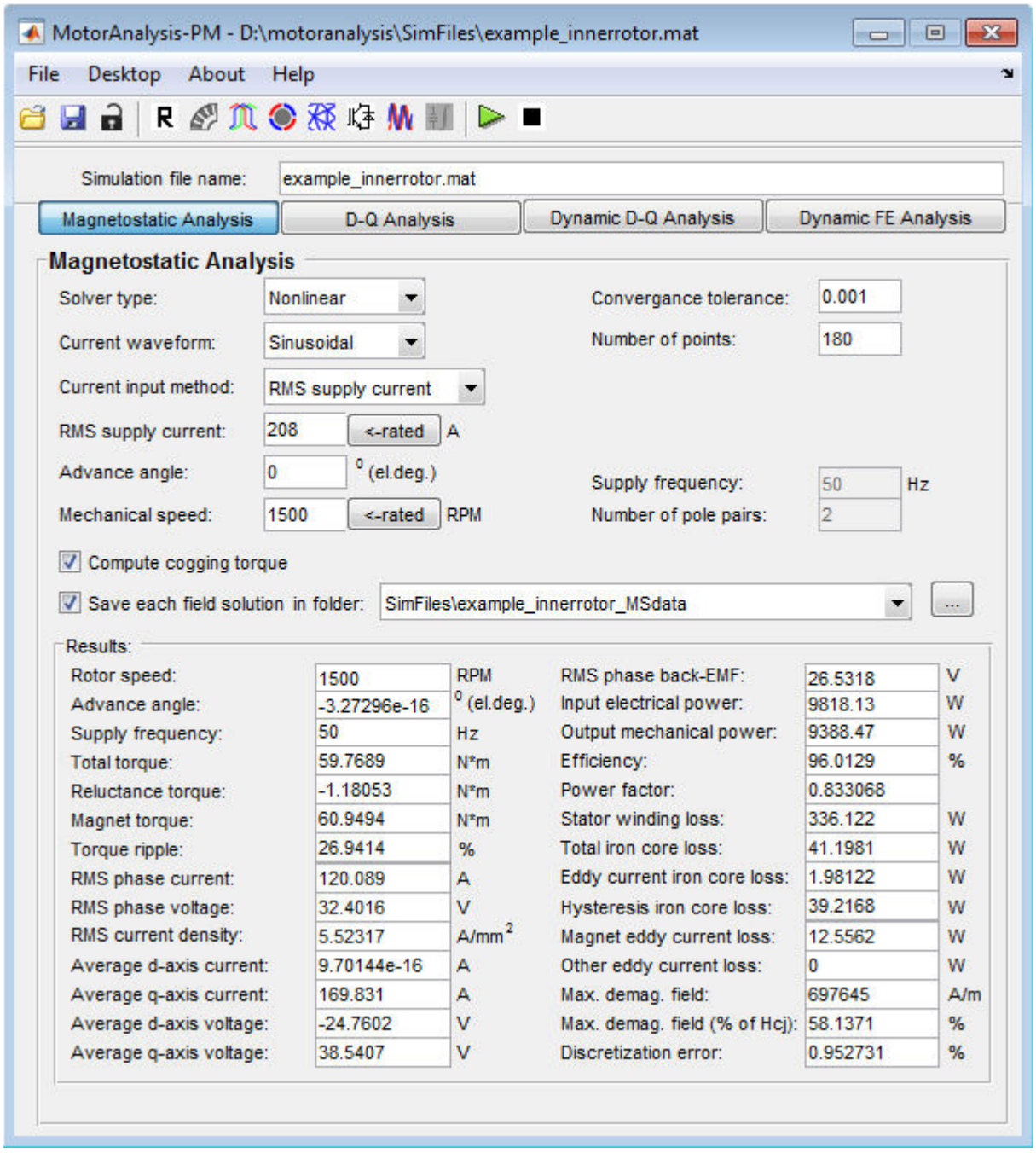


Figure 5.1. MotorAnalysis-PM main window for Magnetostatic Analysis.

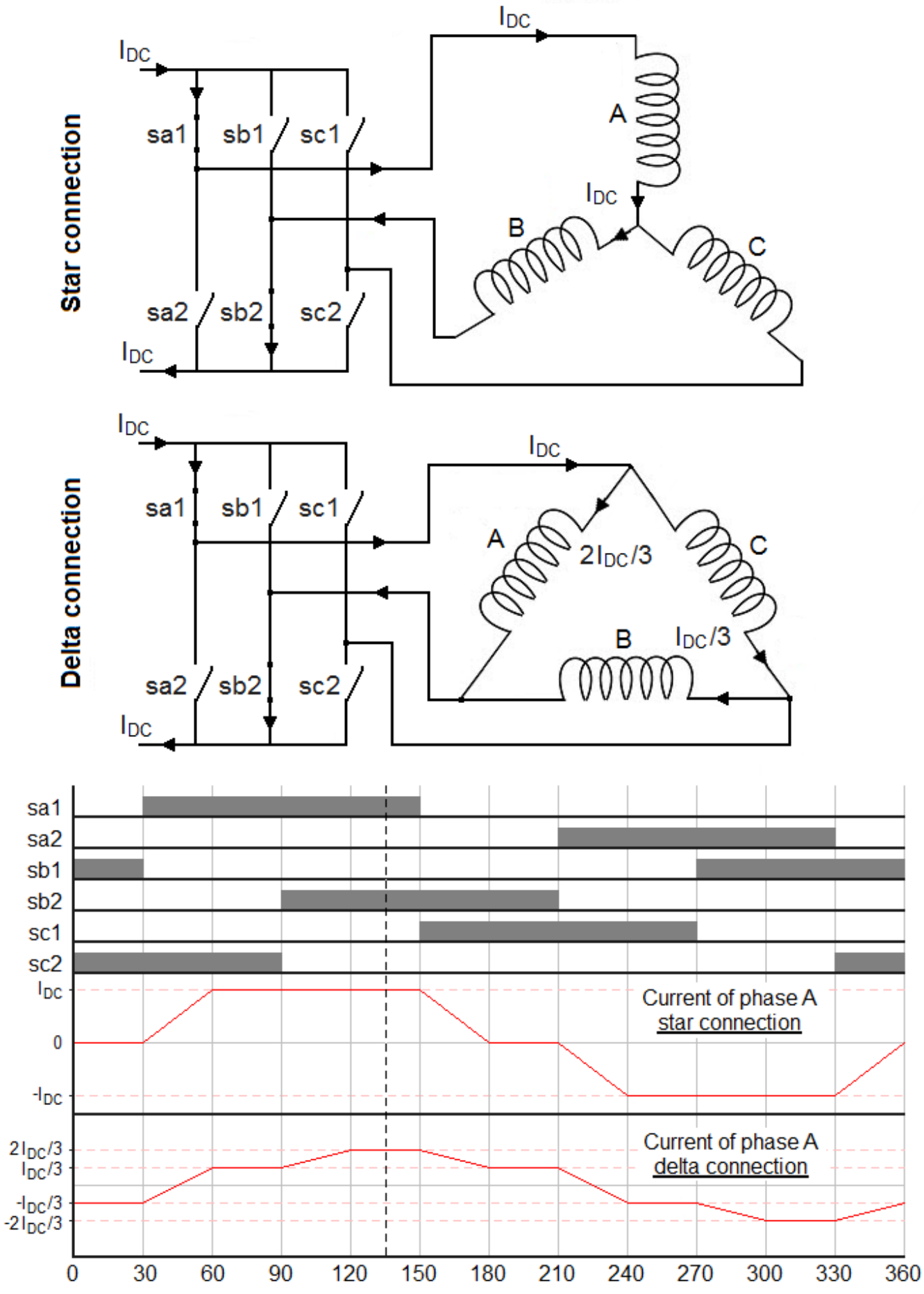


Figure 5.2. Example of trapezoidal current waveforms and distribution for star and delta connection.

Save each field solution checkbox enables (if checked) to store additional data of each FEA solution such as magnetic vector potential distribution and permeability distribution values. These data are not stored by default because of large amount of hard drive space required. Data for each time step are saved in a separate file so the number of files saved is equal to the **Number of points** field value.

To start the analysis, click the **Start magnetostatic simulation** button on the MotorAnalysis-PM main window toolbar. To stop the analysis click the **Stop simulation** button which is to the right of the **Start magnetostatic simulation** button. If the analysis is stopped all the simulation data of the running analysis will be lost.

5.2. Viewing Magnetostatic Analysis results.

The view of the **Plot Wizard** window when Magnetostatic Analysis is chosen is shown in Figure 5.3. Several plot types are available for Magnetostatic Analysis:

- time-series data plot;
- air gap distribution plot and frequency spectrum of the air gap distribution;
- cross-section distribution plot;
- animation.

5.2.1. Time plots.

Time-series data plots are available from the **Time plot** panel of the **Plot Wizard** window. It is organized as a standard MATLAB plotting construction consisting of a set of `subplot` and `plot` functions. **Subplot** checkboxes allow you to control the number of axes or rectangular panes displayed within a current figure window. The corresponding axes are activated or deactivated by a mouse click within a cell of the **Subplot** column. When the subplot is activated, the **change** button allows you to choose quantities to be plotted into the selected axes. Quantities can be chosen from the dialog shown in Figure 5.4. Use Ctrl button to select several quantities. The list of available quantities is given below:

- Stator phase current (phase A, B, C);
- Phase current (d-axis, q-axis);
- Phase voltage (phase A, B, C);
- Phase voltage (d-axis, q-axis);
- Effective advance angle;
- Back-EMF (phase A, B, C);
- Back-EMF (d-axis, q-axis);
- Input electrical power;
- Mechanical power on the rotor shaft;
- Stator winding losses;
- Eddy current magnet losses;

- Electromagnetic torque by Maxwell stress tensor;
- Electromagnetic torque by virtual work method;
- Electromagnetic torque by flux linkage and current;
- Magnet torque (by Maxwell stress tensor);
- Reluctance torque (by Maxwell stress tensor);
- Cogging torque (by Maxwell stress tensor);
- Flux linkage (phase A, B, C);
- Flux linkage, (d-axis, q-axis);

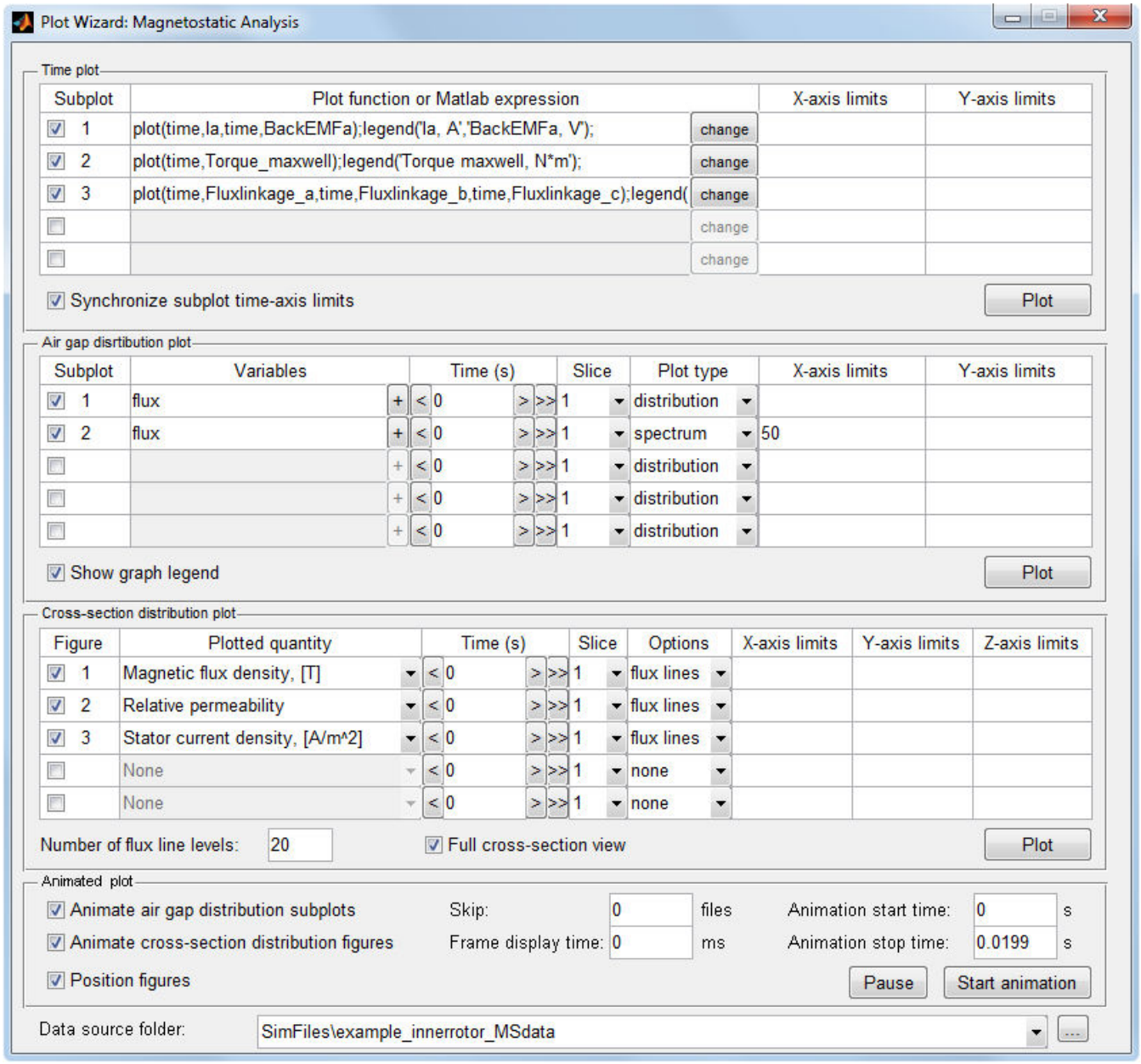


Figure 5.3. Magnetostatic Analysis Plot Wizard window.

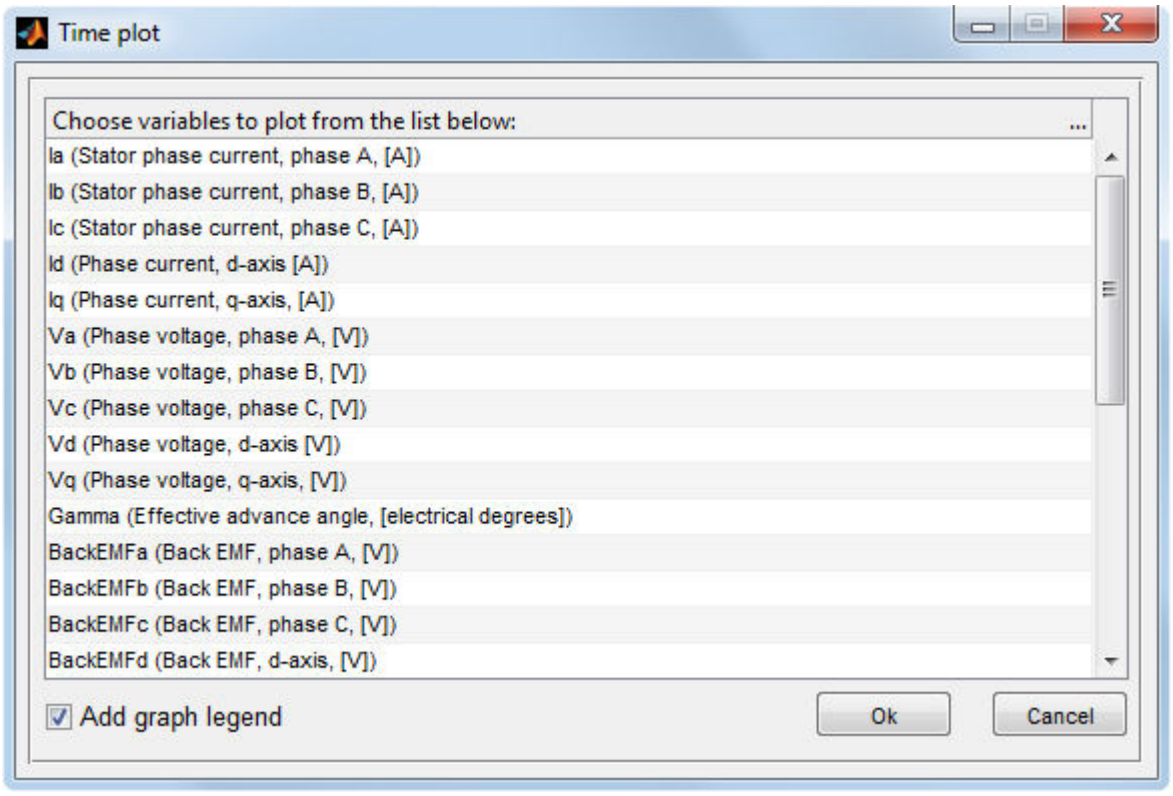


Figure 5.4. Choosing quantities to be plotted.

By clicking the **OK** button of the dialog (Figure 5.4), the MATLAB-expression is constructed to plot the selected quantities appearing in the corresponding line as shown in Figure 5.5 for plotting phase A stator current and back-EMF (first line) and torque (second line). The third line is not active and, therefore, will not be plotted.

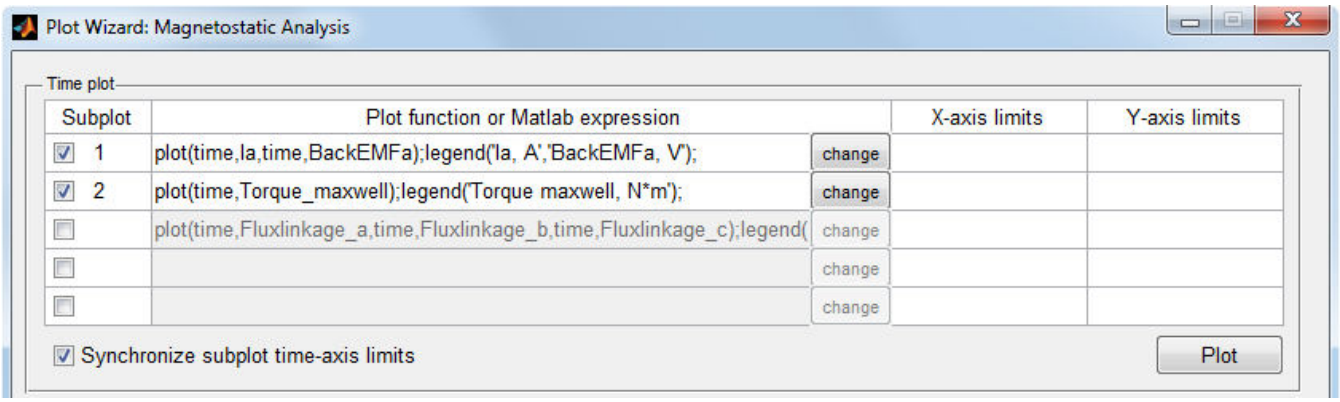


Figure 5.5. Example of using **Plot Wizard** for creating time plots.

When the **Plot** button is clicked, the MATLAB figure window with plots of the selected quantities will appear (see Figure 5.6).

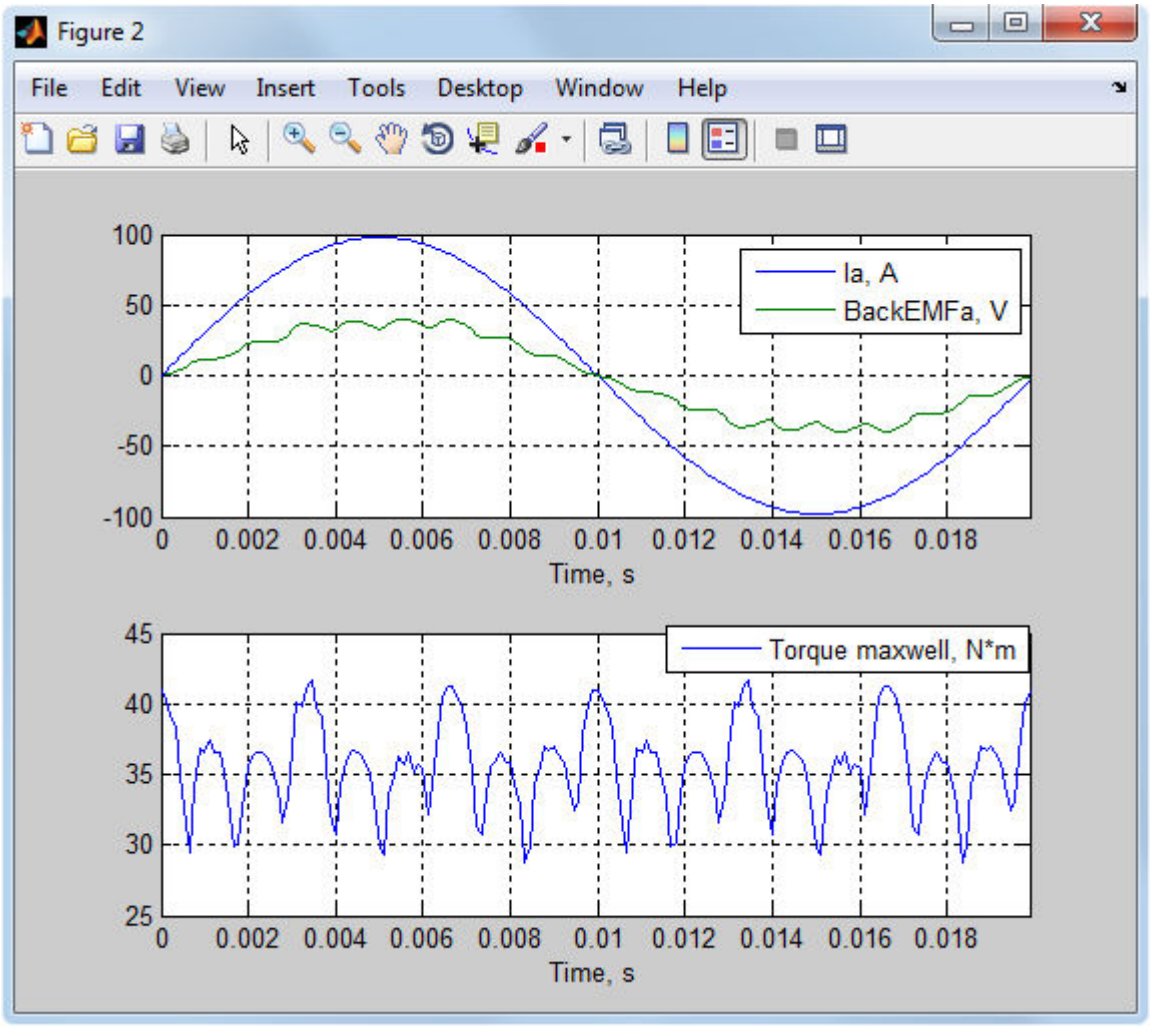


Figure 5.6. MATLAB figure window with time plots of the selected quantities.

As it is seen, the plotting expression consists of the `plot` function with selected variables used as input arguments. If the **Add graph legend** checkbox of the dialog is checked, the `legend` function is added to the plotting expression so the graph legend of the corresponding axes will be shown. All plotting expressions are editable, so you can use any MATLAB plotting options and functions to change the way the plots appear on the screen. You can also change `time` variable to any other variable you would like to use as an input argument of the `plot` function. There is also a list of additional variables which can be used within a plotting expression (see section 9.3).

X-axis limits and **Y-axis limits** fields of the **Time plot** panel allow you to set the x-axis and y-axis limits, respectively, to the specified values. Two limit values within a cell are divided by a space, comma ‘,’ or semicolon ‘;’. If a cell is empty, the limits of the corresponding axis will be chosen automatically. If the **Synchronize subplot time-axis limits** checkbox is checked, all subplots of the

figure will have identical limits along the time-axis when you zoom or pan one of the subplots of the figure.

5.2.2. Air gap distribution plots.

Air gap distribution plots allow you to view the distribution of the particular quantity over the machine's air gap as well as its frequency spectrum. It is available from the **Air gap distribution plot** panel. **Subplot** checkboxes allow you to control the number of axes or rectangular panes displayed within a current figure window. The corresponding axes are activated or deactivated by a mouse click within a cell of the **Subplot** column. When the subplot is activated, the **"+"** button allows you to choose quantities to be plotted from the dialog shown in Figure 5.7. Use Ctrl button to select several quantities.

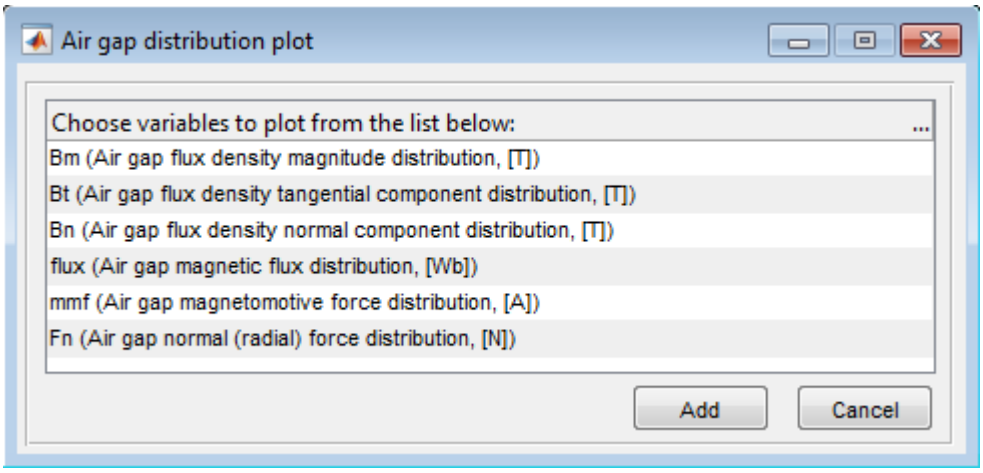


Figure 5.7. Choosing quantities to be plotted.

The following quantities are listed in the dialog of Figure 5.7:

Quantity	Comments
Bm (Air gap flux density magnitude distribution, [T])	Defined as $B_m = \sqrt{B_t^2 + B_n^2}$ where B_n and B_t – normal and tangential components of the magnetic flux density in the center of the air gap, as shown in Figure 2.4.
Bt (Air gap flux density tangential component distribution, [T])	B_n and B_t – normal and tangential components of the magnetic flux density in the center of the air gap, as shown in Figure 2.4.
Bn (Air gap flux density normal component distribution, [T])	

flux (Air gap magnetic flux distribution, [Wb])	Integration of the magnetic flux distribution over an air gap always gives zero: $\Phi_{airgap} = l \oint_{airgap} B_n \cdot d\lambda = 0$, where l – lamination length in z direction. Air gap magnetic flux distribution is calculated over inner and outer contours as shown in Figure 2.4 and the actual values are resulted as the average.
mmf (Air gap magnetomotive force distribution, [A])	Integration of the magnetomotive force distribution over an air gap always gives zero: $F_{airgap} = \frac{1}{\mu_0} \oint_{airgap} B_t \cdot d\lambda = 0$ Air gap magnetomotive force distribution is calculated over inner and outer contours as shown in Figure 2.4 and the actual values are resulted as the average.
Fn (Air gap normal (radial) force distribution, [N])	According to Maxwell stress tensor method the normal force in each point of the round path can be expressed as the following: $F_n = -\frac{B_n^2 - B_t^2}{2\mu_0} d \cdot l$, where l – lamination length in z direction, d – length of the path in the center of the air gap between two consecutive points, μ_0 – permeability of the free space, B_n and B_t – normal and tangential components of the magnetic flux density in the center of the air gap, as shown in Figure 2.4.

By clicking the **Add** button of the dialog (Figure 5.7), the selected quantities are added to the corresponding line separating them by commas as shown in Figure 5.8. Each line of the **Variables** column is editable, so you can you use MATLAB arithmetic operations to obtain desired plots. For example, the second and third lines in Figure 5.8 produce plots of the tangential air gap force density distribution and the radial air gap force density distribution, respectively, where `mu0` – variable corresponding to the permeability of free space. According to Exp. 2.6 the tangential air gap force produces the electromagnetic torque so the plot of the electromagnetic torque distribution over an air gap can be obtained.

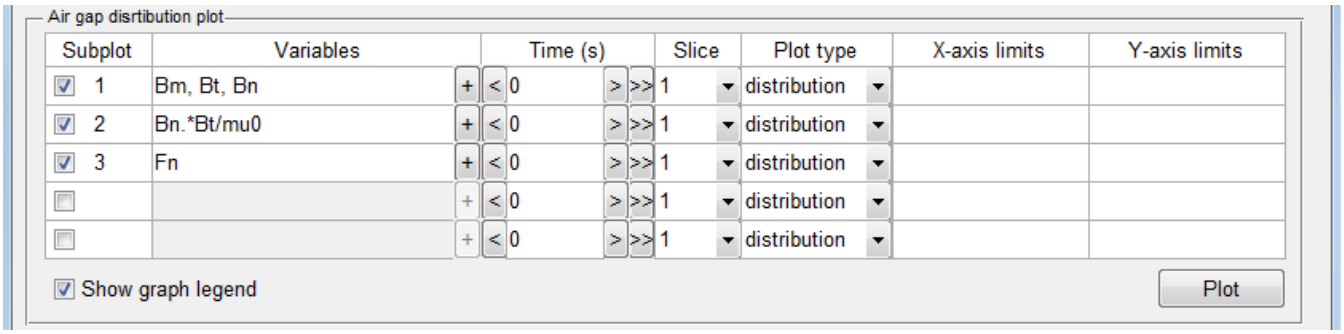


Figure 5.8. Example of using **Plot Wizard** for creating air gap plots.

Time fields of the **Air gap distribution plot** panel allow you to specify the time point which the selected quantities are plotted for. By default, zero time point is set up. Plotting for the time points other than zero is only possible, if the file containing data for the desired time point was previously saved. These data-files are being saved when **Save each field solution** is checked in the MotorAnalysis-PM main window. So, if you are interested in viewing the air gap distribution plots for different time points make sure that the corresponding data-files are being saved. The folder used as a source of data-files is specified in the **Data source folder** field located at the bottom of the **Plot Wizard** window. You can change it by clicking the button to the right, if needed.

X-axis limits and **Y-axis limits** fields allow you to set the x-axis and y-axis limits, respectively, to the specified values in the same way as for the **Time plot** panel. **Slice** fields allow you to choose the machine's cross-section which the selected quantities are plotted for, if several slices are used (see section 2.4 for more details on multi-slice simulations).

Besides the air gap distribution, it is also possible to calculate the air gap harmonic components for the selected quantities. To obtain the frequency spectrum, select the ***spectrum*** item in the corresponding **Plot type** pop-up menu. An example of plotting the air gap distribution of the magnetic flux and its spectrum is shown in Figure 5.9. Only first 50 harmonics are shown, since the value specified in the corresponding **X-axis limits** field is 50 (if the minimum limit equals to 0, it can be omitted).

Show graph legend field allows you to choose whether the graph legend will be displayed.

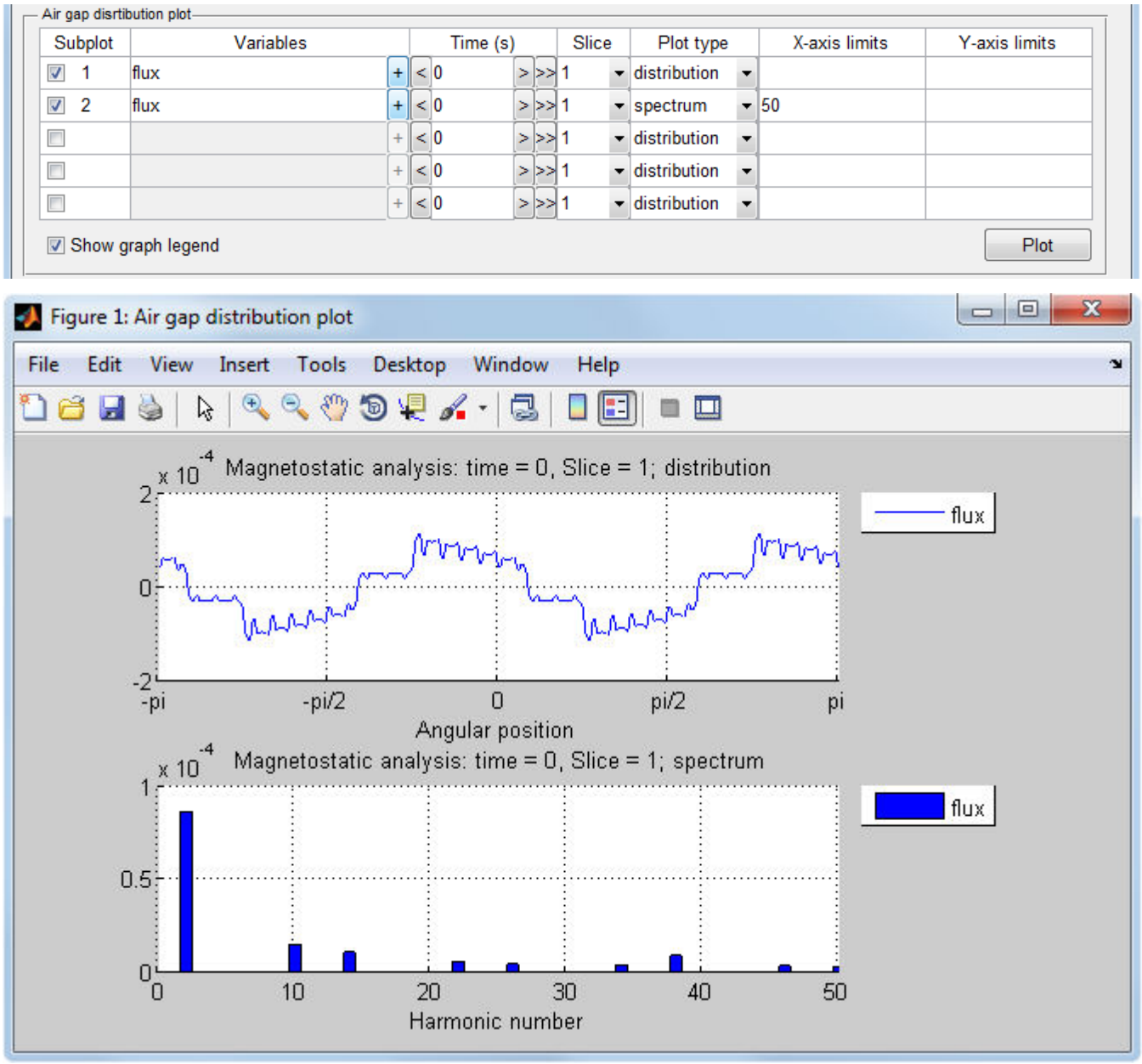


Figure 5.9. Plotting of the air gap distribution of the magnetic flux and its spectrum.

5.2.3. Cross-section distribution plots.

Cross-section distribution plots are available from the **Cross-section distribution plot** panel of the **Plot Wizard** window. As opposed to two previous plot types, the cross-section distribution for each quantity is plotted in a separated window which contains only one axes. **Figure** checkboxes allow you to control the number of windows appearing when the **Plot** button is clicked. The corresponding figure is activated or deactivated by a mouse click within a cell of the **Figure** column. When the figure is activated, the corresponding **Plotted quantity** pop-up menu allows you to choose the quantity to be plotted. If ***None*** is chosen, the machine's cross-section geometry will be plotted.

The following items are available from the **Plotted quantity** pop-up menu:

- Magnetic vector potential, [T*m];
- Magnetic flux density, [T];
- Magnetic field intensity, [A/m];
- Relative permeability;
- Stator current density, [A/m^2];
- Squared flux density, [T];
- Magnetic field energy density, [J/m^3];
- Joule loss density, [W/m^3];
- Iron loss density, [W/m^3];
- Demagnetizing field, [A/m];
- Demagnetizing field, [% of Hcj];
- Finite element mesh.

Time fields of the **Cross-section distribution plot** panel allow you to specify the time point which the selected quantity is plotted for. By default, zero time point is set up. Plotting for the time point other than zero is only possible, if the file containing data for the desired time point was previously saved. These data-files are being saved when **Save each field solution** is checked in the MotorAnalysis-PM main window. So, if you are interested in viewing the cross-section distribution plots for different time points make sure that the corresponding data-files are being saved. The folder used as a source of data-files is specified in the **Data source folder** field located at the bottom of the **Plot Wizard** window. You can change it by clicking the button to the right, if needed.

Slice fields allow you to choose the machine's cross-section which the selected quantity is plotted for, if several slices are used (see section 2.4 for more details on multi-slice simulations).

Options field allows you to show the magnetic flux lines (if ***flux lines*** is chosen) or magnetic flux arrows (if ***flux arrows*** is chosen) on the corresponding plot. Flux arrows are plotted such that the direction of the arrow indicates the direction of the flux and the size of the arrow indicates the magnitude of the flux density. **Number of flux line levels** field allows you to alter the flux lines density. If periodic/antiperiodic boundary conditions are used, by default, only part of the cross-section (as it appears in **Mesh Editor**) will be plotted. Check the **Full cross-section view** checkbox if you want to plot the whole cross-section.

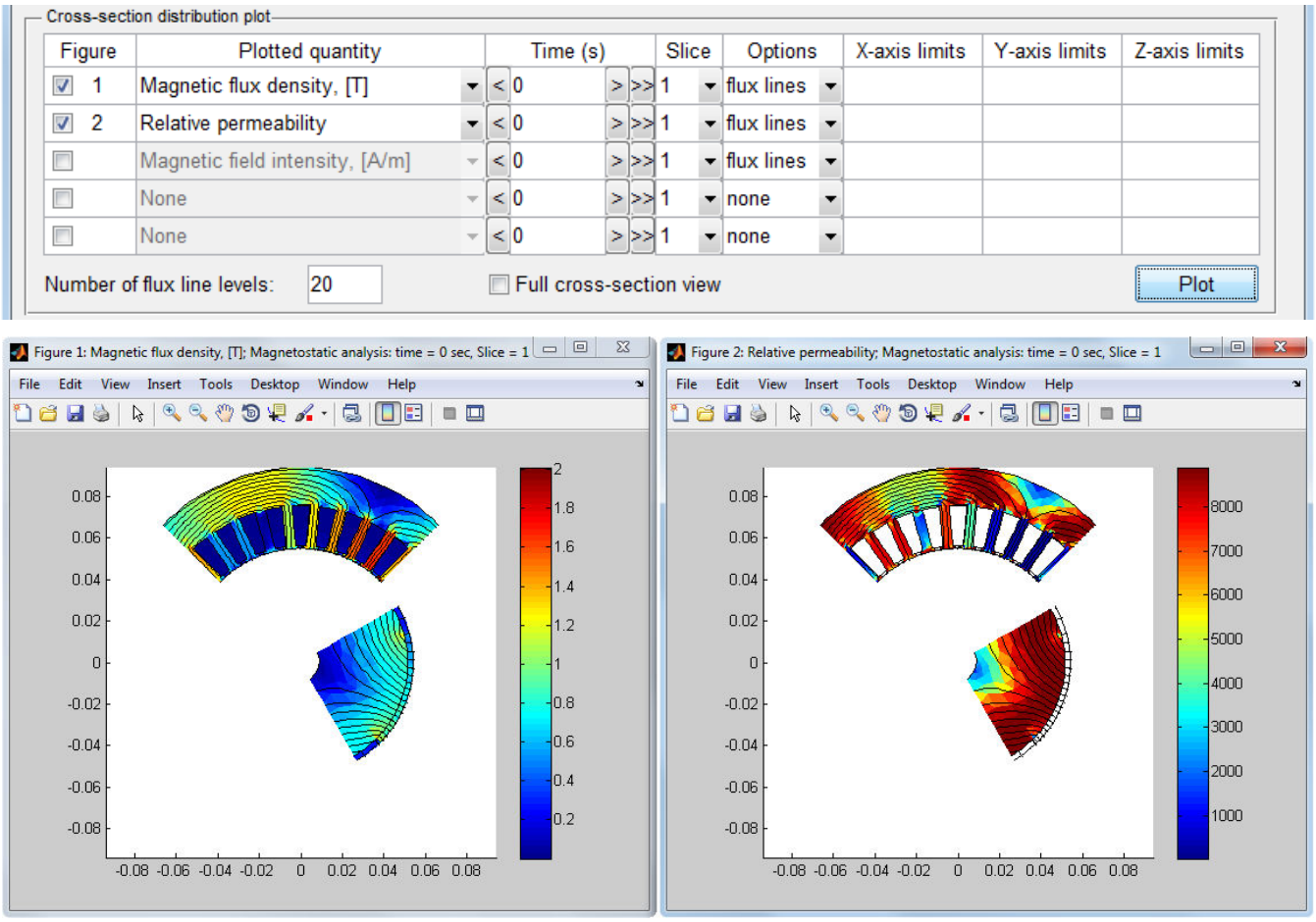


Figure 5.10. Example of using **Plot Wizard** for creating cross-section distribution plots.

X-axis limits and **Y-axis limits** fields allow you to set the x-axis and y-axis limits, respectively. If x-axis and y-axis limits are not specified, their values will be determined by the size of the machine's cross-section. **Z-axis limits** field sets the color scale limits to specified minimum and maximum values divided by a space, comma ',' or semicolon ';'.Values between z-axis limits are linearly mapped to the used color scale (colormap). Data values less or greater than specified z-axis limits are mapped to the minimum limit or to the maximum limit, respectively.

An example of plotting the cross-section distribution of the magnetic flux density and relative permeability is shown in Figure 5.10. When the **Plot** button is clicked, two windows appear. As it is seen, the **flux lines** option is chosen for the both figures, so the flux lines are additionally shown. Since the third figure is not active, the Magnetic field intensity is not plotted. Note that only part of the machine's cross-section is shown, since **Full cross-section view** is not checked.

5.2.4. Animation.

Animated plot panel is used to create animated sequences of the air gap distribution plots and/or the cross-section distribution plots. **Animate air gap distribution subplots** and **Animate cross-section distribution figures** checkboxes allow you to choose plot types to be animated. If **Animate air gap**

distribution subplots is checked, all selected subplots of the **Air gap distribution plot** panel will be animated. Similarly, if **Animate cross-section distribution figures** is checked, all selected figures of the **Cross-section distribution plot** panel will be animated. If **Position figures** control is checked, the size and location on the screen for all figure windows will be specified automatically so that the windows fit best the screen size.

Skip and **Frame display time** fields allow you to specify the way how frames change each other while the animation is in progress. **Skip** field specifies the number of data-files or frames to be skipped. So, if **Skip** field is set to 0, data for each time step will be shown on the screen, if **Skip** field is set to 1, data for each second time step will be shown, if **Skip** field is set to 2 – for each third time step and so on. **Frame display time** field specifies the time each frame is displayed on the screen. Note that the least time the frame is displayed is limited by the animation function runtime. So, if the **Frame display time** field is 0, the actual time the frame is displayed on the screen will be the least possible in this case.

Animation start time and **Animation stop time** fields set up the time interval for the animation. **Data source folder** field specifies the folder with data-files to be animated. The same folder is used for animations and for air gap distribution and cross-section distribution plots. Using animation is only possible, if the files containing data for the time interval to be animated were previously saved.

To start the animation, click the **Start animation** button. The current time point, animated quantity and other information is displayed at the top of each window involved in the animation. The animation continues until the time specified in the **Animation stop time** field is reached or until all windows involved in the animation are closed. You can also pause the animation using the **Pause** button. While the animation is in progress or paused, you can use all MATLAB figure tools, for example, changing the scale and position of the plots.

6. D-Q ANALYSIS

In MotorAnalysis-PM the D-Q Analysis implies the steady-state D-Q analysis. Dynamic D-Q model is discussed in the next chapter. D-Q Analysis allows to estimate steady-state machine parameters and characteristics such as torque – speed curve, torque – advance angle curve and etc. as well as to compute the efficiency map. Since the variation of machine parameters for different rotor positions is not taken into account the accuracy of D-Q Analysis can be lower comparing with Magnetostatic Analysis and Dynamic FEA.

6.1. Building a D-Q model.

The view of the main window when the D-Q Analysis is chosen is shown in Figure 6.1. In order to use the D-Q Analysis the D-Q model of the machine should be previously built. Use the **Build D-Q model** button to compute the D-Q model parameters of the machine. The short description of the d-q representation of a permanent magnet machine is presented in section 2.5. While the D-Q model is being built the model parameters L_d, L_q, L_{dq}, Ψ_{md} and Ψ_{mqd} are derived from the FEA solution for each pair of supply current and advance angle values to include saturation and cross-saturation effects into account. The supply current values are defined by the **Maximum RMS current** and **Current step** field values. The advance angle values are in the range between 0^0 and 360^0 electrical degrees with step defined by the **Advance angle step** field value. **Number of rotor positions** field specifies the number of rotor positions for which the D-Q model parameters are averaged over. Once the D-Q model is built, different machine characteristics can be obtained.

6.2. Viewing D-Q Analysis results.

The view of the **Plot Wizard** window when the D-Q Analysis is chosen is shown in Figure 6.2. Since it assumes that all current and voltage waveforms are sinusoidal the D-Q analysis cannot be used to analyze the six-step drive.

The plotted quantities can be displayed as a function of ***Speed***, ***Supply current*** or ***Advance angle*** depending on the chosen item in the **X-axis quantity** pop-up menu. Speed, current and advance angle values should be entered in the corresponding fields. If a list of values is used, the values can be separated by space, comma or semicolon, alternatively the statement ‘start:step:stop’ can be used to enter linearly spaced values between ‘start’ and ‘stop’.

There are three **Model types** to choose for the D-Q Analysis: ***D-Q with sinusoidal supply***, ***D-Q with PWM supply*** and ***FEA based***. ***D-Q with sinusoidal supply*** assumes ideal sinusoidal current and voltage waveforms, ***D-Q with PWM supply*** uses the dynamic D-Q simulation (refer to chapter 7) and corresponding drive type (***Current hysteresis PWM*** or ***Space vector PWM*** specified in the **Drive Settings** window as described in section 4.4) to obtain the result. ***FEA based*** is the same as D-Q with

sinusoidal supply but the result is determined directly from FEA solution (only for one rotor position) so building of D-Q model is not required.

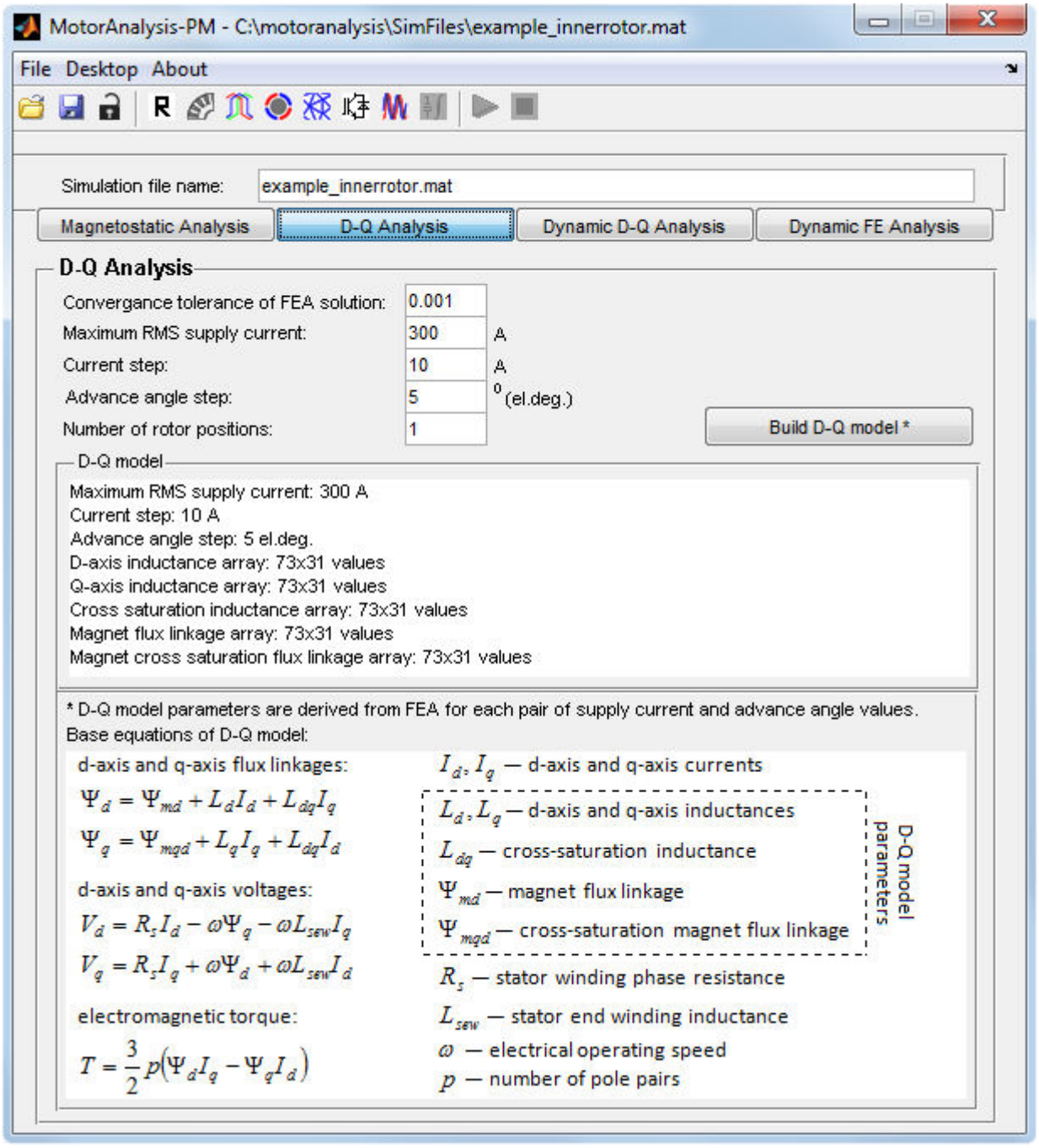


Figure 6.1. MotorAnalysis-PM main window for D-Q Analysis.

Max. phase voltage selection defines either the ideal current source with no voltage limit is used (if ***unlimited voltage*** is chosen) or the limited supply voltage is used and how the **Max. RMS phase voltage** available from the inverter is determined. If ***by Vdc and PWM method*** is chosen the **Max. RMS phase voltage** is determined based on the DC supply voltage V_{dc} and drive type (***Current***

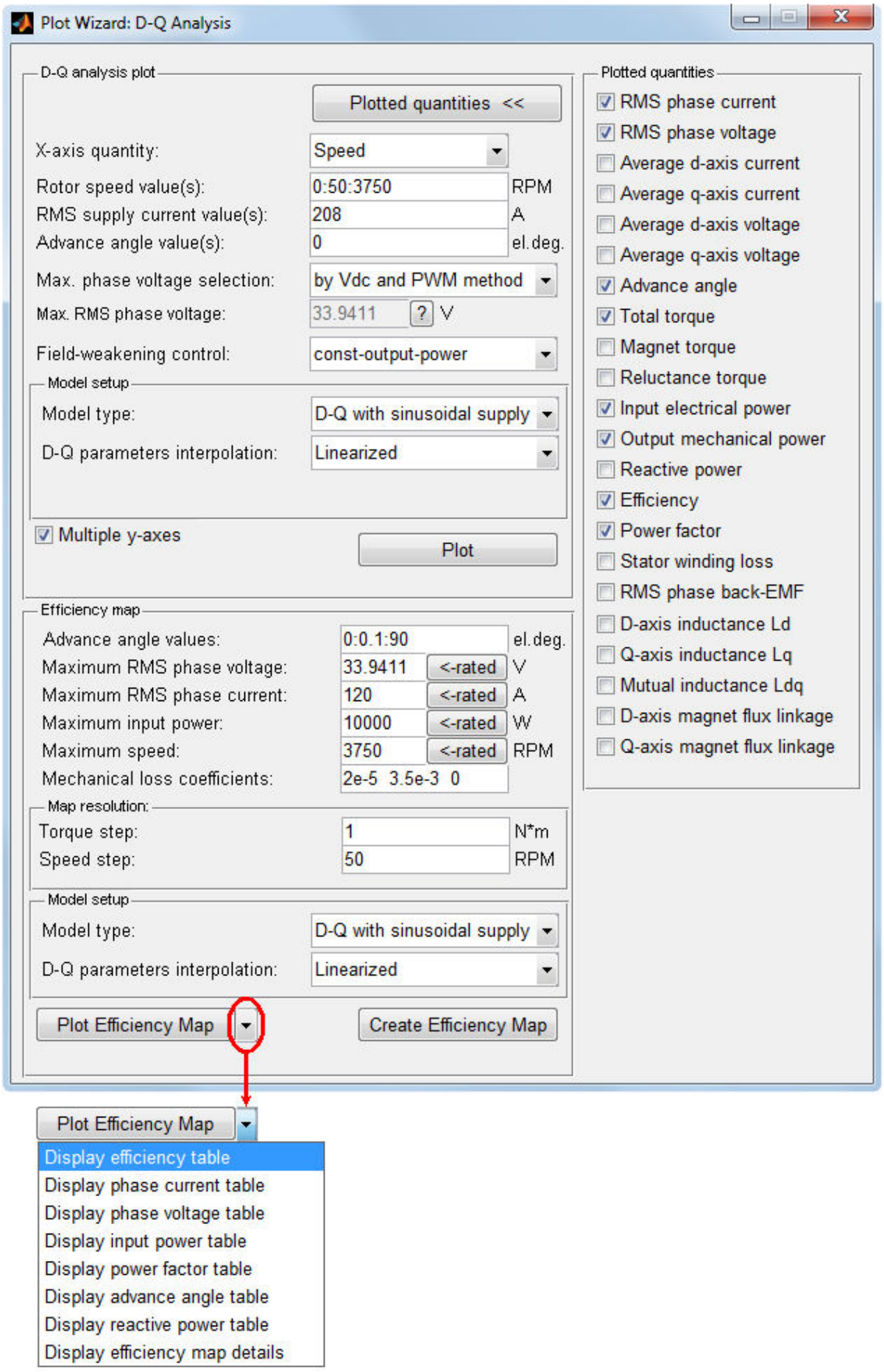


Figure 6.2. D-Q Analysis Plot Wizard window.

hysteresis PWM or ***Space vector PWM***) specified in the Drive Setting window as well as stator winding connection (star or delta). For the space vector PWM the maximum inverter phase peak voltage is $\frac{V_{DC}}{\sqrt{3}}$

For the current hysteresis PWM in order to keep the voltage and current sinusoidal, the inverter phase peak voltage should not exceed $\frac{V_{DC}}{2}$. Note that for delta-connected winding the phase voltage of the machine is line-to-line voltage of the inverter.

If the motor operates with limited voltage the field-weakening operation above the base speed can be analyzed. There two field-weakening strategies available: ***max-torque-limited-current*** and ***const-output-power***. If the field-weakening is applied the current and advance angle are adjusted to meet the corresponding requirements of the field-weakening control algorithm. If ***const-advance-angle*** is chosen, the current and advance angle are no longer controlled - current starts to naturally decrease above the base speed.

D-Q parameters interpolation specifies how D-Q model parameters are interpolated. ***Linearized*** interpolation is sufficient in most cases provided that the D-Q model has been build with enough number of supply current and advance angle pairs.

When **Model type** is set to ***D-Q with PWM supply*** the simulation speed/accuracy setup can be additionally adjusted by clicking the **Simulation speed/accuracy** button as shown in Figure 6.3.

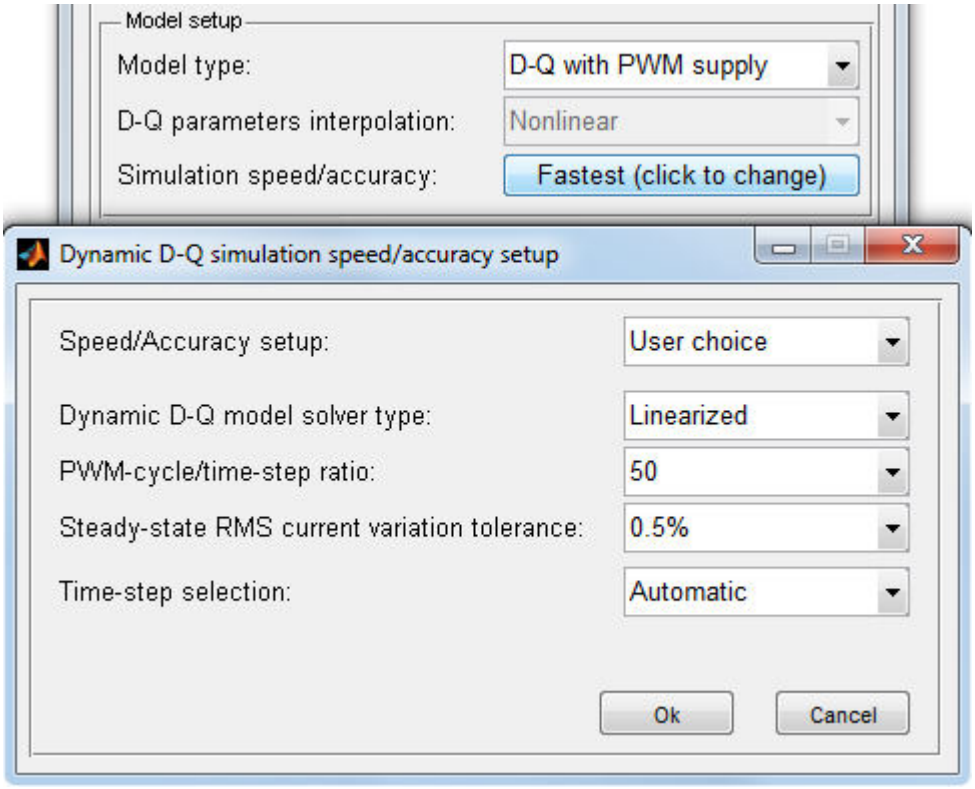


Figure 6.3. Dynamic D-Q simulation speed/accuracy setup.

You can choose one of the predefined setups (***Fastest***, ***Balanced*** or ***Accurate***) from the **Speed/Accuracy setup** pop-up menu or manually adjust the simulation speed/accuracy setup choosing the ***User choice*** item.

Dynamic D-Q model solver type set to ***Linearized*** is sufficient in most cases. Be careful when using ***Linear*** solver type since in this case fixed D-Q model parameters are used – linear solver can only be used if the current waveform is very close to sinusoidal.

PWM-cycle/time-step ratio determines the choice of the time step when **Time-step selection** is set to ***Automatic***. Good practice is to manually specify the time-step since in some cases the optimal time-step cannot be determined by the automatic selection.

Steady-state RMS current variation tolerance defines the acceptable variation of RMS current from one period to the next for the steady-state. The simulation stops when the steady state is reached.

Several examples of D-Q Analysis plots are shown in Figures 6.4-6.5. Figure 6.4 shows the plot corresponding to the D-Q Analysis setup shown in Figure 6.2 with ideal sinusoidal supply and "constant output power" field-weakening control above the base speed. Figure 6.5 compares plots obtained with ideal sinusoidal supply with no voltage limit (***D-Q with sinusoidal supply***) and with space vector PWM inverter supply (***D-Q with PWM supply***) for 1500RMP rotor speed. Supply current is varied in the range between 0 and 300A in both cases. As shown for the ideal sinusoidal supply (top) the phase current changes from 0 to 173A (delta-connected winding), the phase voltage is freely increasing and the advance angle is strictly 0 electrical degrees. For the PWM supply (bottom) after some point the inverter is not longer able to maintain the desired current since the phase voltage reaches its limit. Note that the effective advance angle is shown for PWM supply.

6.3. Efficiency map.

Another option of the D-Q Analysis is creation of the efficiency map. The efficiency map shows the maximum efficiency which can be reached for the certain speed and load torque. The example of the plot of the efficiency map for the motor mode is shown in Figure 6.6. Note that the part of efficiency map above the input power limit 10kW specified in **PlotWizard** is not shown. Along with the efficiency map some additional maps of the supply current, voltage, power, advance angle, power factor, reactive power as well as maximum power (power envelop) are available (see Figure 6.7).

Advance angle values field specifies the range of advance angles the efficiency map is computed for. Use statement 'start:step:stop' to enter linearly spaced values between 'start' and 'stop', where 'step' determines the accuracy of the efficiency map. Advance angle values in the range between 0^0 and 90^0 electrical degrees corresponds to the motor operation mode, the range of advance angle values between 90^0 and 180^0 degrees corresponds to the generator mode. **Maximum RMS phase voltage, Maximum RMS phase current** and **Maximum input power** define the supply voltage, current and power

limitations applied while the efficiency map is computed. If the **Maximum input power** field is left empty when the input power is not limited (limited only by maximum voltage and current).

Button '**<-rated**' to the right of some fields allows you to set up the corresponding parameter to its rated value specified in the **Rated Data** window.

Use the **Mechanical loss coefficients** field to apply mechanical losses while the efficiency map is plotted. Mechanical loss coefficients are stored in a vector whose elements are the coefficients in descending powers of the mechanical loss equation. For mechanical loss coefficients shown in Figure 6.2 the mechanical loss equation is as follows:

$$W_{mech.loss}=0.00002(rpm)^2+0.0035(rpm)+0,$$

where *rpm* – rotor speed in RPM.

Number of points (speed-torque pairs) at which the efficiency map is computed is defined by the **Map resolution** panel.

D-Q parameters interpolation specifies how D-Q model parameters are interpolated. ***Linearized*** interpolation is sufficient in most cases provided that the D-Q model has been build with enough number of supply current and advance angle pairs. Efficiency map is computed assuming ideal sinusoidal current and voltage waveforms.

To compute the efficiency map click the **Create Efficiency Map** button. Once the efficiency map is computed for the first time it will be saved within the simulation file. Every time you compute the efficiency map once again you will be asked whether you want to replace previously computed efficiency map with the new one. Use the **Plot Efficiency Map** button to plot the efficiency map previously saved. There are also some additional options available to the right of the **Plot Efficiency Map** button as shown in Figure 6.2. For example, you can display the efficiency map as a table choosing the **Display efficiency table** item.

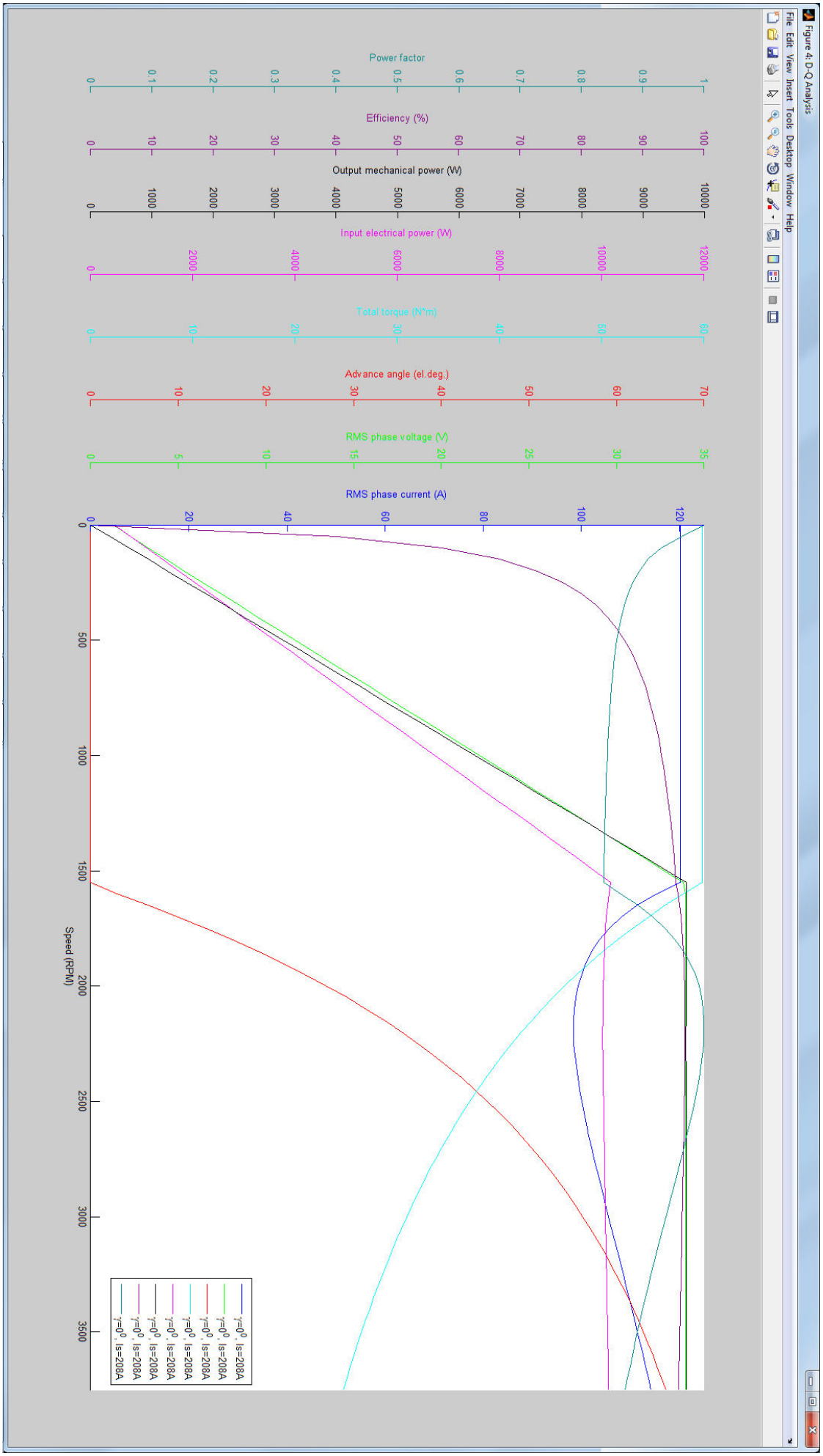


Figure 6.4. D-Q Analysis plot with ideal sinusoidal supply and ***const-output-power*** field-weakening control above the base speed.

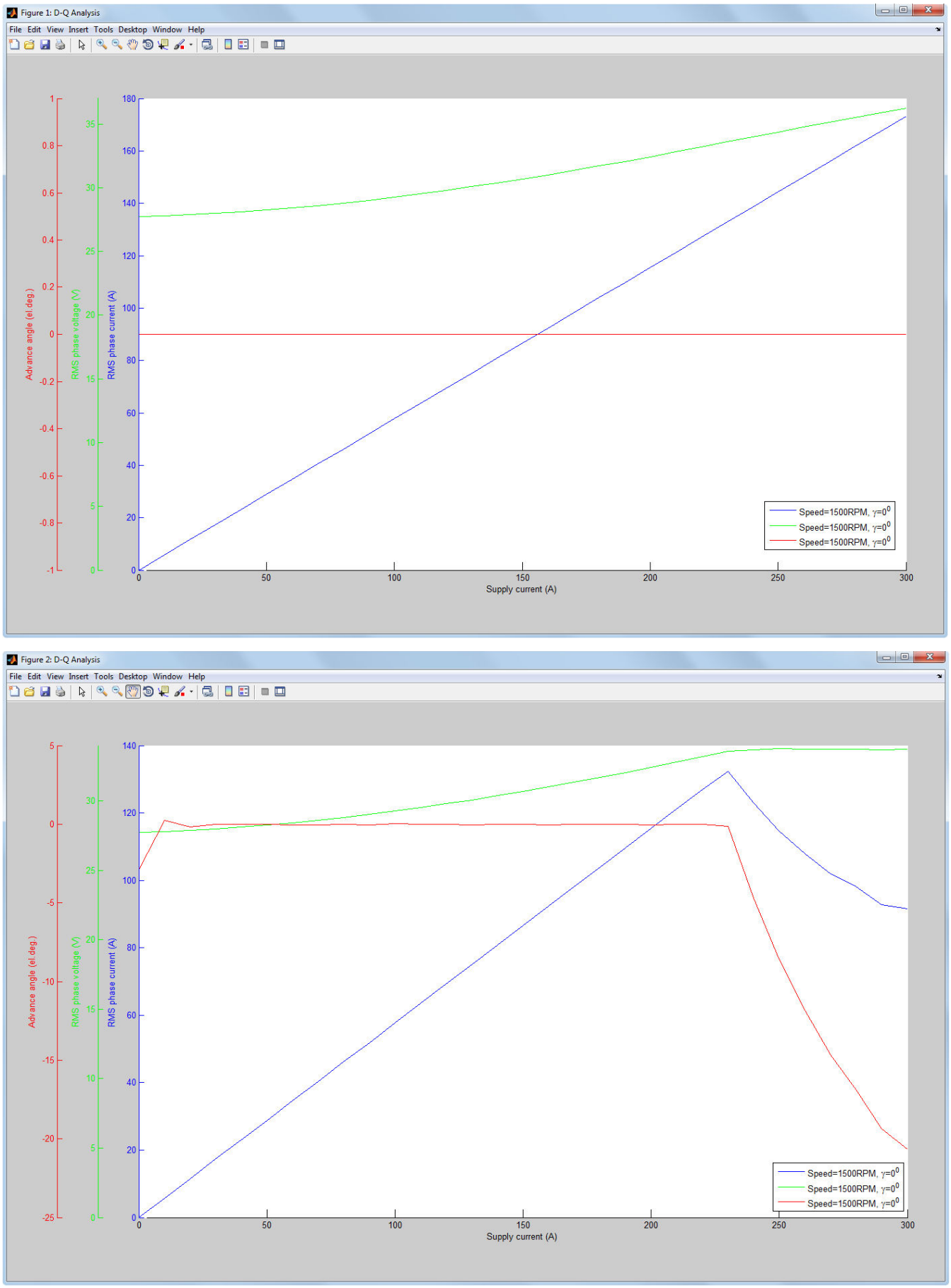


Figure 6.5. Comparison between ***D-Q with sinusoidal supply*** and ***D-Q with PWM supply*** model types.

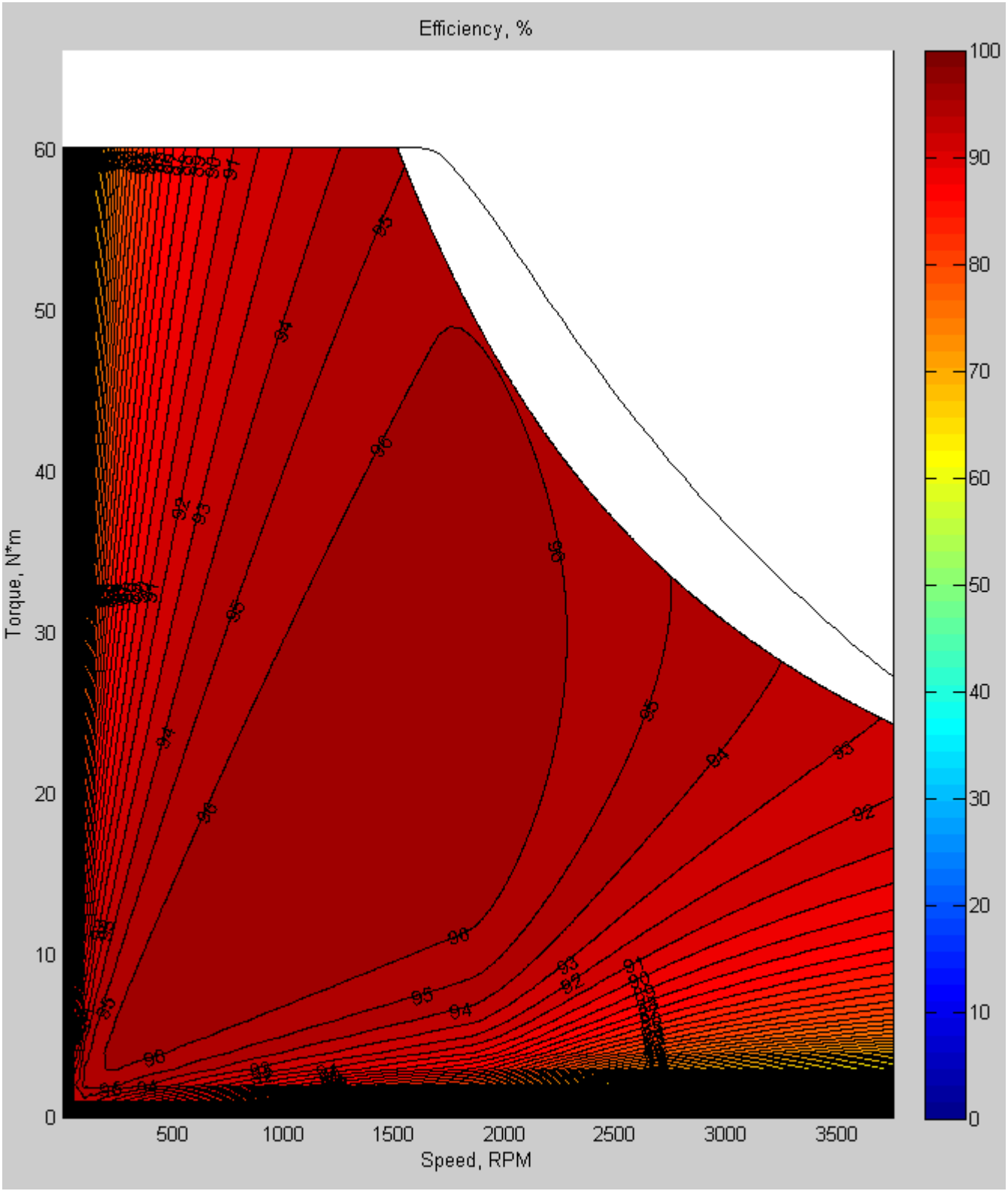


Figure 6.6. Efficiency map.

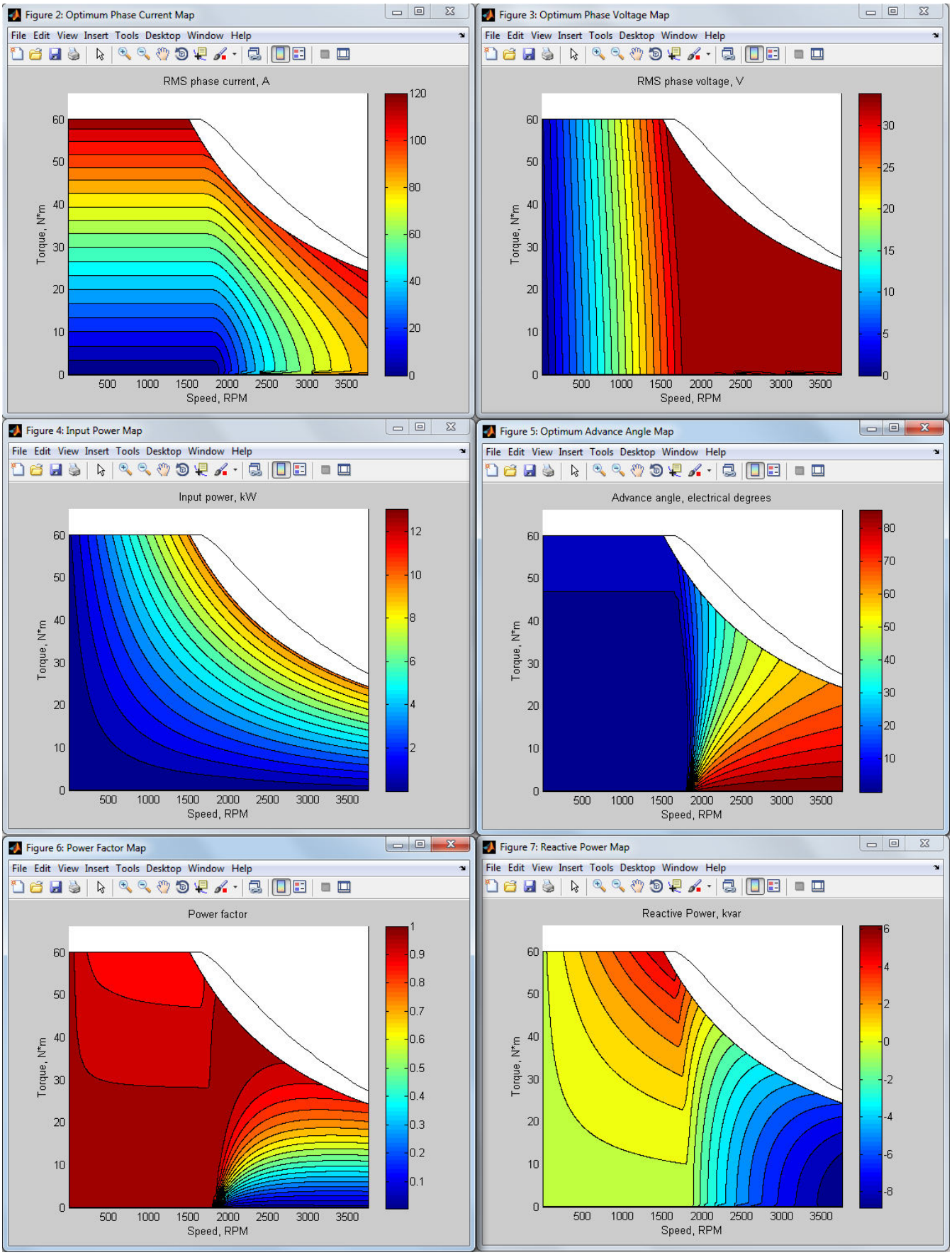


Figure 6.7. Plots of additional maps computed together with efficiency map.

7. DYNAMIC D-Q ANALYSIS

Dynamic D-Q Analysis is based on Exp. 2.11 and allows you to simulate the dynamic behavior of the machine taking into account nonsinusoidal voltage and current waveforms imposed by inverter switching. In order to avoid misunderstanding of the results you should be aware of the following limitations of the Dynamic D-Q Analysis:

- Back-EMF waveform is assumed to be sinusoidal. To examine the Back-EMF waveform use the Magnetostatic Analysis or Dynamic FE Analysis instead.
- States of the diodes are not considered while simulating the inverter. In some cases taking into consideration states of the diode is critical for getting the correct results, therefore the Dynamic D-Q Analysis is not suitable for simulation of the six-step drive with 120 degrees switch duty cycle.

7.1. Running Dynamic D-Q Analysis.

The view of the main window when the Dynamic D-Q Analysis is chosen is shown in Figure 7.1. In order to use the Dynamic D-Q Analysis the D-Q model of the machine described in the previous chapter should be built. Use toolbar buttons  and  to start and stop the simulation. If the simulation is stopped all the data of the running simulation will be lost.

Solver type specifies how D-Q model parameters are interpolated. ***Linearized*** solver type is sufficient in most cases provided that the D-Q model has been build with enough number of supply current and advance angle pairs. Be careful when using ***Linear*** solver since in this case fixed D-Q model parameters are used – linear solver can only be used if the current waveform is very close to sinusoidal, i.e. the switching ripple in the current is considerably small comparing with the current amplitude.

Be aware that **Time step** should be small enough comparing with the PWM sampling period to get accurate results. Control the ***Discretization error*** in the **Time Average Quantities** window shown in Figure 7.3 to adjust the time step. It is recommended that the discretization error does not exceed 1%.

There are two ways to stop the dynamic D-Q simulation in MotorAnalysis-PM: by specifying the **Simulation stop time value** or by reaching the steady state, i.e. the simulation will be automatically stopped when the steady state is reached. **Steady-state RMS current variation tolerance** defines the acceptable variation of RMS current from one period to the next to check if the steady-state is reached.

The phase current of the machine depends on the **RMS supply current** field value and on the stator winding connection. Since the supply current is a line current, for star connection the phase current is equal to the supply current, for delta connection the phase current is equal to the supply current divided by $\sqrt{3}$. **Advance angle** determines an angle between the current phasor and q-axis of the rotor as shown in Figure 2.6. For the current hysteresis and space vector PWM drive types the current control algorithm is applied to adjust the inverter voltage in order to maintain d-axis and q-axis current components defined by the **RMS supply current** and **Advance angle** field values. For the space vector

PWM the field oriented current control is used, for the current hysteresis PWM the hysteresis (bang-bang) current control is used. For the six-step drive the current is not controlled – the current is determined by the DC voltage, rotor speed, switch duty cycle (120 or 180 electrical degrees) and commutation advance angle.

Rotor speed dependency – specifies whether the rotor speed is fixed (when ***Fixed speed simulation*** is chosen) or varies depending on the load and electromagnetic torque of the motor (when ***Variable speed simulation*** is chosen). If ***Fixed speed simulation*** is chosen you should also specify the speed in the field appearing to the right. If ***Variable speed simulation*** is chosen the initial speed should be specified.

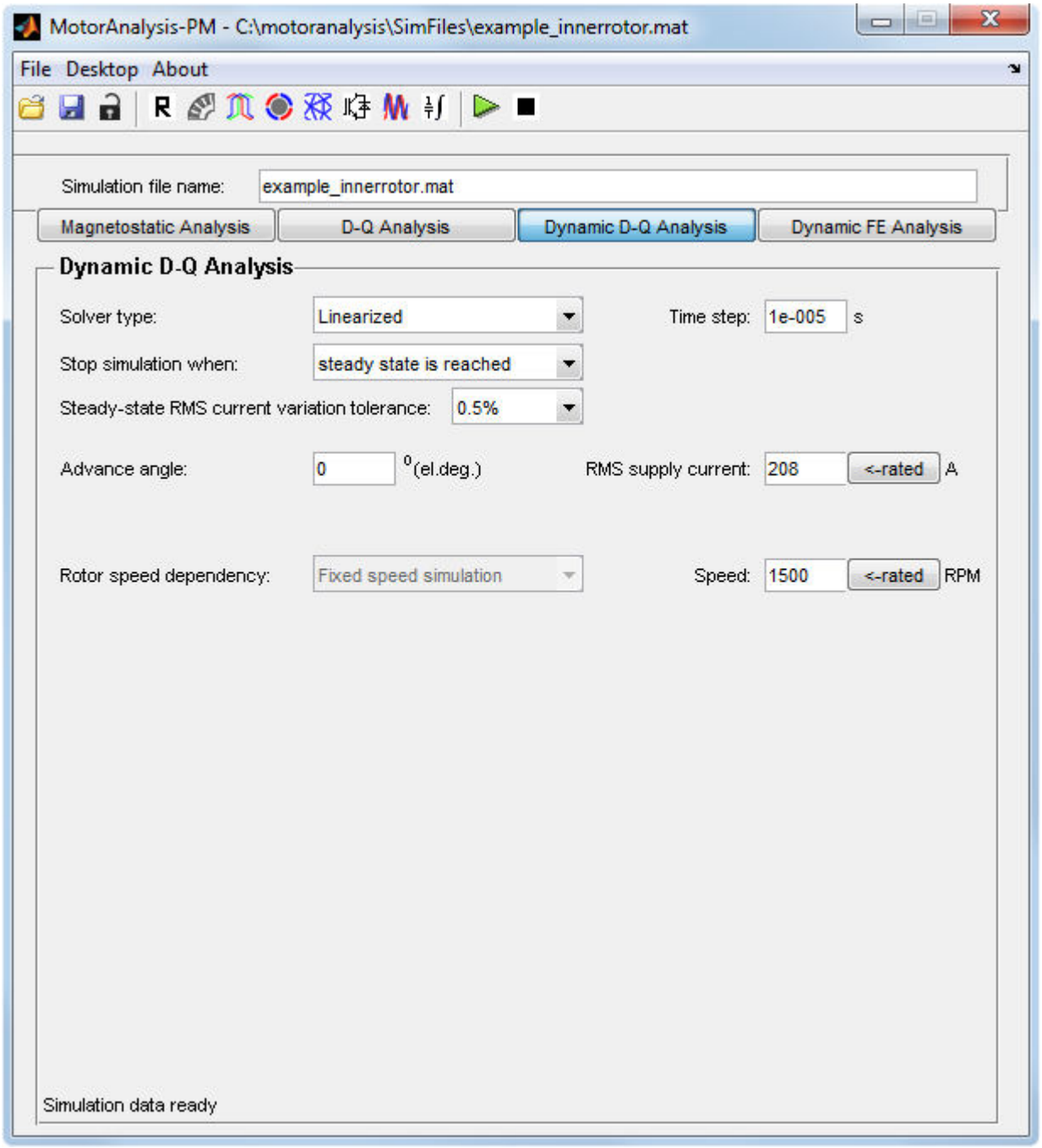


Figure 7.1. MotorAnalysis-PM main window for the Dynamic D-Q Analysis.

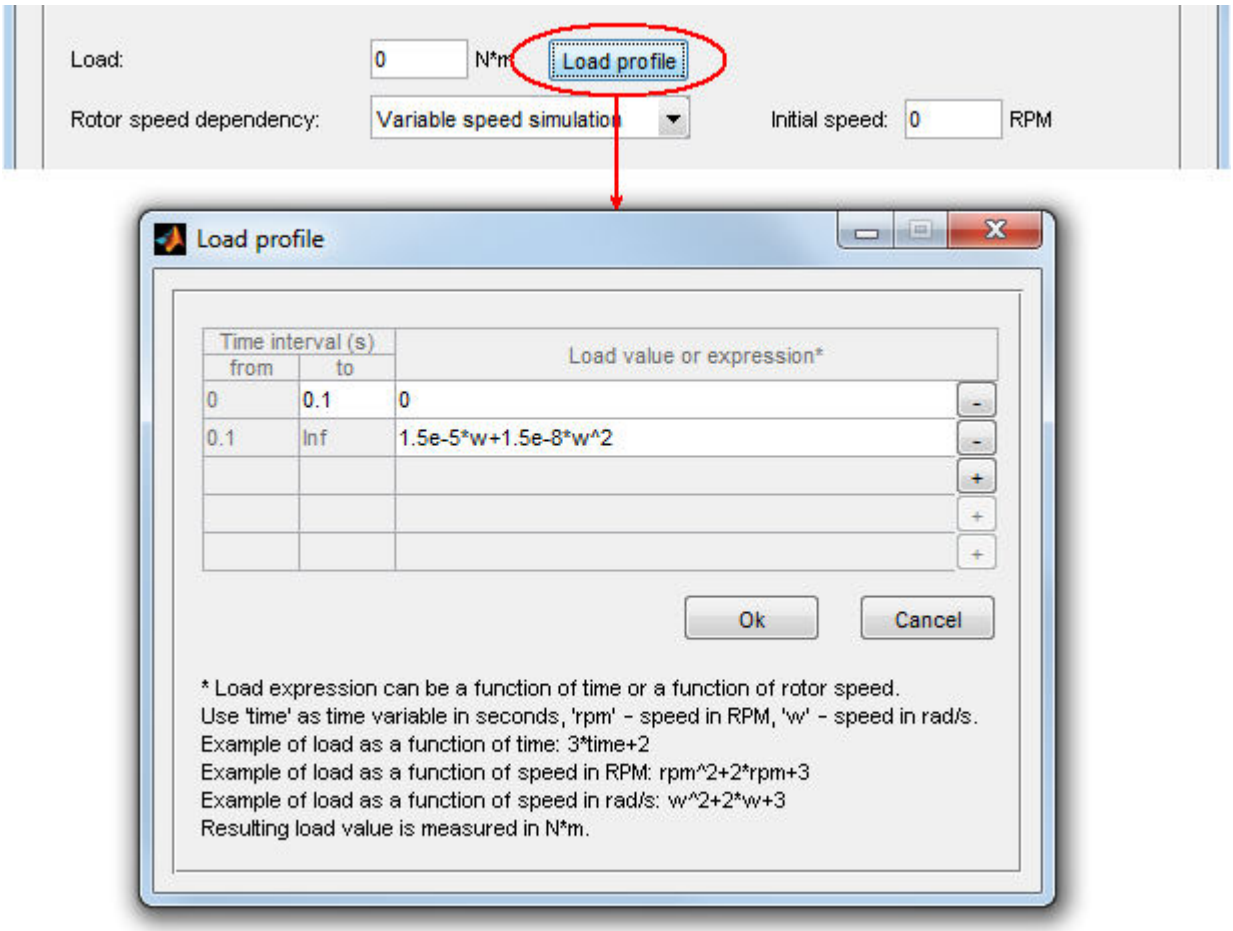


Figure 7.2. Specifying the load for the Dynamic D-Q simulation.

Button '**<-rated**' to the right of the field allows you to set up the corresponding parameter to its rated value specified in the **Rated Data** window.

If simulation with variable speed is used the load should be specified. The load can be specified either by a single value entered in the **Load** field or different load values can be specified for each time interval using the **Load profile** window as shown in Figure 7.2. Load can be a function of time or speed. Speed can be measured in rad/s or in RPM. Use +/- buttons to add or delete the time interval.

7.2. Viewing Dynamic D-Q Analysis results.

It is possible to view the Dynamic D-Q Analysis results as time-averaged quantities or waveform plots.

Use $\frac{1}{T}\int$ toolbar button to view time-averaged quantities as shown in Figure 7.3. The displayed quantities can be averaged over one, two or three electrical periods or the averaging time can be defined directly choosing ***User choice*** from the **Averaging time selection** pop-up menu.

The view of the **Plot Wizard** window when the Dynamic D-Q Analysis is chosen is shown in Figure 7.4. It is organized as a standard MATLAB plotting construction consisting of a set of `subplot` and `plot` functions. **Subplot** checkboxes allow you to control the number of axes or rectangular panes displayed within a current figure window. The corresponding axes are activated or deactivated by a

mouse click within a cell of the **Subplot** column. When the subplot is activated, the **change** button allows you to choose quantities to be plotted into the selected axes.

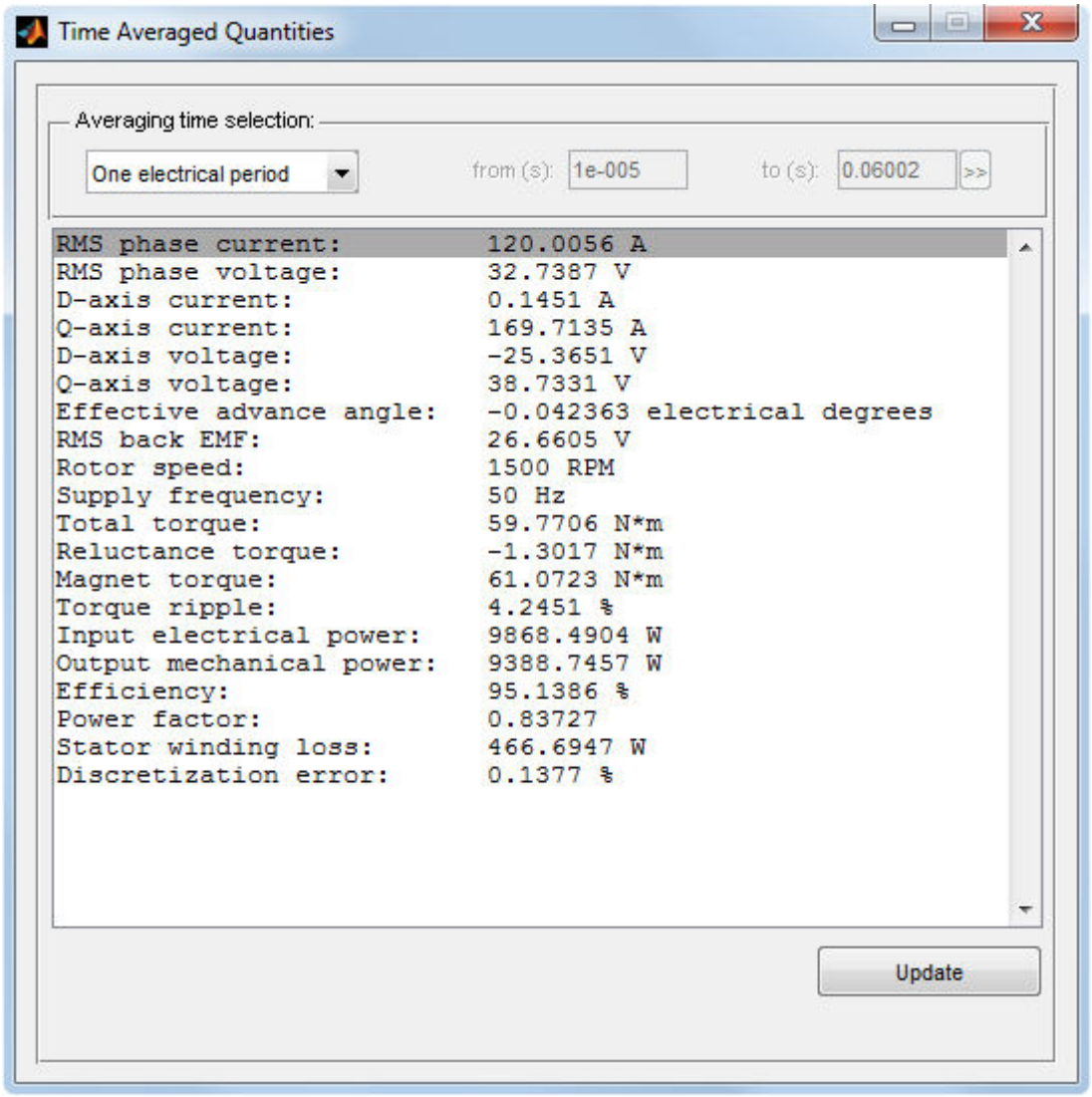


Figure 7.3. Time-averaged quantities.

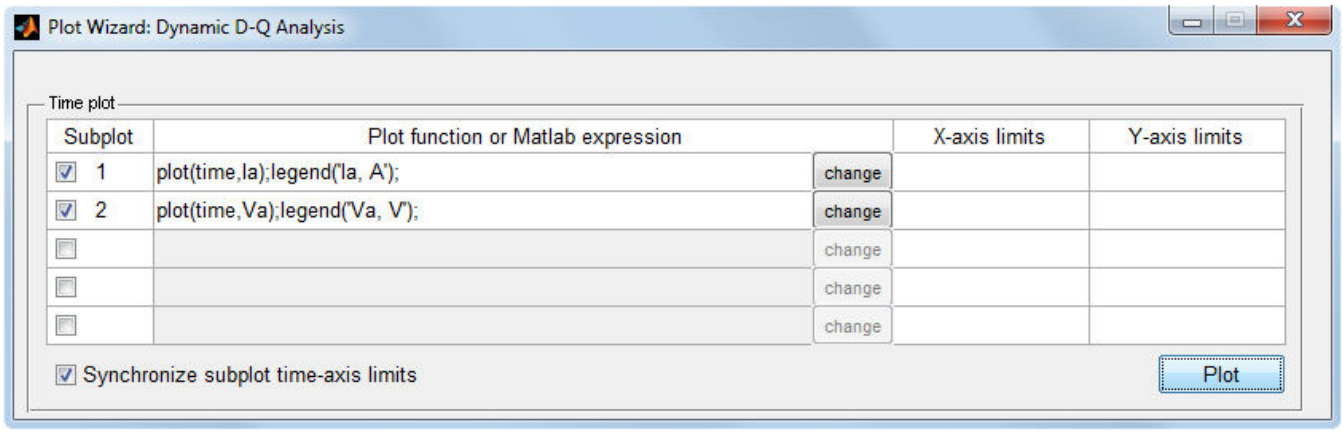


Figure 7.4. Dynamic D-Q Analysis Plot Wizard window.

Quantities to plot can be chosen from the dialog shown in Figure 7.5. Use Ctrl button to select several quantities. The list of available quantities is given below:

- Stator phase current (phase A, B, C);
- Phase current, d-axis;
- Phase current, q-axis;
- Phase voltage (phase A, B, C);
- Phase voltage, d-axis;
- Phase voltage, q-axis;
- Effective advance angle;
- Back-EMF (phase A, B, C);
- Back-EMF, d-axis;
- Back-EMF, q-axis;
- Electromagnetic torque (by flux linkage and current);
- Load torque on the motor shaft;
- Rotor speed;
- Rotor angular position;
- Input active electrical power;

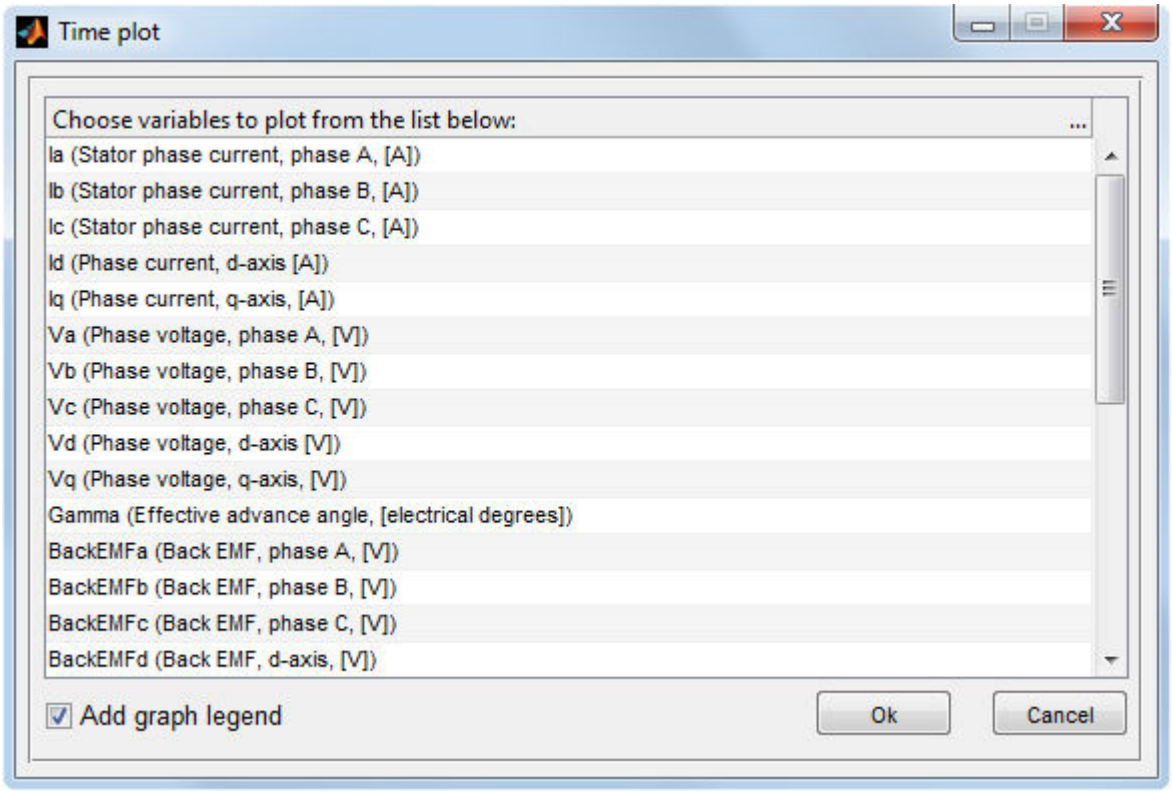


Figure 7.5. Choosing quantities to be plotted.

- Reactive electrical power;
- Mechanical power on the rotor shaft;
- Stator winding losses;
- Time derivative of the magnetic filed energy;
- Flux linkage (phase A, B, C);
- Flux linkage, d-axis;
- Flux linkage, q-axis;
- Magnet torque (by flux linkage and current);
- Reluctance torque (by flux linkage and current);
- D-axis inductance;
- Q-axis inductance;
- Cross-saturation inductance;
- D-axis magnet flux linkage;
- Q-axis cross-saturation magnet flux linkage;

By clicking the **OK** button of the dialog (Figure 7.5), the MATLAB-expression is constructed to plot the selected quantities appearing in the corresponding line as shown in Figure 7.4 for plotting phase A stator current (first line) and phase A stator voltage (second line).

When the **Plot** button is clicked, the MATLAB figure window with plots of the selected quantities will appear (see Figure 7.6).

As it is seen, the plotting expression consists of the `plot` function with selected variables used as input arguments. If the **Add graph legend** checkbox of the dialog is checked, the `legend` function is added to the plotting expression so the graph legend of the corresponding axes will be shown. All plotting expressions are editable, so you can use any MATLAB plotting options and functions to change the way the plots appear on the screen. You can also change `time` variable to any other variable you would like to use as an input argument of the `plot` function. There is also a list of additional variables which can be used within a plotting expression (see section 9.3).

X-axis limits and **Y-axis limits** fields allow you to set the x-axis and y-axis limits, respectively, to the specified values. Two limit values within a cell are divided by a space, comma ‘,’ or semicolon ‘;’. If a cell is empty, the limits of the corresponding axis will be chosen automatically. If the **Synchronize subplot time-axis limits** checkbox is checked, all subplots of the figure will have identical limits along the time-axis when you zoom or pan one of the subplots of the figure.

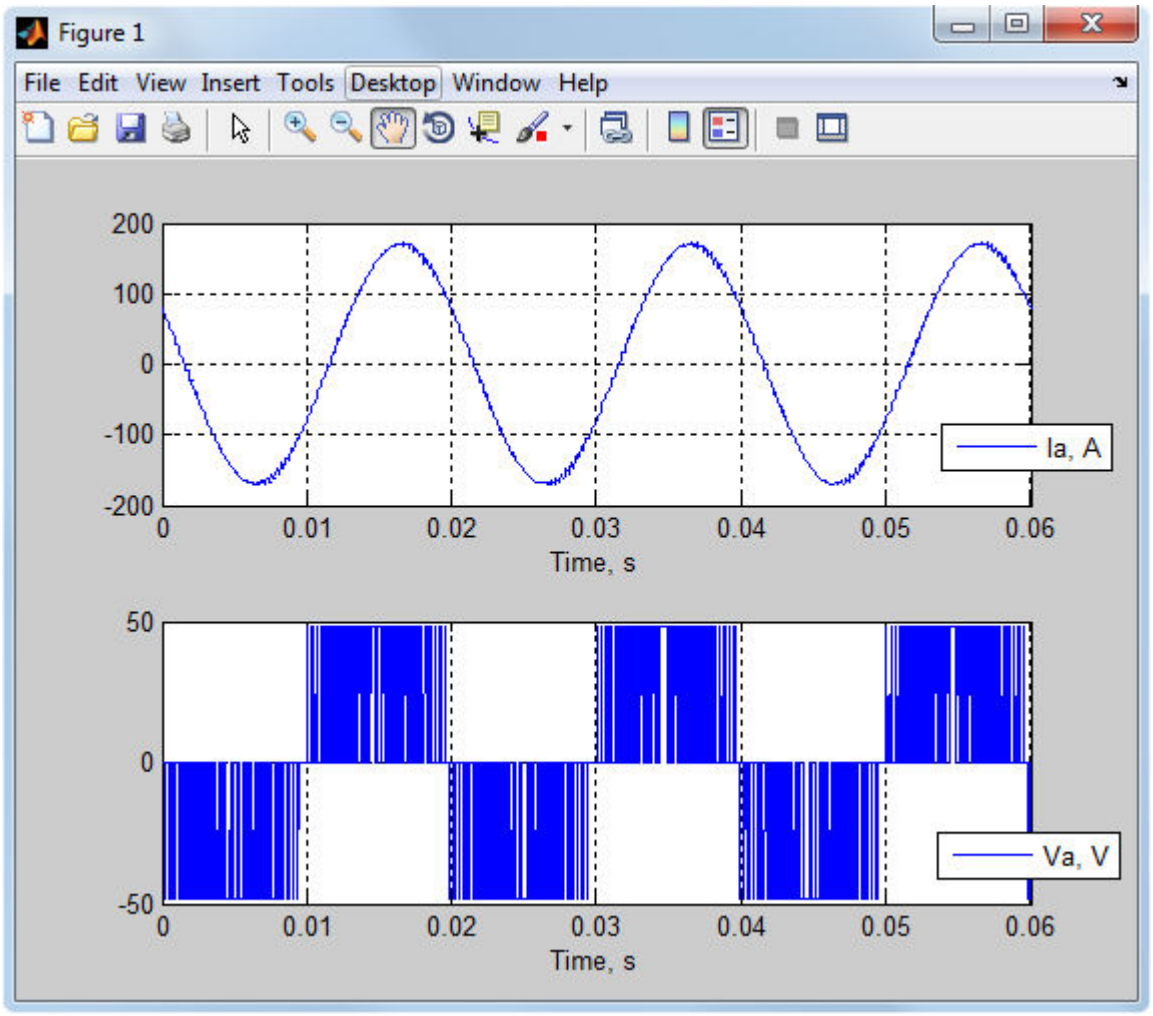


Figure 7.6. MATLAB figure window with time plots of the selected quantities.

8. DYNAMIC FINITE ELEMENT ANALYSIS

The Dynamic FE Analysis is the most powerful and most accurate of all the analysis methods taking into account rotation of the rotor, eddy currents, higher harmonics, PWM switching and etc. It simulates the machine connected to a power supply electronic circuit with the time-stepping transient finite element method. By default, the electrical circuit is a three-phase inverter as shown in Figure 10.1, but any power supply circuit can be used (see section 10). However, this type of analysis takes considerably more time comparing with previous analysis types.

8.1. Running Dynamic FE Analysis.

The view of the main window when the Dynamic FE Analysis is chosen is shown in Figure 8.1. To start the simulation, click the **Start dynamic FE simulation** button on the MotorAnalysis-PM main window toolbar. The first run of the simulation begins at zero time. Every subsequent run of the simulation resumes from the time where it was previously paused. The simulation continues, until you pause the simulation, until the simulation reaches **Simulation stop time** or until an error occurs. The first run of the simulation starts with the initialization which may take a few minutes. To pause the simulation, click the **Pause simulation** button which is to the right of the **Start dynamic FE simulation** button**.** The simulation will be paused when the calculation of the current time step is completed. It is not recommended to interrupt the simulation using Ctrl+C shortcut otherwise the simulation file may be corrupted and all data will be lost.

Solver type specifies whether the linear or nonlinear FEA is used. Using linear solver type is recommended only for testing purposes. **Convergence tolerance** specifies the accuracy of FEA solution as defined by Exp. 2.5.

Simulation settings field specifies either the default **Simulation script file** and the default **Stator electrical circuit file** are used (if ***General*** is chosen) or simulation script and stator circuit are defined by the user (if ***Advanced*** is chosen). **Simulation script file** is a MATLAB-function which is called on each simulation time step and allows you to change all general simulation settings, compute and store user defined variables, control power supply sources and electronic switches, implement user defined motor control algorithms. Refer to chapter 9 for more details on using simulation script files. **Stator electrical circuit file** is a MATLAB-function defining the coupled electrical circuit which is solved simultaneously with the magnetic field. Refer to chapter 10 for more details on using stator electrical circuits.

Be aware that the **Time step** value should be small enough comparing with the PWM sampling period to get the accurate results. Control the ***Discretization error*** in the **Time Average Quantities** window shown in Figure 8.3 to adjust the time step. It is recommended that the discretization error does not exceed 1%.

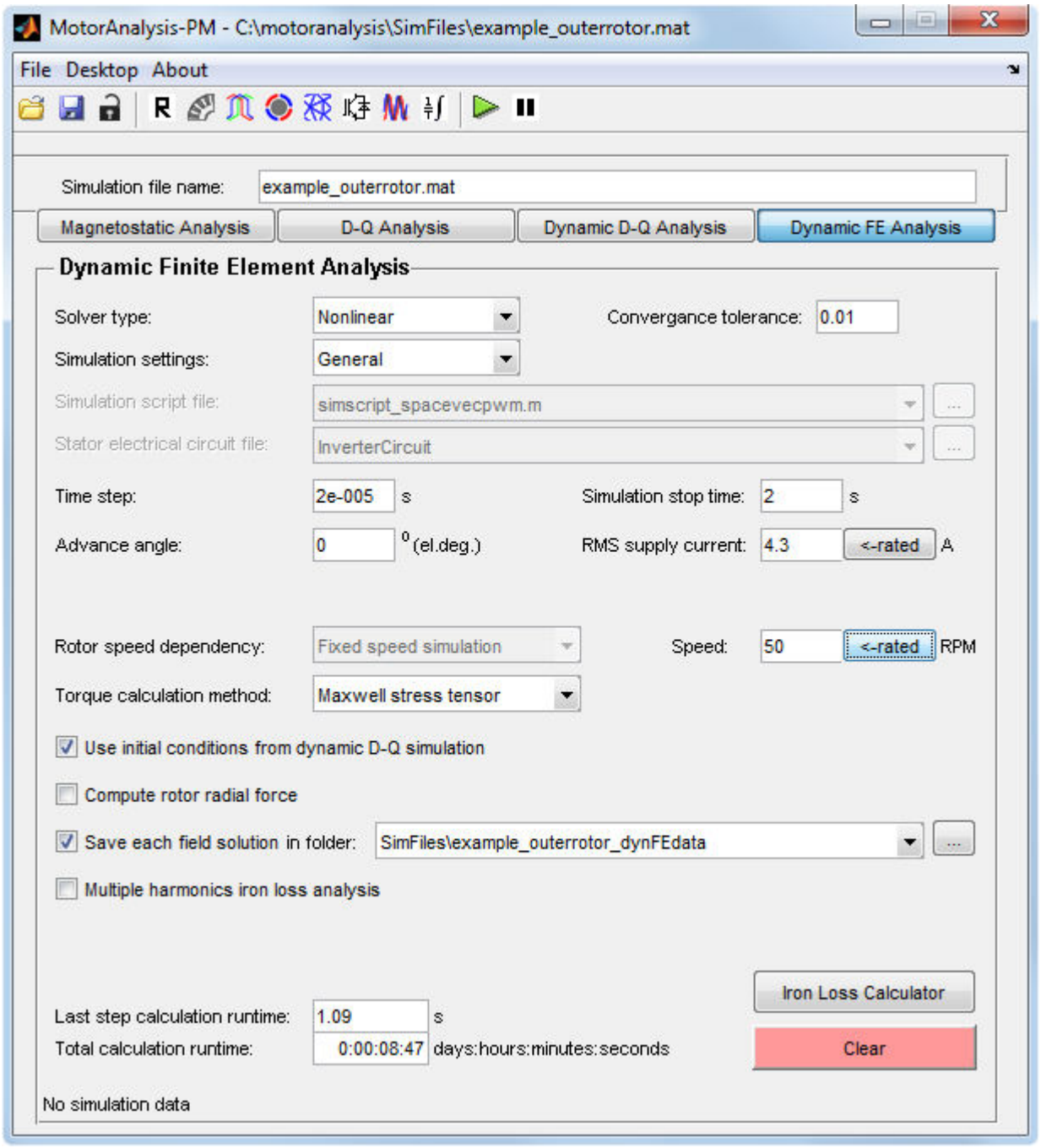


Figure 8.1. MotorAnalysis-PM main window for Dynamic FE Analysis.

The phase current of the machine depends on the **RMS supply current** field value and on the stator winding connection. Since the supply current is a line current, for star connection the phase current is equal to the supply current, for delta connection the phase current is equal to the supply current divided by $\sqrt{3}$. **Advance angle** determines an angle between the current phasor and q-axis of the rotor as shown in Figure 2.6. For the current hysteresis and space vector PWM drive types the current control algorithm is applied to adjust the inverter voltage in order to maintain d-axis and q-axis current components defined by the **RMS supply current** and **Advance angle** field values. For the space vector PWM the field oriented current control is used, for the current hysteresis PWM the hysteresis

(bang-bang) current control is used. For the six-step drive the current is not controlled – the current is determined by the DC voltage, rotor speed, switch duty cycle (120 or 180 electrical degrees) and commutation advance angle.

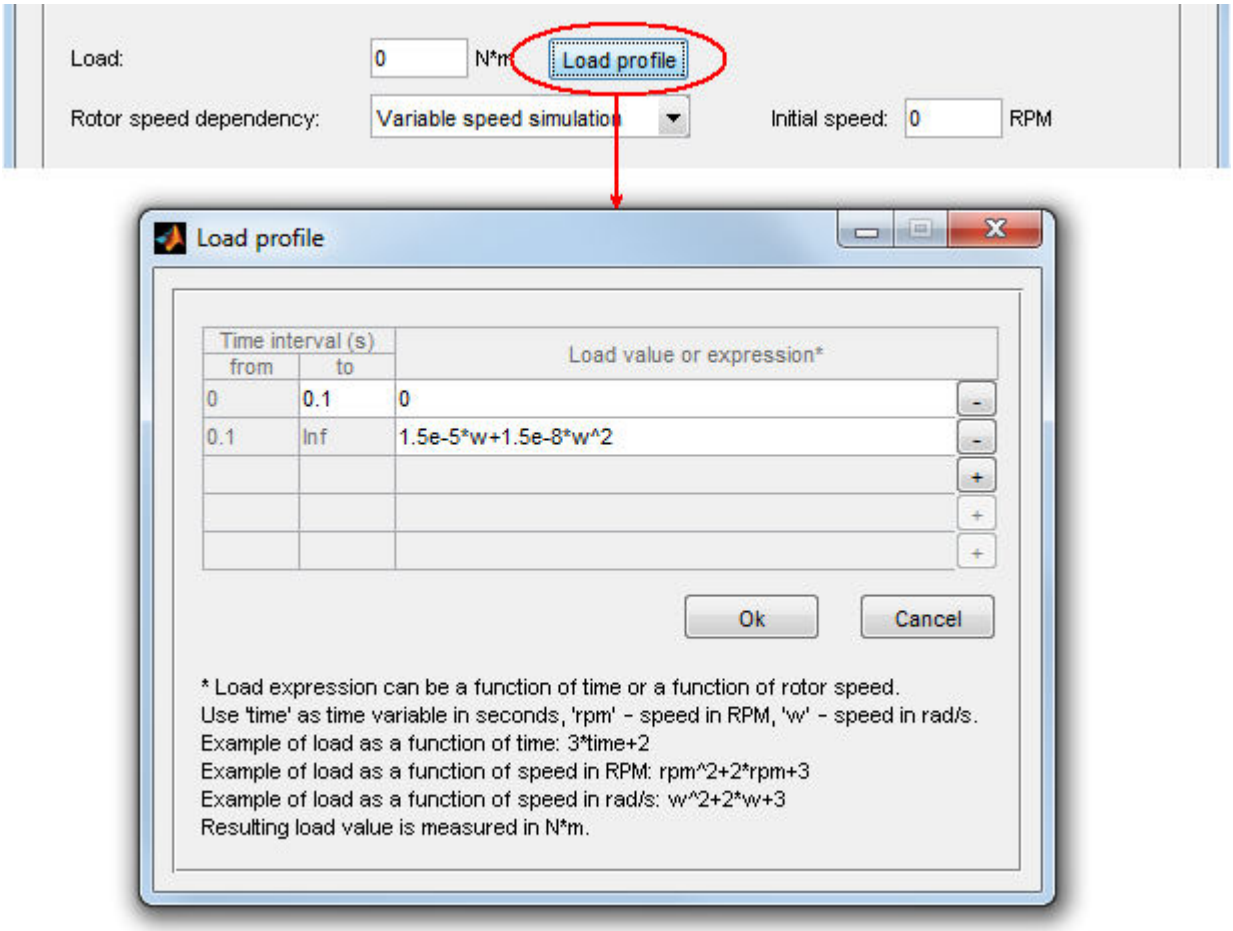


Figure 8.2. Specifying the load for the Dynamic FE simulation.

Rotor speed dependency – specifies whether the rotor speed is fixed (when ***Fixed speed simulation*** is chosen) or varies depending on the load and electromagnetic torque of the motor (when ***Variable speed simulation*** is chosen). If ***Fixed speed simulation*** is chosen you should also specify the speed in the field appearing to the right. If ***Variable speed simulation*** is chosen the initial speed should be specified. If simulation with variable speed is used the load should be specified. The load can be specified either by a single value entered in the **Load** field or different load values can be specified for each time interval using the **Load profile** window as shown in Figure 8.2. Load can be a function of time or speed. Speed can be measured in rad/s or in RPM. Use +/- buttons to add or delete the time interval. Button '**<-rated**' to the right of the field allows you to set up the corresponding parameter to its rated value specified in the **Rated Data** window.

Torque calculation method – two torque calculation methods are available: ***Maxwell stress tensor*** and ***Virtual work***. Maxwell stress tensor method is usually used. Refer to section 2.3 for more details.

If **Use initial conditions from dynamic D-Q simulation** is checked the FE simulation is started with the initial state (initial values of magnetic field, currents and voltages) computed using the dynamic D-Q model so the dynamic FE simulation enters the steady-state in less number of time steps.

Compute rotor radial force checkbox enables (if checked) to compute the radial force between the stator and rotor.

Save each field solution checkbox enables (if checked) to store additional data of each FEA solution such as magnetic vector potential distribution and permeability distribution values. These data are not saved by default because of large amount of hard drive space required. Data for each time step are saved in a separate file so the number of files saved is equal to the number of computed time steps.

Refer to section 8.3 to learn about **Multiple harmonics iron loss analysis**.

Use **Clear** button to delete all FEA simulation data in order to start the simulation from the beginning.

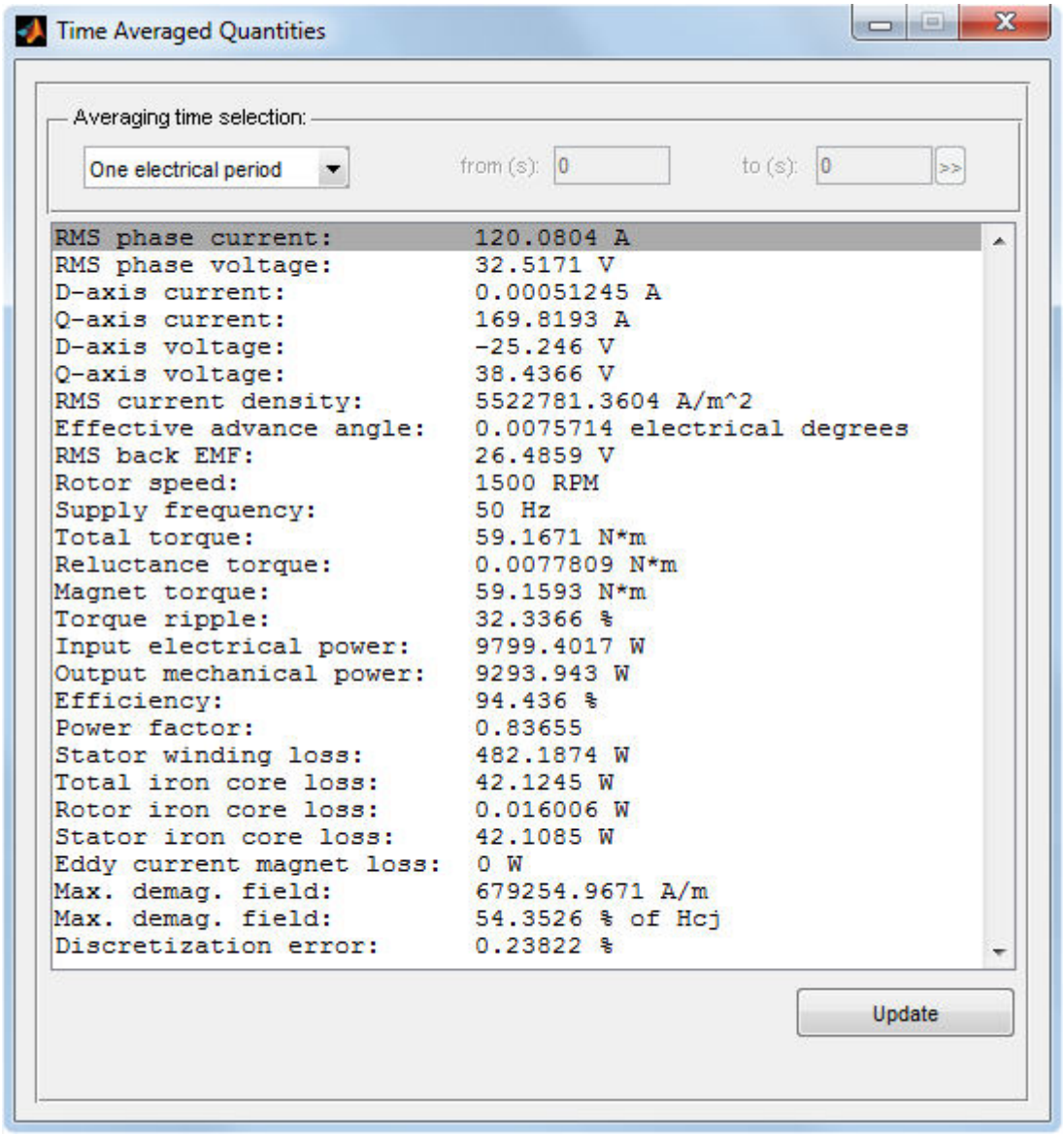


Figure 8.3. Time averaged quantities.

8.2. Viewing Dynamic FE Analysis results.

It is possible to view the Dynamic D-Q Analysis results as time-averaged quantities or plots.

8.2.1. Time-averaged quantities.

Use  toolbar button to view time-averaged quantities as shown in Figure 8.3. The displayed quantities can be averaged over one, two or three electrical periods or the averaging time can be defined directly choosing ***User choice*** from the **Averaging time selection** pop-up menu.

For more details on the interpretation of the maximum demagnetizing field result see section 2.7.

8.2.2. Plot wizard.

The view of the **Plot Wizard** window when the Dynamic D-Q Analysis is chosen is shown in Figure 8.4. Several plot types are available for the Dynamic D-Q Analysis:

- time-series data plot;
- air gap distribution plot and frequency spectrum of the air gap distribution;
- cross-section distribution plot;
- animation.

8.2.3. Time plots.

Time-series data plots are available from the **Time plot** panel of the **Plot Wizard** window. It is organized as a standard MATLAB plotting construction consisting of a set of `subplot` and `plot` functions. **Subplot** checkboxes allow you to control the number of axes or rectangular panes displayed within a current figure window. The corresponding axes are activated or deactivated by a mouse click within a cell of the **Subplot** column. When the subplot is activated, the **change** button allows you to choose quantities to be plotted into the selected axes. Quantities can be chosen from the dialog shown in Figure 8.5. Use Ctrl button to select several quantities. The list of available quantities is given below:

- Stator phase current (phase A, B, C, d-axis, q-axis);
- Phase voltage (phase A, B, C, d-axis, q-axis);
- Effective advance angle;
- Back-EMF (phase A, B, C, d-axis, q-axis);
- Input apparent electrical power;
- Consumed apparent power;
- Apparent power (real and reactive) consumed by a stator circuit;
- Mechanical power on the rotor shaft;
- Time derivative of the magnetic field energy;
- Electromagnetic torque;
- Load torque on the motor shaft;
- Rotor speed;

- Rotor angular position;
- Electromagnetic torque by Maxwell stress tensor;
- Electromagnetic torque by virtual work method;
- Electromagnetic torque by flux linkage and current;
- Magnet torque (by Maxwell stress tensor);
- Reluctance torque (by Maxwell stress tensor);
- Cogging torque (by Maxwell stress tensor);
- Flux linkage (phase A, B, C, d-axis, q-axis);
- Radial electromagnetic force between stator and rotor along x-direction;
- Radial electromagnetic force between stator and rotor along y-direction;

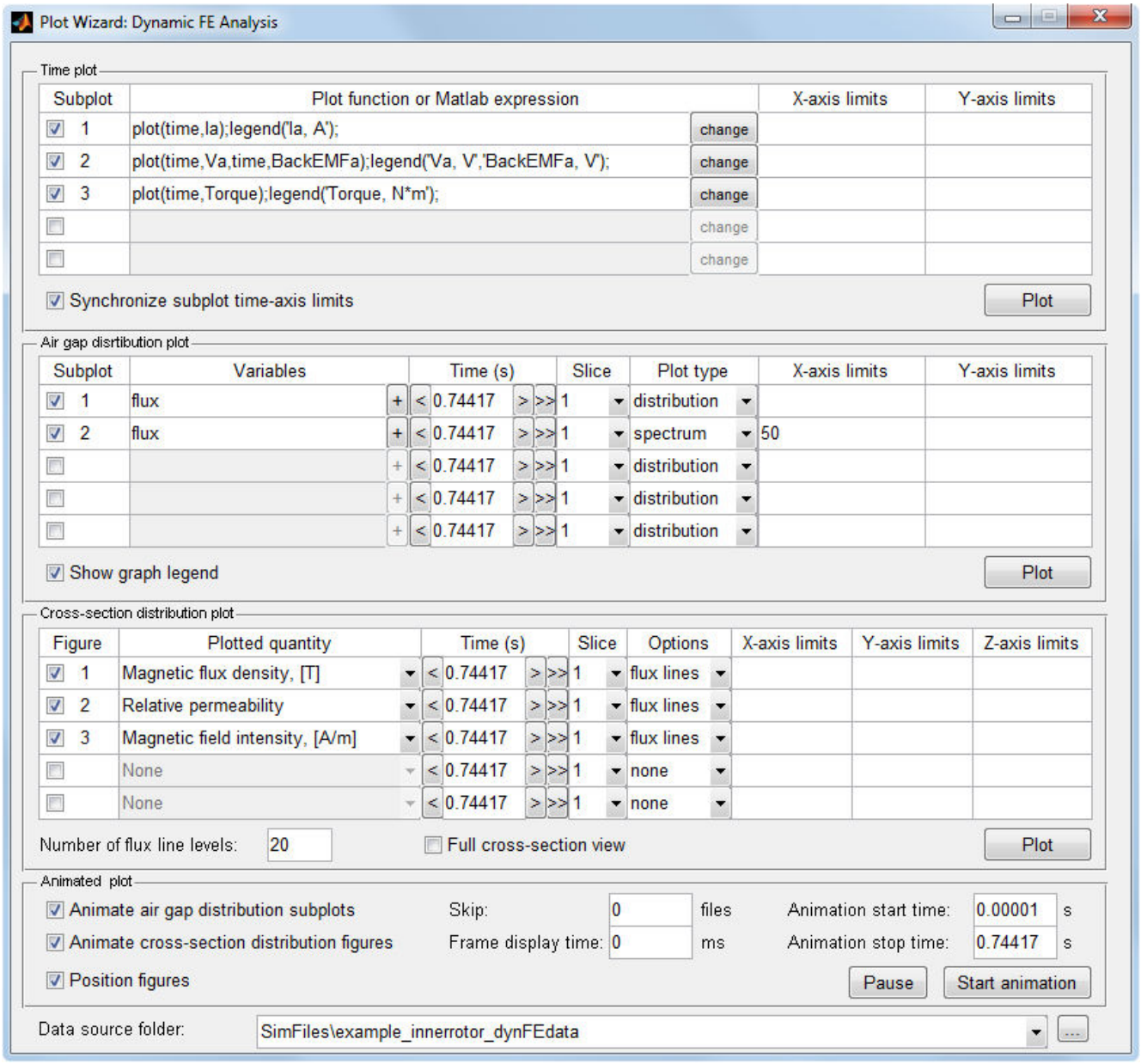


Figure 8.4. Dynamic FE Analysis Plot Wizard window.

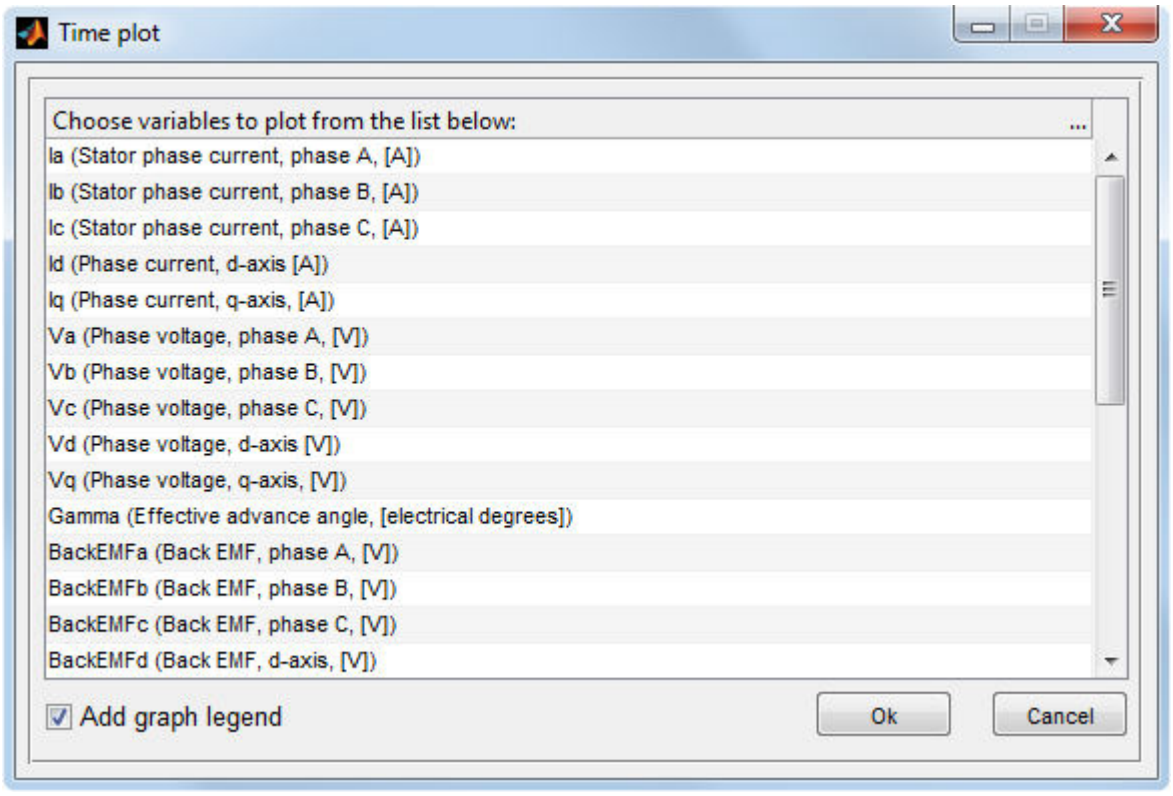


Figure 8.5. Choosing quantities to be plotted.

All user defined variables stored in the `Userdata` structure appear at the end of the list shown in Figure 8.5. For more details on saving your own variables and using the `Userdata` structure refer to section 9.4.

By clicking the **OK** button of the dialog (Figure 8.5), the MATLAB-expression is constructed to plot the selected quantities appearing in the corresponding line as shown in Figure 8.6 for plotting phase A stator current (first line), phase A voltage and back-EMF (second line) and torque (third line) for six-step drive.

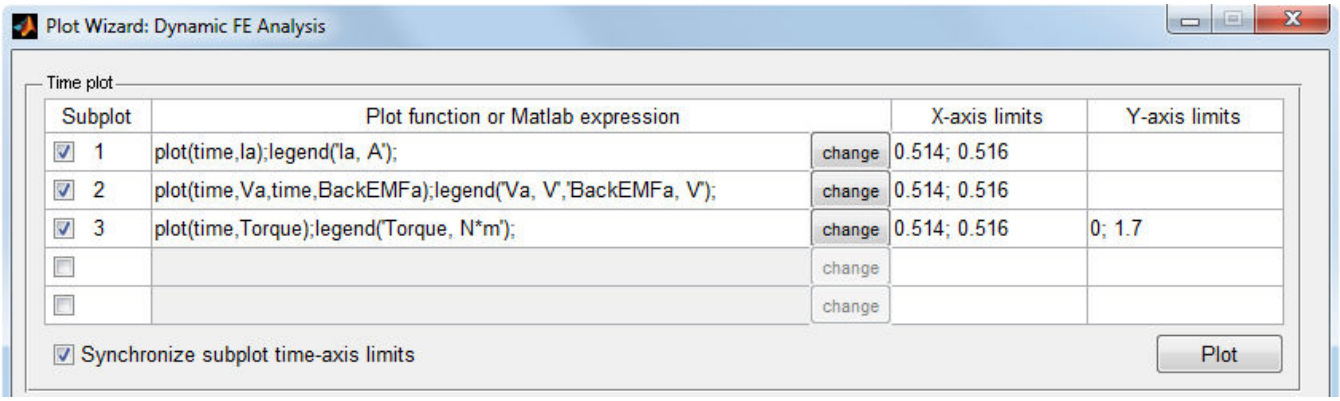


Figure 8.6. Example of using **Plot Wizard** for creating time plots.

When the **Plot** button is clicked, the MATLAB figure window with plots of the selected quantities will appear (see Figure 8.7).

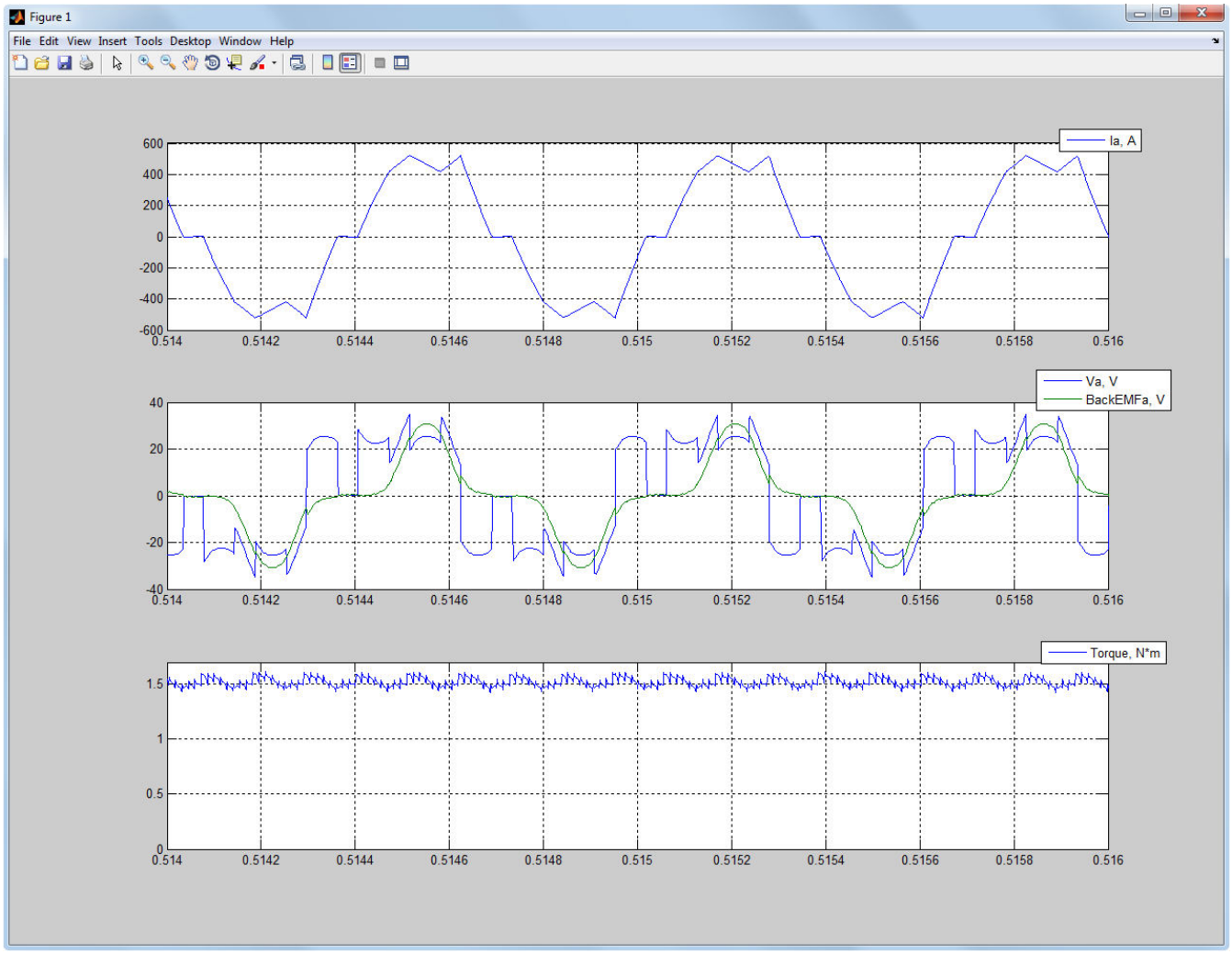


Figure 8.7. MATLAB figure window with time plots of the selected quantities.

As it is seen, the plotting expression consists of the `plot` function with selected variables used as input arguments. If the **Add graph legend** checkbox of the dialog is checked, the `legend` function is added to the plotting expression so the graph legend of the corresponding axes will be shown. All plotting expressions are editable, so you can use any MATLAB plotting options and functions to change the way the plots appear on the screen. You can also change `time` variable to any other variable you would like to use as an input argument of the `plot` function. There is also a list of additional variables which can be used within a plotting expression (see section 9.3).

X-axis limits and **Y-axis limits** fields of the **Time plot** panel allow you to set the x-axis and y-axis limits, respectively, to the specified values. Two limit values within a cell are divided by a space, comma ‘,’ or semicolon ‘;’. If a cell is empty, the limits of the corresponding axis will be chosen automatically. If the **Synchronize subplot time-axis limits** checkbox is checked, all subplots of the

figure will have identical limits along the time-axis when you zoom or pan one of the subplots of the figure.

8.2.4. Air gap distribution plots.

Air gap distribution plots allow you to view the distribution of the particular quantity over the machine's air gap as well as its frequency spectrum. It is available from the **Air gap distribution plot** panel. **Subplot** checkboxes allow you to control the number of axes or rectangular panes displayed within a current figure window. The corresponding axes are activated or deactivated by a mouse click within a cell of the **Subplot** column. When the subplot is activated, the **"+"** button allows you to choose quantities to be plotted from the dialog shown in Figure 8.8. Use Ctrl button to select several quantities.

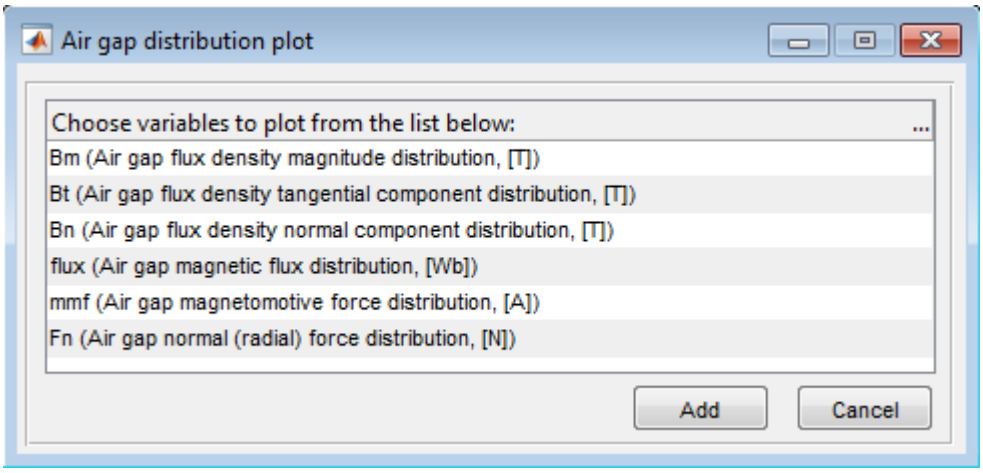


Figure 8.8. Choosing quantities to be plotted.

The following quantities are listed in the dialog of Figure 8.8:

Quantity	Comments
Bm (Air gap flux density magnitude distribution, [T])	Defined as $B_m = \sqrt{B_t^2 + B_n^2}$ where B_n and B_t – normal and tangential components of the magnetic flux density in the center of the air gap, as shown in Figure 2.4.
Bt (Air gap flux density tangential component distribution, [T])	B_n and B_t – normal and tangential components of the magnetic flux density in the center of the air gap, as shown in Figure 2.4.
Bn (Air gap flux density normal component distribution, [T])	

flux (Air gap magnetic flux distribution, [Wb])	Integration of the magnetic flux distribution over an air gap always gives zero: $\Phi_{airgap} = l \oint_{airgap} B_n \cdot d\lambda = 0$, where l – lamination length in z direction. Air gap magnetic flux distribution is calculated over inner and outer contours as shown in Figure 2.4 and the actual values are resulted as the average.
mmf (Air gap magnetomotive force distribution, [A])	Integration of the magnetomotive force distribution over an air gap always gives zero: $F_{airgap} = \frac{1}{\mu_0} \oint_{airgap} B_t \cdot d\lambda = 0$ Air gap magnetomotive force distribution is calculated over inner and outer contours as shown in Figure 2.4 and the actual values are resulted as the average.
Fn (Air gap normal (radial) force distribution, [N])	According to Maxwell stress tensor method the normal force in each point of the round path can be expressed as the following: $F_n = -\frac{B_n^2 - B_t^2}{2\mu_0} d \cdot l$, where l – lamination length in z direction, d – length of the path in the center of the air gap between two consecutive points, μ_0 – permeability of the free space, B_n and B_t – normal and tangential components of the magnetic flux density in the center of the air gap, as shown in Figure 2.4.

By clicking the **Add** button of the dialog (Figure 8.8), the selected quantities are added to the corresponding line separating them by commas as shown in Figure 8.9. Each line of the **Variables** column is editable, so you can you use MATLAB arithmetic operations to obtain desired plots. For example, the second and third lines in Figure 8.9 produce plots of the tangential air gap force density distribution and the radial air gap force density distribution, respectively, where `mu0` – variable corresponding to the permeability of free space. According to Exp. 2.6 the tangential air gap force produces the electromagnetic torque so the plot of the electromagnetic torque distribution over an air gap can be obtained.

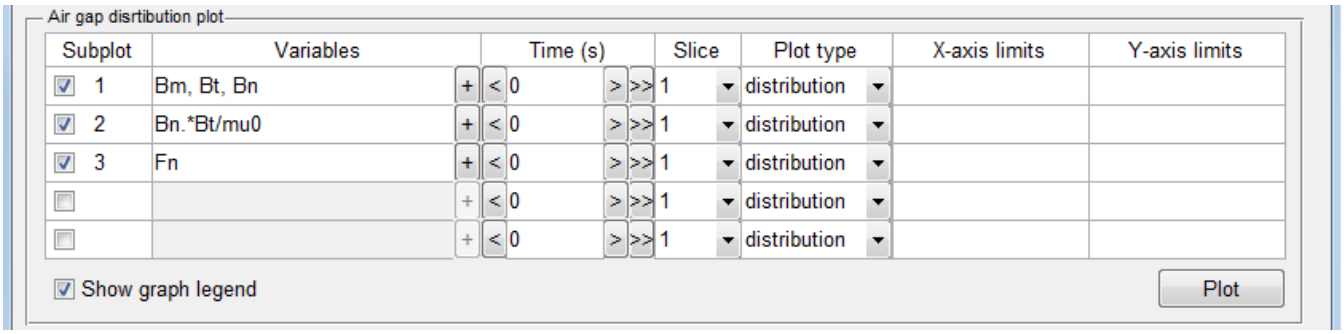


Figure 8.9. Example of using **Plot Wizard** for creating air gap plots.

Time fields of the **Air gap distribution plot** panel allow you to specify the time point which the selected quantities are plotted for. By default, last computed time point is set up. Plotting for the time points other than the last computed time point is only possible, if the file containing data for the desired time point was previously saved. These data-files are being saved when **Save each field solution** is checked in the MotorAnalysis-PM main window. So, if you are interested in viewing the air gap distribution plots for different time points make sure that the corresponding data-files are being saved. The folder used as a source of data-files is specified in the **Data source folder** field located at the bottom of the **Plot Wizard** window. You can change it by clicking the button to the right, if needed.

X-axis limits and **Y-axis limits** fields allow you to set the x-axis and y-axis limits, respectively, to the specified values in the same way as for the **Time plot** panel. **Slice** fields allow you to choose the machine's cross-section which the selected quantities are plotted for, if several slices are used (see section 2.4 for more details on multi-slice simulations).

Besides the air gap distribution, it is also possible to calculate the air gap harmonic components for the selected quantities. To obtain the frequency spectrum, select the ***spectrum*** item in the corresponding **Plot type** pop-up menu. An example of plotting the air gap distribution of the magnetic flux and its spectrum is shown in Figure 8.10. Only first 50 harmonics are shown, since the value specified in the corresponding **X-axis limits** field is 50 (if the minimum limit equals to 0, it can be omitted).

Show graph legend field allows you to choose whether the graph legend will be displayed.

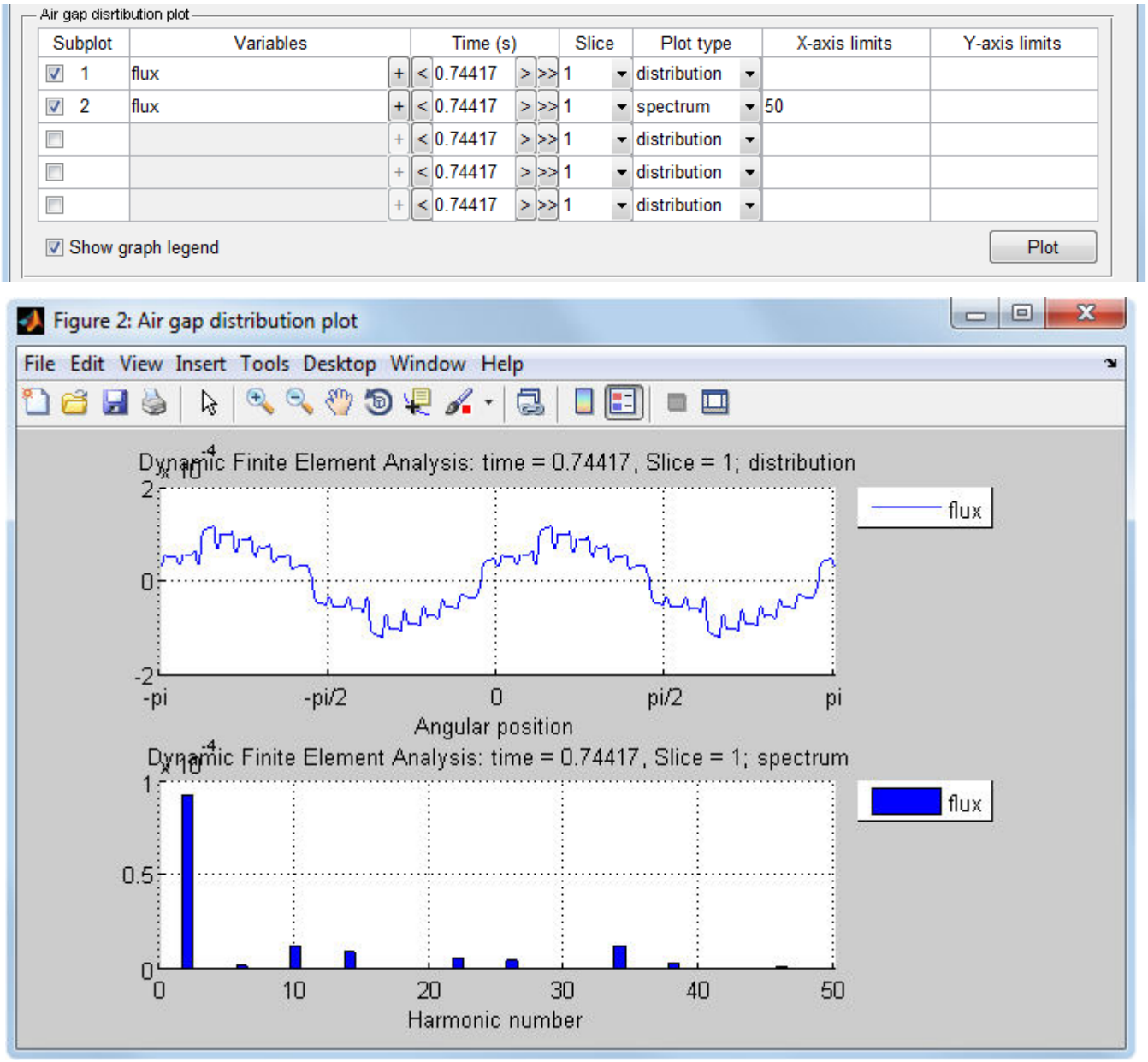


Figure 8.10. Plotting of the air gap distribution of the magnetic flux and its spectrum.

8.2.5. Cross-section distribution plots.

Cross-section distribution plots are available from the **Cross-section distribution plot** panel of the **Plot Wizard** window. As opposed to two previous plot types, the cross-section distribution for each quantity is plotted in a separated window which contains only one axes. **Figure** checkboxes allow you to control the number of windows appearing when the **Plot** button is clicked. The corresponding figure is activated or deactivated by a mouse click within a cell of the **Figure** column. When the figure is activated, the corresponding **Plotted quantity** pop-up menu allows you to choose the quantity to be plotted. If ***None*** is chosen, the machine's cross-section geometry will be plotted.

The following items are available from the **Plotted quantity** pop-up menu:

- Magnetic vector potential, [T*m];
- Magnetic flux density, [T];
- Magnetic field intensity, [A/m];
- Relative permeability;
- Stator current density, [A/m^2];
- Squared flux density, [T];
- Magnetic field energy density, [J/m^3];
- Joule loss density, [W/m^3];
- Iron loss density, [W/m^3];
- Demagnetizing field, [A/m];
- Demagnetizing field, [% of Hcj];
- Finite element mesh.

Time fields of the **Cross-section distribution plot** panel allow you to specify the time point which the selected quantity is plotted for. By default, last computed time point is set up. Plotting for the time points other than the last computed time point is only possible, if the file containing data for the desired time point was previously saved. These data-files are saved when **Save each field solution** is checked in the MotorAnalysis-PM main window. So, if you are interested in viewing the cross-section distribution plots for different time points make sure that the corresponding data-files are being saved. The folder used as a source of data-files is specified in the **Data source folder** field located at the bottom of the **Plot Wizard** window. You can change it by clicking the button to the right, if needed.

Slice fields allow you to choose the machine's cross-section which the selected quantity is plotted for, if several slices are used (see section 2.4 for more details on multi-slice simulations).

Options field allows you to show the magnetic flux lines (if ***flux lines*** is chosen) or magnetic flux arrows (if ***flux arrows*** is chosen) on the corresponding plot. Flux arrows are plotted such that the direction of the arrow indicates the direction of the flux and the size of the arrow indicates the magnitude of the flux density. **Number of flux line levels** field allows you to alter the flux lines density. If periodic/antiperiodic boundary conditions are used, by default, only part of the cross-section (as it appears in **Mesh Editor**) will be plotted. Check the **Full cross-section view** checkbox if you want to plot the whole cross-section.

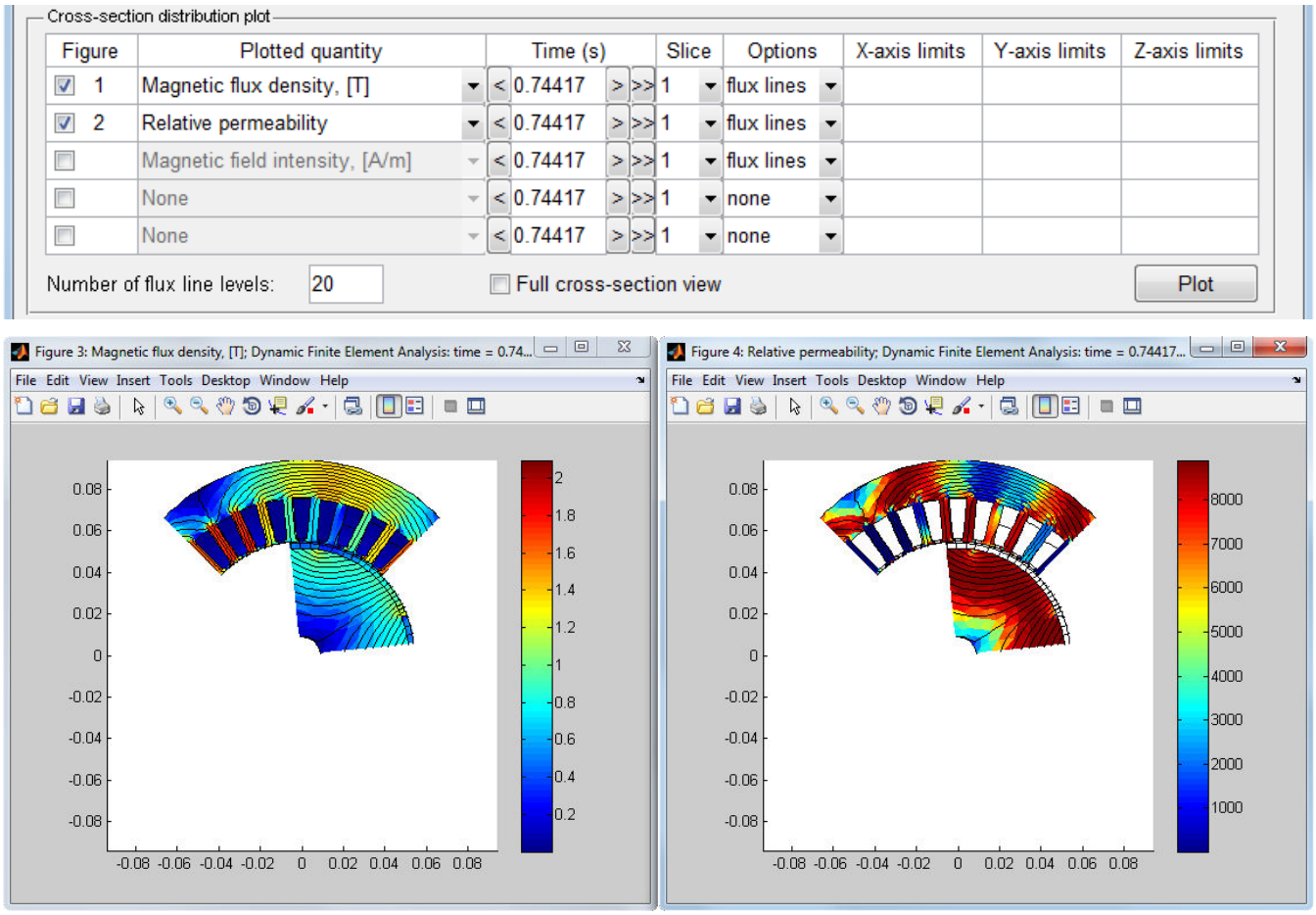


Figure 8.11. Example of using **Plot Wizard** for creating cross-section distribution plots.

X-axis limits and **Y-axis limits** fields allow you to set the x-axis and y-axis limits, respectively. If x-axis and y-axis limits are not specified, their values will be determined by the size of the machine's cross-section. **Z-axis limits** field sets the color scale limits to specified minimum and maximum values divided by a space, comma ',' or semicolon ';'.Values between z-axis limits are linearly mapped to the used color scale (colormap). Data values less or greater than specified z-axis limits are mapped to the minimum limit or to the maximum limit, respectively.

An example of plotting the cross-section distribution of the magnetic flux density and relative permeability is shown in Figure 8.11. When the **Plot** button is clicked, two windows appear. As it is seen, the **flux lines** option is chosen for the both figures, so the flux lines are additionally shown. Since the third figure is not active, the Magnetic field intensity is not plotted. Note that only part of the machine's cross-section is shown, since **Full cross-section view** is not checked.

8.2.6. Animation.

Animated plot panel is used to create animated sequences of the air gap distribution plots and/or the cross-section distribution plots. **Animate air gap distribution subplots** and **Animate cross-section distribution figures** checkboxes allow you to choose plot types to be animated. If **Animate air gap**

distribution subplots is checked, all selected subplots of the **Air gap distribution plot** panel will be animated. Similarly, if **Animate cross-section distribution figures** is checked, all selected figures of the **Cross-section distribution plot** panel will be animated. If **Position figures** control is checked, the size and location on the screen for all figure windows will be specified automatically so that the windows fit best the screen size.

Skip and **Frame display time** fields allow you to specify the way how frames change each other while the animation is in progress. **Skip** field specifies the number of data-files or frames to be skipped. So, if **Skip** field is set to 0, data for each time step will be shown on the screen, if **Skip** field is set to 1, data for each second time step will be shown, if **Skip** field is set to 2 – for each third time step and so on. **Frame display time** field specifies the time each frame is displayed on the screen. Note that the least time the frame is displayed is limited by the animation function runtime. So, if the **Frame display time** field is 0, the actual time the frame is displayed on the screen will be the least possible in this case.

Animation start time and **Animation stop time** fields set up the time interval for the animation. **Data source folder** field specifies the folder with data-files to be animated. The same folder is used for animations and for air gap distribution and cross-section distribution plots. Using animation is only possible, if the files containing data for the time interval to be animated were previously stored.

To start the animation, click the **Start animation** button. The current time point, animated quantity and other information is displayed at the top of each window involved in the animation. The animation continues until the time specified in the **Animation stop time** field is reached or until all windows involved in the animation are closed. You can also pause the animation using the **Pause** button. While the animation is in progress or paused, you can use all MATLAB figure tools, for example, changing the scale and position of the plots.

8.3. Iron Loss Calculator.

Iron Loss Calculator shown in Figure 8.12 is used for the iron loss estimation on the postprocessing stage of Dynamic FE Analysis. For more details on iron loss estimation in MotorAnalysis-PM refer to section 2.6.

Iron Loss Calculator is available from the MotorAnalysis-PM main window when the Dynamic FE Analysis is chosen. It estimates the average iron loss over the specified time interval defined by the **Time interval** panel. To use Iron Loss Calculator you should previously store simulation data for each time step of the specified time interval using the **Save each field solution** option of the main window or, alternatively, check the **Multiple harmonics iron loss analysis** checkbox before the simulation is started. Directory with the data-files is defined by the **Data source folder** field.

Maximum fft frequency of flux density defines the maximum frequency of flux density harmonics for which the iron loss is calculated. This value is limited by the time-step as 1/timestep/2 (Nyquist frequency). Make sure that the maximum fft frequency is higher the PWM sampling frequency but be

aware that too high maximum fft frequency increase the computation time. The accuracy of the iron loss estimation depends on the frequency resolution, i.e. on the time interval to be processed (the frequency resolution equals to one divided by time interval), but on the other hand the longer time interval together with high maximum fft frequency may result in less frequency resolution if the available memory is not enough to process all data-files at once.

To start calculation, click the **Start Iron Loss Calculation** button. Result will be displayed at the bottom of the Iron Loss Calculator window.

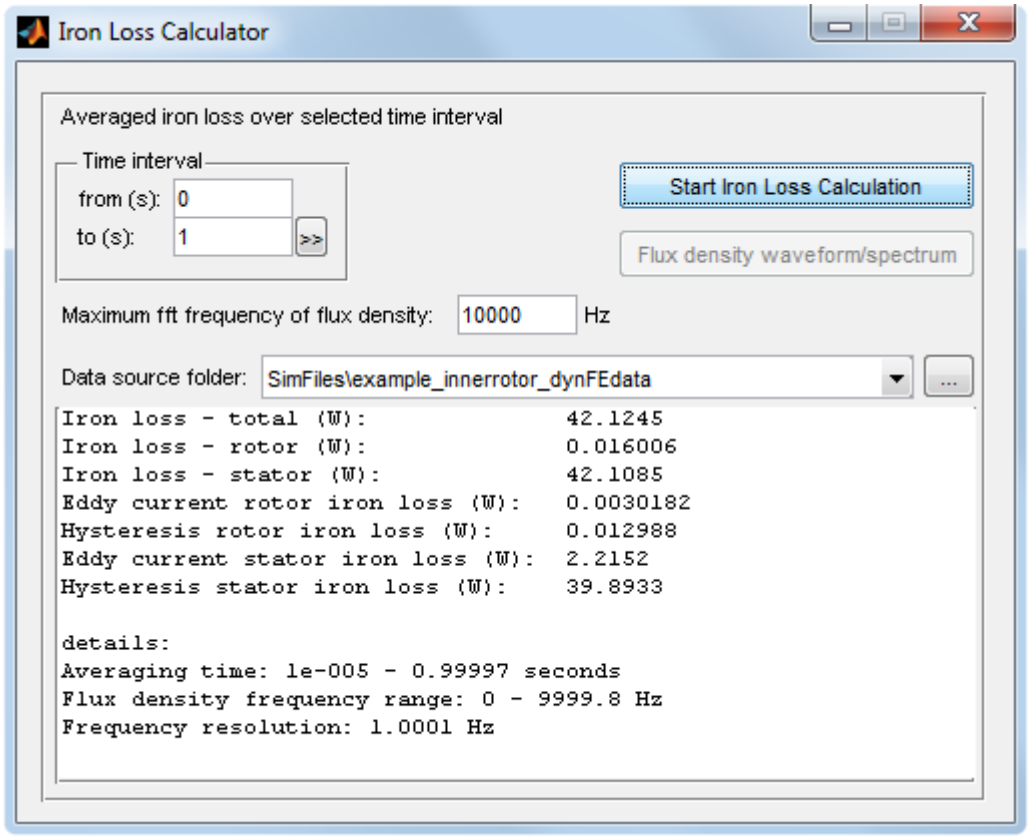


Figure 8.12. Iron Loss Calculator.

9. DYNAMIC FE ANALYSIS SIMULATION SCRIPT FUNCTIONS

A simulation script is a file with .m extension containing MATLAB-function of a specific format. While the dynamic FEA simulation is in progress this function is called on each simulation time step and allows you to automatically change all simulation settings, compute and store your own variables, control power supply sources and electronic switches, implement motor control algorithms and etc. For example, the simulation script function is used to control electronic switches (IGBT, etc.) to implement PWM switching sequence.

To understand this chapter, some familiarity with MATLAB programming is required. Refer to MATLAB help documentation for more details on MATLAB programming.

9.1. General information.

MotorAnalysis-PM offers you a variety of options to properly set up your simulation and in most cases there is no need to use simulation scripts. On the other hand, simulation scripts give you much more flexibility in using MotorAnalysis-PM. Note that simulation scripts are used only for the Dynamic FE Analysis and only with MATLAB version of MotorAnalysis-PM. Refer to section 3.2 for more details on standalone version limitations.

The chosen simulation script file is displayed in the **Simulation script file** field of the MotorAnalysis-PM main window when the Dynamic FE Analysis is chosen (see Figure 8.1). There are three simulation scripts used by default: simscript_hystpwm.m, simscript_spacevecpwm.m, simscript_sixstep.m. If the **Simulation settings** field is set to ***General*** (see Figure 8.1), the corresponding simulation script is chosen automatically depending on the drive type. If the **Simulation settings** field is set to ***Advanced***, the simulation script should by specified by the user. Click the button to the right of the **Simulation script file** field to choose your own simulation script.

If the simulation script file is used all simulation settings displayed in the MotorAnalysis-PM main window can be overlaid with those specified in the simulation script function. For example, you can specify any RMS supply current or advance angle values regardless of those specified in the **RMS supply current** and **Advance angle** fields.

9.2. Writing a simulation script function.

The definition of the simulation script function `simscript` and a minimum amount of code appearing in file simscript.m is as follows:

```
function [ShaftPosition,CircuitControl,Settings,Enforce,Userdata]...
= simscript(Outputs,Settings,Userdata,Circuit, Drive,Geometry,Mesh,...
             Windings,A,cell_p,cell_t,cell_Nu,path)
ShaftPosition = [];
CircuitControl = [];
Enforce = [];
```

The first line of the function always starts with the keyword `function`. The names of the M-file and of the function should be the same. Refer to MATLAB help documentation for more details on writing MATLAB-functions.

Be aware that the presented example of the simulation script function will not be working with stator electrical circuit file InverterCircuit.m since states of switches are not defined.

The function's output arguments:

`ShaftPosition` – rotor shaft position; 1×2 array, containing rotor shaft coordinates `[x; y]` in meters. This variable allows you to apply static or dynamic eccentricity to the rotor. If array `ShaftPosition = []` or `ShaftPosition = [0; 0]`, there is no rotor eccentricity.

`CircuitControl` – structure containing current and voltage values which are supplied by current and voltage courses as well as states of electronic switches. See chapter 10 for more details on using current and voltage sources and electronic switches.

`Settings` – structure of simulation settings, the same as input argument `Settings`.

`Enforce` – structure to control rotor speed. If `Enforce=[]`, when rotor speed is computed depending on the load and electromagnetic torque (variable speed simulation). To set the rotor speed to some fixed value use the statement `Enforce.speed=fixedspeed`, where `fixedspeed` is the rotor speed in rad/s.

`Userdata` – structure to store user data, the same as input argument `Userdata`. By default `Userdata` is empty. Refer to section 9.4 to figure out how to use the `Userdata` structure to store your own data using simulation script functions.

The function's input arguments:

`Outputs` – structure containing the dynamic FEA simulation results.

`Settings` – structure containing the dynamic FEA simulation settings.

`Userdata` – structure to store user data.

`Circuit` – structure containing stator electrical circuit parameters.

`Drive` – structure containing drive parameters.

`Geometry` – structure containing the machine's dimensions data.

`Mesh` – structure containing mesh data.

`Windings` – structure containing the machine's windings data.

`A` – array of magnetic vector potential node values for the current time step; `A[1:np]` – first slice values, `A[np+1:2*np]` – second slice values (if several slices are used; see section 2.4), `A[2*np+1:3*np]` – third slice values and etc., where `np` – number of mesh nodes in one slice.

`cell_p` – cell-array containing *x* and *y* coordinates of finite element mesh nodes for every slice, as it is described in MATLAB PDE Toolbox help documentation (see help on `initmesh` function for more details).

`cell_t` – cell-array containing finite elements data for every slice, as it is described in MATLAB PDE Toolbox help documentation (see help on `initmesh` function for more details).

`cell_Nu` – cell-array containing midpoint magnetic reluctivity values for every slice. Magnetic reluctivity is $\nu = \frac{1}{\mu_0 \mu_r}$, where μ_0 – permeability of the free space, μ_r - relative permeability.

`path` – directory with application files.

9.3. Main data structures.

`Outputs` structure fields:

Field	Size, type	Units	Description
`Outputs.CurrentTime`	1×1, double	second	Simulation current time point.
`Outputs.nPolePairs`	1×1, double		Number of pole pairs.
`Outputs.gamma0`	1×1, double	Electrical degree	Angle between a-phase axis and rotor q-axis for initial rotor position.
`Outputs.Va`	1×n, double	Volt	Instantaneous values of voltages measured at stator winding terminals; n – number of computed time steps.
`Outputs.Vb`			
`Outputs.Vc`			
`Outputs.Vd`	1×n, double	Volt	Instantaneous values of d-axis and q-axis voltage components.
`Outputs.Vq`			
`Outputs.Ia`	Npp×n, double	Ampere	Current flowing in each parallel path of each phase; number of array rows corresponds to number of parallel paths. Npp – number of parallel paths.
`Outputs.Ib`			
`Outputs.Ic`			
`Outputs.Id`	1×n, double	Ampere	Instantaneous values of d-axis and q-axis current components.
`Outputs.Iq`			
`Outputs.time`	1×n, double	second	Time points.
`Outputs.Gamma`	1×n, double	Electrical degree	Instantaneous values of advance angle.
`Outputs.BackEMFa`	1×n, double	Volt	Instantaneous values of phase back-EMF.
`Outputs.BackEMFb`			
`Outputs.BackEMFc`			
`Outputs.BackEMFd`	1×n, double	Volt	Instantaneous values of d-axis and q-axis back-EMF components.
`Outputs.BackEMFq`			
`Outputs.Psi`	nCircuitNodes×n, double	Volt	Stator electrical circuit potentials. nCircuitNodes – total number of nodes of stator electrical circuits.

`Outputs.Torque`	1×n, double	N*m	Electromagnetic torque; equals to one of the variables `Torque_maxwell` or `Torque_vwork` depending on the selected torque calculation method.
`Outputs.Load`	1×n, double	N*m	Load torque on the motor shaft.
`Outputs.Speed`	1×n, double	rad/s	Rotor angular speed.
`Outputs.Rotang`	1×n, double	rad	Rotor angular position.
`Outputs.Pinput`	1×n, double	Watt	Input apparent power delivered by all current and voltage sources.
`Outputs.Pcons`	1×n, double	Watt	Consumed apparent power.
`Outputs.Ps`	1×n, double	Watt	Apparent power (real and reactive) consumed by the stator electrical circuit.
`Outputs.Pmf`	1×n, double	Watt	Magnetic energy time derivative.
`Outputs.Pmech`	1×n, double	Watt	Mechanical power on the shaft.
`Outputs.Piron.stator`	1×1, double	Watt	Stator iron loss.
`Outputs.Piron.rotor`	1×1, double	Watt	Rotor iron loss.
`Outputs.Torque_maxwell`	1×n, double	N*m	Electromagnetic torque calculated with the Maxwell stress tensor method.
`Outputs.Torque_vwork`	1×n, double	N*m	Electromagnetic torque calculated with the Virtual work method.
`Outputs.Torque_fluxlinkage`	1×n, double	N*m	Electromagnetic torque calculated by currents and flux linkages using Exp. 2.10.
`Outputs.Torque_magnet`	1×n, double	N*m	Magnet component of electromagnetic torque.
`Outputs.Torque_reluctance`	1×n, double	N*m	Reluctance component of electromagnetic torque.
`Outputs.Fluxlinkage_a`	1×n, double	Weber	Instantaneous values of flux linkages.
`Outputs.Fluxlinkage_b`			
`Outputs.Fluxlinkage_c`			
`Outputs.Fluxlinkage_d`	1×n, double	Weber	Instantaneous values of d-axis and q-axis flux linkage components.
`Outputs.Fluxlinkage_q`			
`Outputs.Fx`	1×n, double	N	Radial electromagnetic force between stator and rotor along *x*-direction.

`Outputs.Fy`	1×n, double	N	Radial electromagnetic force between stator and rotor along *y*-direction.

`Settings` structure fields:

Field	Value	Description
`Settings.DF_script`	Directory and/or file name	Corresponds to the **Simulation script file** field of the main window.
`Settings.DF_solvertype`	`'Nonlinear'`	Corresponds to the **Solver type** field of the main window.
	`'Linear'`	
`Settings.DF_tol`	Number > 0	Corresponds to the **Convergence tolerance** field of the main window.
`Settings.DF_statorcircuit`	Function name	Name of the function specifying the stator electrical circuit; corresponds to the **Stator electrical circuit file** field of the main window.
`Settings.DF_timestep`	Number > 0	Simulation time step; corresponds to the **Time step** field of the main window.
`Settings.DF_stoptime`	Number > 0	Corresponds to the **Simulation stop time** field of the main window.
`Settings.DF_Isrms`	Number > = 0	RMS supply current to be supplied by inverter; corresponds to the **RMS supply current** field of the main window.
`Settings.DF_Gamma`	Number > = 0	Corresponds to the **Advance angle** field of the main window.
`Settings.DF_motorload`	Number > = 0	Load torque on the motor shaft, corresponds to the **Load** field of the main window.
`Settings.DF_InitSpeed`	Number	Initial speed of the rotor; corresponds to the **Initial Speed** field of the main window when variable speed simulation is chosen.
`Settings.DF_saveeachsolution`	`0`	Corresponds to the **Save each filed solution** checkbox value of the main window.
	`1`	
`Settings.DF_saveeachsolutionfolder`	Directory	Folder to store data-files when **Save each filed solution** is checked.

Settings. DF_torquemethod	'Maxwell stress tensor'	Electromagnetic torque calculation method; corresponds to the **Torque calculation method** field of the main window.
	'Virtual work'	
Settings.DF_force	0	Enables calculation of the radial force acting between stator and rotor; corresponds to the **Compute rotor radial force** field of the main window.
	1	

Geometry structure fields:

Field	Value	Description
Geometry.D1s	Number > = 0	Machine's cross section dimensions according to section 4.1 of this manual.
Geometry.D2s		
Geometry.lag		
Geometry.Ods		
Geometry.Ows		
Geometry.Tas		
Geometry.Sds		
Geometry.Rcs		
Geometry.Ws		
Geometry.Dch		
Geometry.Rch		
Geometry.Tach		
Geometry.Rcs_ag		
Geometry.motortype	'Inner rotor'	Corresponds to the **Machine type** field of **Geometry Editor**.
	'Outer rotor'	
Geometry.statorslottype	'Parallel tooth'	Corresponds to the **Slot type** field of **Geometry Editor**.
	'Parallel slot'	
Geometry.slotlayertype	'Single layer'	Determined either single layer or double layer winding is used; corresponds to the **Number of winding layers** field of **Geometry Editor**.
	'Double layer'	

`Geometry.Ns`	Number > 0	Corresponds to the **Number of slots** field of **Geometry Editor**.
`Geometry.layerpos`	`Upper/Lower'`	Determined either winding layers are oriented horizontally or vertically inside the slot; corresponds to the **Winding layers position** field of **Geometry Editor**.
	`'Left/right'`	
`Geometry. statorslotcornertype`	`'General'`	Stator slot bottom shape; corresponds to the stator **Bottom corner type** field of **Geometry Editor**.
	`'Round'`	
`Geometry.dxfstator`	`0`	Determines either the stator geometry is imported from DXF-file or defined by stator dimensions; corresponds to the **Import stator from DXF-file** checkbox value.
	`1`	
`Geometry.l`	Number > 0	Machine's axial dimensions according to section 4.1 of this manual.
`Geometry.rotorskew`	Number > 0	
`Geometry.statorskew`	Number > 0	

`Windings` structure fields:

Field	Value	Description
`Windings.Npp`	Number > 0	Number of parallel paths of the stator winding per phase; corresponds to the **Number of parallel paths** field of **Winding Editor**.
`Windings.Lsew`	Number > = 0	Leakage inductance of the stator winding end-turns per phase; corresponds to the **End winding inductance** field of **Winding Editor**.
`Windings.Rs`	Number > = 0	Active resistance of the stator winding per phase at 20^0C; corresponds to the **Winding phase resistance** field of **Winding Editor**.

`Windings.layout`	Structure with three cell arrays	Stator winding layout; corresponds to the **Winding layout** tables of **Winding Editor**.
`Windings.W`	Number > 0	Total number of conductors in one stator slot; corresponds to the **Number of conductors per slot** field of **Winding Editor**.
`Windings.nstrands`	Number > 0	Corresponds to the **Number of strands in conductor** field of **Winding Editor**.
`Windings.fillfactor`	0 < Number <= 1	Stator slot (or coil) fill factor; corresponds to the **Slot fill factor** field of **Winding Editor**.
`Windings. wiresizemethod`	`'Wire diameter'`	Corresponds to the **Wire size method** field of **Winding Editor**.
	`'AWG'`	
	`'SWG'`	
	`'Slot fill factor'`	
`Windings. wiresize`	Wire size number	Corresponds to the **Wire size** field of **Winding Editor**.
`Windings.wirediameter`	Number > 0	Corresponds to the **Wire diameter** field of **Winding Editor**.
`Windings.ks`	Number > 0	Stator winding material conductivity.
`Windings.statorcircuit`	`'StarConnection'`	Determines the stator winding connection; corresponds to the **Stator winding connection** field of **Winding Editor**.
	`'DeltaConnection'`	
`Windings.Hsew`	Number > 0	Corresponds to the **End winding axial overhang** field of **Winding Editor**.
`Windings.layoutmethod`	`'Automatic'`	Stator winding layout input method; corresponds to the **Layout input method** field of **Winding Editor**.
	`'Manual'`	

`Windings.nPolePairs`	Number > 0	Corresponds to the **Number of pole pairs** field of **Winding Editor**.
`Windings.coilspan`	Number > 0	Corresponds to the **Coil span in slot pitches** field of **Winding Editor**.
`Windings.windingtype`	`'Lap'`	Corresponds to the **Winding type** field of **Winding Editor**.
	`'Concentric'`	
`Windings.Lsew_inputmethod`	`'Automatic'`	Corresponds to the **End winding inductance input method** field of **Winding Editor**.
	`'Manual'`	
`Windings.Rs_inputmethod`	`'Automatic'`	Corresponds to the **Winding phase resistance input method** field of **Winding Editor**.
	`'Manual'`	

`Drive` structure fields:

Field	Value	Description
`Drive.Vdc`	Number >= 0	Corresponds to the **DC supply voltage (Vdc)** field of **Drive Settings**.
`Drive.DriveType`	`'Current hysteresis PWM'`	Corresponds to the **Drive type** field of **Drive Settings**.
	`'Space vector PWM'`	
	`'Six-step'`	
`Drive.Is_hyst_perc`	Number > 0	Current hysteresis in percentage of RMS current; corresponds to the **Current hysteresis** field of **Drive Settings**.
`Drive.fspwm`	Number > 0	Corresponds to the **PWM sampling frequency** field of **Drive Settings**.
`Drive.SwitchDutyCycle_sixstep`	`'120'`	Corresponds to the **Switch duty cycle** field of **Drive Settings**.
	`'180'`	

`Drive.CommutationAdvanceAngle_sixstep`	Number > 0	Corresponds to the **Commutation advance angle** field of **Drive Settings**.
`Drive.SixStepOptions`	`'General'`	Corresponds to the **Options** field of **Drive Settings**.
	`'Six-step with limited maximum current'`	
	`'Six-step with variable DC voltage'`	
`Drive.Is_max_sixstep`	Number > 0	Corresponds to the **Motor phase current limit** field of **Drive Settings**.
`Drive.Is_hyst_perc_sixstep`	Number > 0	Phase current hysteresis in percentage of current limit; corresponds to the **Phase current hysteresis** field of **Drive Settings**.
`Drive.Vdc_perc_sixstep`	0 <= Number <= 100	DC voltage usage percentage in % of Vdc; corresponds to the **Vdc usage percentage** field of **Drive Settings**.

`Mesh` structure fields:

Field	Value	Description
`Mesh.nAirGapLayers`	3, 5, 7, 9	Number of mesh layers in the air gap; corresponds to the **Number of layers in air gap** field of **Mesh Editor**.
`Mesh.Hgrad`	1 < number < 2	Corresponds to the **Mesh growth rate** field of **Mesh Editor**.
`Mesh.agmeshqlt`	`'High'`	Corresponds to the **Air gap mesh quality** field of **Mesh Editor**.
	`'Medium'`	
	`'Low'`	
`Mesh.nSlices`	Number > 0	Corresponds to the **Number of slices** field of **Mesh Editor**.
`Mesh.perbndcnd`	`'None'`	Periodic/antiperiodic boundary conditions; corresponds to the **Boundary conditions** field of **Mesh Editor**.
	`'Periodic'`	
	`'Antiperiodic'`	

Mesh.nPolePairs	Number > 0	Corresponds to the **Periodicity factor** field of **Mesh Editor**.
Mesh.p	2×np, double	Initial mesh data; corresponding to p, t and e arrays used in MATLAB PDE Toolbox as mesh data.
Mesh.t	4×nt, double	
Mesh.e	7×ne, double	
Mesh.b	1×np, double	Array of boundary conditions.
Mesh.Rotate	Structure of double arrays	Variable for internal use.

9.4. Simple example of simulation script function.

This example shows how to write a simple simulation script function which changes simulation time step and stores some user defined variables in the Userdata structure. The source code of the function presented below can be found in file simscript_example.m.

```
function [ShaftPosition, CircuitControl, Settings, Enforce, Userdata] = ...
simscript_example(Outputs,Settings,Userdata,Circuit,Drive,Geometry,Mesh,...
                  Windings, A,cell_p,cell_t,cell_Nu,path)
% Example of simulation script function (should be used with circuit function
InvertedCircuit.m)

% call default simulation script function for space vector PWM
[ShaftPosition, CircuitControl, Settings, Enforce, Userdata] = ...
simscript_spacevecpwm(Outputs,Settings,Userdata,Circuit,Drive,Geometry,Mesh, ...
Windings,A,cell_p,cell_t,cell_Nu,path);

% Current simulation time
if isempty(Outputs.time)
    CurrentTime = 0;
else
    CurrentTime = Outputs.time(end);
end
% Change time step if simulation time reaches 0.1 sec
if CurrentTime>0.1
    Settings.DF_timestep=10^-6;
else
    Settings.DF_timestep=10^-5;
end
```

At the beginning of the code the default simulation script function for the space vector PWM simscript_spacevecpwm is called so there is no need to worry about assigning ShaftPosition, CircuitControl and Enforce structures since they are assigned by simscript_spacevecpwm.

In this example the simulation time step is changed when the simulation reaches 0.1 sec. You can define your own condition to change any simulation parameter in the Settings structure.

To store user defined variables (switching functions for this example) the following piece of code is used:

```
% Save switching function for each phase in Userdata structure
if ~isfield(Userdata,'Switch_a')
    % Create fields 'Switch_a', 'Switch_b', 'Switch_c' in Userdata when
    % simscript_example is called for the first time
    Userdata.Switch_a=[];
    Userdata.Switch_b=[];
    Userdata.Switch_c=[];
else
    % CircuitControl.switch_x1 - upper switch state
    if CircuitControl.switch_a1==1
        sA=1;
    else
        sA=-1;
    end
    if CircuitControl.switch_b1==1
        sB=1;
    else
        sB=-1;
    end
    if CircuitControl.switch_c1==1
        sC=1;
    else
        sC=-1;
    end
    % Collect switching function values for each time step
    Userdata.Switch_a=[Userdata.Switch_a sA];
    Userdata.Switch_b=[Userdata.Switch_b sB];
    Userdata.Switch_c=[Userdata.Switch_c sC];
end
```

When the function is called for the first time, fields `Switch_a`, `Switch_b` and `Switch_c` are created in the `Userdata` structure. The switching function values are stored in the corresponding fields of the `Userdata` structure on each subsequent call of the function. Variable saved in the `Userdata` structure can be plotted as a function of time using **Time plot** panel of **Plot Wizard** as discussed in section 8.2.3. Be aware that in order to plot the variable, the length of the variable should be the same as number of time steps, i.e. the variable should be an array and its elements should be saved on each time step as shown in the example above.

Fields of the `CircuitControl` structure corresponding to states of electronic switches used in default simulation scripts are discussed in section 10.3.

10. USING ELECTRICAL CIRCUITS

MotorAnalysis-PM allows you to connect the stator winding to the electrical circuit consisting of the following components:

- resistors;
- capacitors;
- inductances;
- ideal diodes;
- electronic switches;
- voltage sources;
- current sources.

There is no graphical user interface for editing electrical circuits in this version of MotorAnalysis-PM. Special functions are used instead. User defined electrical circuits can be used only for the Dynamic FE Analysis and only with MATLAB version of MotorAnalysis-PM. Refer to section 3.2 for more details on standalone version limitations.

The electrical circuit is specified through the function which retrieves the electrical circuit object. The name of the function appears in the **Stator electrical circuit file** pop-up menu of the main window when the Dynamic FE Analysis is chosen. Clicking the button to the right allows you to specify your own electrical circuit choosing corresponding M-file from the list of files. Function used for the default electrical circuits of the three phase inverter (shown in Figure 10.1) is implemented in file InverterCircuit.m. You can use this file as an example to write your own electrical circuit functions.

To understand this chapter some familiarity with MATLAB programming is required. Refer to MATLAB help documentation for more details on MATLAB programming.

10.1. Writing an electrical circuit function.

The definition of the electrical circuit function `InvertorCircuit` appears in file InvertorCircuit.m as follows:

```
function Schematic=InvertorCircuit(p,t,Subdomains,Circuit,l,nper,~,~)
```

The function retrieves one output argument `Schematic` which contains the electrical circuit description.

The input arguments of the function:

`p` – variable taken from `Mesh.p`.

`t` – variable taken from `Mesh.t`,

Refer to section 9.3 with the `Mesh` structure description for more details on `Mesh.p` and `Mesh.t`.

`Subdomains` – array of subdomains.

`Circuit` – structure containing stator electrical circuit parameters.

`l` – lamination length; this variable is taken from the **Lamination length** field of the **Geometry Editor** window.

`nper` – number of periodicities to define periodic/antiperiodic boundary conditions.

The following fields of the `Circuit` structure can be used:

`Circuit.Npp` – number of parallel paths of the stator winding per phase; this variable is taken from the **Number of parallel paths** field of the **Winding Editor** window.

`Circuit.Rs` – active resistance of the stator winding per phase for the specified stator winding conductors temperature.

`Circuit.Lsew` – leakage inductance of the stator winding end-turns per phase; this variable is taken from the **End winding inductance** field of the **Winding Editor** window.

`Circuit.layout` – stator winding layout.

Circuit construction procedure always begins with the following function:

```
Schematic = CircuitCreateSchematic();
```

This function does not have input arguments and retrieves the empty circuit object `Schematic`.

The procedure of circuit construction consists of building circuit branches and adding branches to the circuit object.

10.1.1. Electrical circuit branches.

Building of an electrical circuit branch begins with the following function:

```
Branch = CircuitCreateBranch(node1,node2);
```

All circuit nodes should be previously numbered beginning with zero node. The function receives start and end node numbers of the circuit branch, respectively, and retrieves an empty branch object `Branch`.

When all circuit components are added to the branch object (see section 10.1.2), the branch should be added to the circuit object using the following function:

```
Schematic = CircuitAddBranch(Schematic,Branch);
```

The function receives the circuit and branch objects and retrieves the circuit objects with the new branch added to the circuit.

10.1.2. Adding circuit components.

10.1.2.1. To add a resistor to the branch, the following function is used:

```
Branch = CircuitAddR(Branch,name,value);
```

`Branch` – variable corresponding to the branch which the resistor is added to, `name` – unique circuit component name, `value` – resistance value in Ohm.
Example of adding resistor R1 of 1 kOhm to the branch defined by variable `Branch`:

```
Branch = CircuitAddR(Branch,'R1',1000);
```

10.1.2.2. To add an inductance to the branch, the following function is used:

```
Branch = CircuitAddL(Branch,name,value);
```

`Branch` – variable corresponding to the branch which the inductance is added to, `name` – unique circuit component name, `value` – inductance value in Henries.
Example of adding inductance L1 of 100 mH to the branch defined by variable `Branch`:

```
Branch = CircuitAddL(Branch,'L1',0.1);
```

10.1.2.3. To add a capacitor to the branch, one of the following sentences can be used:

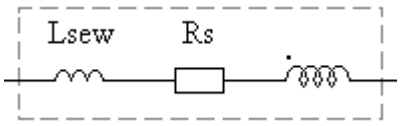
```
Branch = CircuitAddC(Branch,name,value);
```

```
Branch = CircuitAddC(Branch,name,value,Uinit)
```

`Branch` – variable corresponding to the branch which the capacitor is added to, `name` – unique circuit component name, `value` – capacitance value in Farads, `Uinit` – initial voltage on the capacitor.
Example of adding capacitor C1 of 100 μF to the branch defined by variable `Branch`:

```
Branch = CircuitAddC(Branch,'C1',100*10^-6);
```

Since in this case the input variable `Uinit` is not used the initial voltage is zero.
10.1.2.4. Each stator winding coil or a group of coils can be treated as an independent circuit component consisting of an active resistance, stator end winding inductance and conductors within stator slots designated as a spiral with a dot at the input terminal:



A coil or a group of coils is herein referred to as a coil object or a coil component. Each coil object combines all stator slots or stator slot layers associated with the same phase and the same parallel path.

To add a coil object to the branch, the following function is used:

```
Branch = CircuitAddCoil(Branch,name,phase,Subdomains,R,Lsew,...
         npath,Npp,coilorientation,p,t,l,nper);
```

`Branch` – variable corresponding to the branch which the coil object is added to, `name` – unique circuit component name, `phase` – phase accepts one of the following values: `'a'`, `'b'`, `'c'`; `Subdomains` – array of subdomains; `R` and `Lsew` – active resistance and end winding inductance of the coil component, respectively, `npath` – parallel path number (`npath`=1 if there are no parallel paths), `Npp` – number of parallel paths of the stator winding per phase; `coilorientation = 1` if the coil component is oriented concordantly with the branch which the coil component is added to, otherwise `coilorientation = -1`. The branch orientation is defined by `node1` and `node2` arguments of `CircuitAddBranch` function and the coil component orientation is defined by a dot at the input terminal. Input arguments `p`, `t`, `l`, `nper` are the same as described in section 10.1.

Note that in order to prevent incorrect results **each branch should include only one coil component**, otherwise coil components must be divided by additional nodes. For example if you have two coils connected in series you should put additional note between coils so each coil has its own branch object.

Example of adding the coil component to the branch defined by variable `Branch`:

```
Branch = CircuitAddCoil(Branch,'coil_c2','c',Subdomains,...
                Rs,Lsew,2,Npp,1,p,t,l,nper);
```

In this case the coil component corresponds to the second group (parallel path 2) of coils of phase “c” named as coil_c2, oriented concordantly with the branch, with the active resistance `Rs` and end winding inductance `Lsew`.

10.1.2.5. To add a diode to the branch, the following function is used:

```
Branch = CircuitAddDiode(Branch,name,Roff,Ron,direction);
```

`Branch` – variable corresponding to the branch which the diode is added to, `name` – unique circuit component name, `Roff` and `Ron` – backward and forward resistance of the diode in Ohm; `direction = 1` if the diode forward direction is oriented from `node1` to `node2` of the branch (see description of `CircuitCreateBranch` in section 10.1.1), otherwise `direction = -1`.

Note that backward and forward resistances should be as follows:

$0 < \text{Roff} < \infty$

$0 < \text{Ron} < \infty$

Example of adding diode D1 to the branch defined by variable `Branch`:

```
Branch = CircuitAddDiode(Branch,'D1',10^8,10^-6,1);
```

10.1.2.6. To add a voltage source to the branch, the following function is used:

```
Branch = CircuitAddE(Branch,name,data);
```

`Branch` – variable corresponding to the branch which the voltage source is added to, `name` – unique circuit component name, `data` – field of the `CircuitControl` structure used by a simulation script function to control the voltage source (see section 10.2).

10.1.2.7. To add a current source to the branch, the following function is used:

```
Branch = CircuitAddJ(Branch,name,data);
```

`Branch` – variable corresponding to the branch which the current source is added to, `name` – unique circuit component name, `data` – field of the `CircuitControl` structure used by a simulation script function to control the current source (see section 10.2).

10.1.2.8. To add a switch to the branch, the following function is used:

```
Branch = CircuitAddSwitch(Branch,name,data,Roff,Ron);
```

`Branch` – variable corresponding to the branch which the switch is added to, `name` – unique circuit component name, `data` – field of the `CircuitControl` structure used by a simulation script function to control the switch state (see section 10.2), `Roff` and `Ron` – 'off' and 'on' resistance of the switch in Ohm.

Note that 'off' and 'on' resistances should be as follows:

$0 < \text{Roff} < \infty$

$0 < \text{Ron} < \infty$

10.2. Controlling power sources and electronic switches using simulation script functions.

When the Dynamic FE Analysis is in progress, the voltage and current values supplied by the voltage and current sources as well as the state of each electronic switch can be specified at each time step by the simulation script function. For this purpose the `CircuitControl` structure, output argument of the simulation script function, is used. Each voltage or current source and switch is associated with its field of the `CircuitControl` structure through the input argument `data`, when the `CircuitAddE`, `CircuitAddJ` or `CircuitAddSwitch` function is called. For the power sources, the value assigned to the corresponding field of the `CircuitControl` structure at the specified time

step is the voltage or current output value of the voltage or current source associated with this field. Similarly, the state of the switch is determined by the value assigned to the corresponding field of the `CircuitControl` structure at the specified time step: 1 for state ‘on’, 0 – for state ’off’ (see section 10.3 for more details on using electronic switches).

The following example provides an explanation for the power source control principle. Assume that the voltage source named V1 was added to the stator circuit calling the function

```
Branch = CircuitAddE(Branch,'V1','CircuitControl.v1');
```

In this case the voltage source V1 is associated with the field `v1` of the `CircuitControl` structure. The following piece of code can be used in the simulation script function to specify the voltage value supplied by the voltage source V1 on a given simulation time step:

```
CurrentTime = Outputs.CurrentTime;
CircuitControl.v1 = 220*sqrt(2)*sin(2*pi*50*CurrentTime);
```

In this case we have the source of sinusoidal voltage with frequency 50Hz and RMS value 220V. Note that power sources can have any desired voltage or current waveform.

Similarly, for the switch S1 created by calling the following function:

```
Branch = CircuitAddSwitch(Branch,’S1’,'CircuitControl.s1',Roff,Ron);
```

Writing simulation script function you can use the following command to set the switch S1 to ‘on’ state:

```
CircuitControl.s1 = 1;
```

The following command will set the switch S1 to ‘off’ state:

```
CircuitControl.s1 = 0;
```

10.3. Default three-phase inverter circuit function.

As mentioned before the three-phase inverter circuit function `InverterCircuit` is used by default for the Dynamic FE Simulation. The source code of the function can be found in file InverterCircuit.m. Example of the electrical circuit with the star-connected stator winding and two parallel paths is shown in Figure 10.1. The electrical circuit is modified automatically depending on the winding connection and number of parallel paths. The following piece of code constructs the inverter circuit:

```
Schematic = CircuitCreateSchematic();
Branch = CircuitCreateBranch(0,1);
Branch = CircuitAddE(Branch,'Vdc','CircuitControl.vdc');
Schematic = CircuitAddBranch(Schematic,Branch);
```

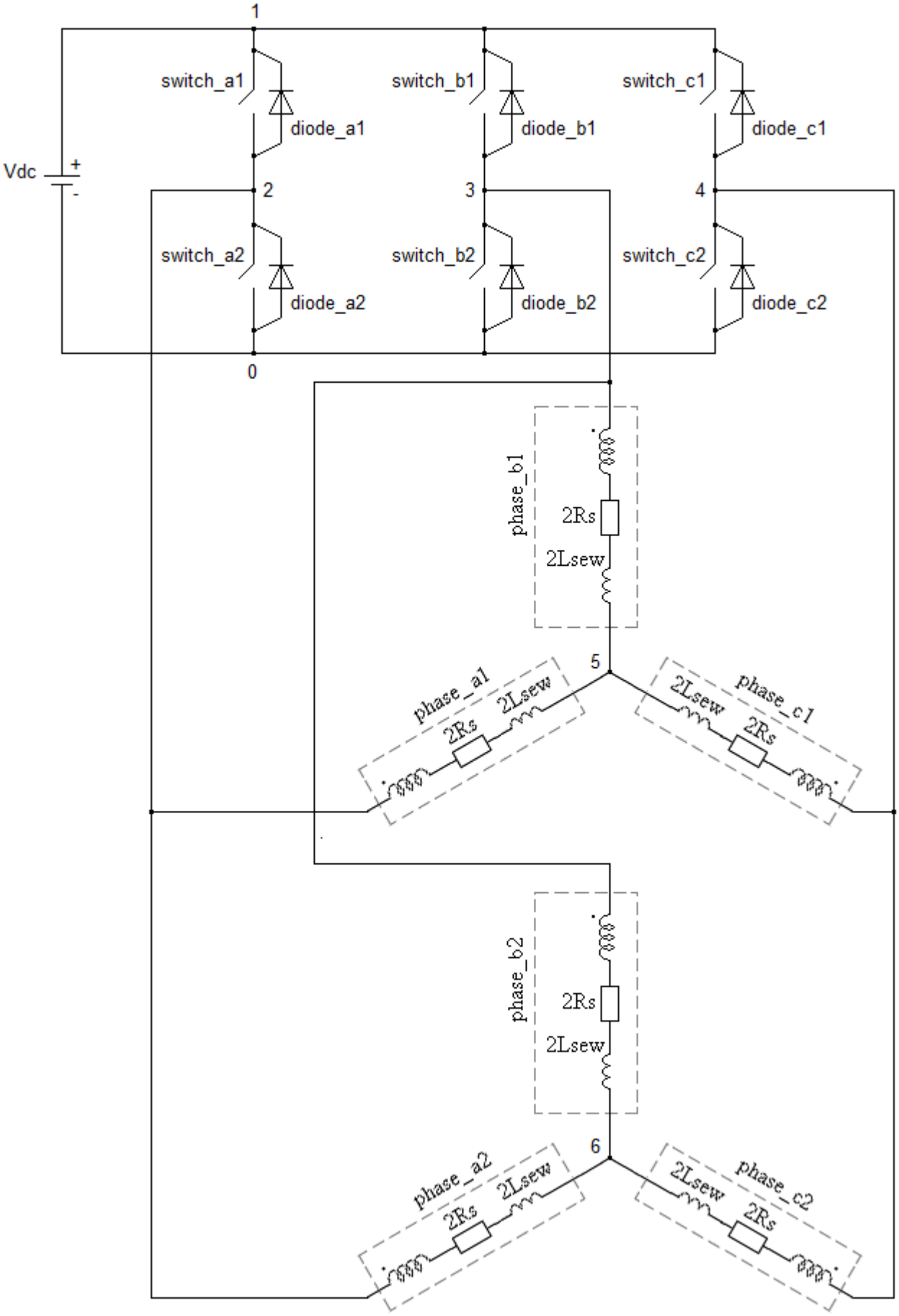


Figure 10.1. Three-phase inverter electrical circuit with star-connected winding and two parallel paths.

```
Branch = CircuitCreateBranch(2,1);
Branch = CircuitAddSwitch(Branch,'switch_a1','CircuitControl.switch_a1',Roff,Ron);
Schematic = CircuitAddBranch(Schematic,Branch);
Branch = CircuitCreateBranch(2,1);
Branch = CircuitAddDiode(Branch,'diode_a1',Roff,Ron,1);
Schematic = CircuitAddBranch(Schematic,Branch);

Branch = CircuitCreateBranch(3,1);
Branch = CircuitAddSwitch(Branch,'switch_b1','CircuitControl.switch_b1',Roff,Ron);
Schematic = CircuitAddBranch(Schematic,Branch);
Branch = CircuitCreateBranch(3,1);
Branch = CircuitAddDiode(Branch,'diode_b1',Roff,Ron,1);
Schematic = CircuitAddBranch(Schematic,Branch);

Branch = CircuitCreateBranch(4,1);
Branch = CircuitAddSwitch(Branch,'switch_c1','CircuitControl.switch_c1',Roff,Ron);
Schematic = CircuitAddBranch(Schematic,Branch);
Branch = CircuitCreateBranch(4,1);
Branch = CircuitAddDiode(Branch,'diode_c1',Roff,Ron,1);
Schematic = CircuitAddBranch(Schematic,Branch);

Branch = CircuitCreateBranch(0,2);
Branch = CircuitAddSwitch(Branch,'switch_a2','CircuitControl.switch_a2',Roff,Ron);
Schematic = CircuitAddBranch(Schematic,Branch);
Branch = CircuitCreateBranch(0,2);
Branch = CircuitAddDiode(Branch,'diode_a2',Roff,Ron,1);
Schematic = CircuitAddBranch(Schematic,Branch);

Branch = CircuitCreateBranch(0,3);
Branch = CircuitAddSwitch(Branch,'switch_b2','CircuitControl.switch_b2',Roff,Ron);
Schematic = CircuitAddBranch(Schematic,Branch);
Branch = CircuitCreateBranch(0,3);
Branch = CircuitAddDiode(Branch,'diode_b2',Roff,Ron,1);
Schematic = CircuitAddBranch(Schematic,Branch);

Branch = CircuitCreateBranch(0,4);
Branch = CircuitAddSwitch(Branch,'switch_c2','CircuitControl.switch_c2',Roff,Ron);
Schematic = CircuitAddBranch(Schematic,Branch);
Branch = CircuitCreateBranch(0,4);
Branch = CircuitAddDiode(Branch,'diode_c2',Roff,Ron,1);
Schematic = CircuitAddBranch(Schematic,Branch);
```

Some part of the code connecting stator winding to the inverter is not shown.

As described in the previous section the state of each switch is controlled by the corresponding field of the `CircuitControl` structure. There are three default simulation script functions used together with the `InverterCircuit` function: `simscript_hystpwm`, `simscript_spacevecpwm` and `simscript_sixstep`. The following piece of code is used to control switch states in the `simscript_spacevecpwm` function:

```
[sA,sB,sC] = spacevecpwm(Vai_ref,Vbi_ref,Vci_ref,fspwm,CurrentTime,Vdc);
if sA==1
    CircuitControl.switch_a1=1;
    CircuitControl.switch_a2=0;
else  % sA==-1
    CircuitControl.switch_a1=0;
    CircuitControl.switch_a2=1;
end
```

```
if sB==1
    CircuitControl.switch_b1=1;
    CircuitControl.switch_b2=0;
else  % sB==-1
    CircuitControl.switch_b1=0;
    CircuitControl.switch_b2=1;
end
if sC==1
    CircuitControl.switch_c1=1;
    CircuitControl.switch_c2=0;
else  % sC==-1
    CircuitControl.switch_c1=0;
    CircuitControl.switch_c2=1;
end
```

Use the source code of InverterCircuit.m, simscript_hystpwm.m, simscript_spacevecpwm.m and simscript_sixstep.m as an example to write your own electrical circuit functions and simulation scripts.

11. MOTORANALYSIS-PM SETTINGS

MotorAnalysis-PM settings are available from menu **File -> Settings** and shown in Figure 11.1.

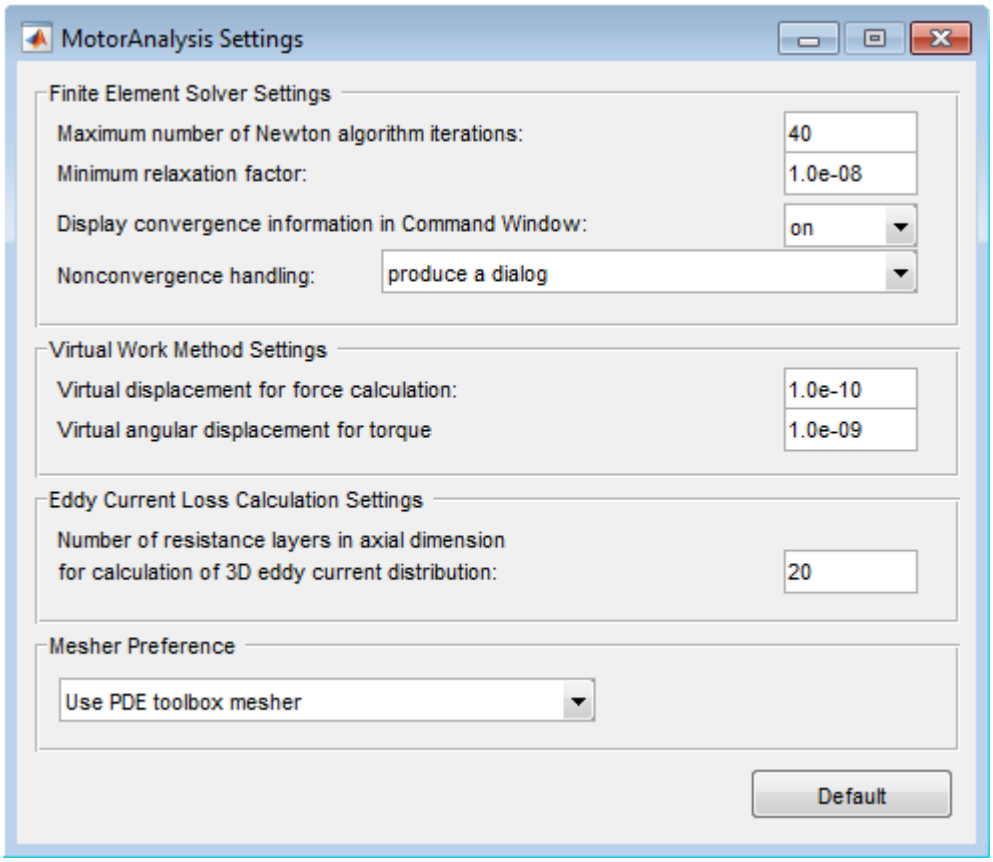


Figure 11.1. MotorAnalysis-PM settings.

Finite Element Solver Settings.

Maximum number of Newton algorithm iterations – if the number of Newton algorithm iterations exceeds the value specified by this field it is said that the solution does not converge. The subsequent action will depend on the **Nonconvergence handling** field value.

Minimum relaxation factor – if the solution does not converge on the current Newton algorithm iteration, the nonlinear solver damps down the search step with relaxation factor $0 < \alpha < 1$. The relaxation factor iteratively decreases until the convergence occurs. The smaller relaxation factor, the worse convergence. If the relaxation factor becomes less than **Minimum relaxation factor**, it is said that the solution does not converge. The subsequent action will depend on the **Nonconvergence handling** field value.

Display convergence information in Command Window – determines either the convergence information (i.e. Newton algorithm iteration number, the maximum and average residual and the relaxation factor) is displayed in the Command Window.

Nonconvergence handling – determines an action the program takes when the solution does not converge. If ***ignore nonconvergence and continue simulation*** is chosen, the simulation is continued with the accuracy reached regardless the value specified in the **Convergence tolerance** field of the main window; if ***produce a dialog*** is chosen, the dialog with the convergence information is provided to allow the user to decide either to continue or interrupt the simulation; if i***nterrupt simulation and report an error*** is chosen, the simulation is interrupted with an error report.

Virtual Work Method Settings.

Fields of this panel define the virtual displacement for torque and force calculation with the virtual work method as discussed in section 2.3.

Eddy Current Loss Calculation Settings.

This panel influences the accuracy of the magnet loss calculation. By default, number of the resistance layers is 20.

Mesher Preference.

This parameter is only valid for MATLAB version of MotorAnalysis-PM. By default, PDE toolbox mesher is used. If PDE toolbox is not installed, the internal mesher will be used instead. In some cases depending on the MATLAB version, PDE toolbox mesher can cause an error so the internal mesher should be used instead.

APPENDIX

Appendix A. Power balance, discretization error and accuracy of the results.

If the power balance is satisfied it means that the input apparent power delivered by all current and voltage sources equals to the consumed apparent power:

$$P_{input} = P_{cons}$$

The input power is calculated from voltages and currents of all power sources:

$$P_{input} = \sum_{n=1}^{m} i_n u_n$$

The consumed power is defined as follows:

$$P_{cons} = P_s + P_{mech} + \frac{\Delta W_{mf}}{\Delta t},$$

where P_s – apparent power (real and reactive) of the stator electrical circuit, P_{mech} – mechanical power on the shaft, $\Delta W_{mf}/\Delta t$ – magnetic field energy time derivative.

The discretization error of the simulation is a difference between the input power and consumed power in percentage terms. The discretization error is mostly influenced by the simulation time step and can be used to assess the accuracy of the simulation results.

Appendix B. Out of memory errors handling.

"Out of memory" errors can occur when using the mesh of too large size. According to the MATLAB help documentation, to avoid "out of memory" errors when using MotorAnalysis-PM, you can increase the size of the swap file or add more memory to the system. Restarting MATLAB when "out of memory" error occurs may also help in some cases.